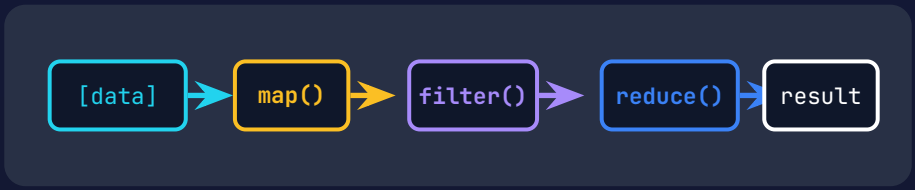
JAVASCRIPT

Higher-Order Functions

A Comprehensive Guide for Web Developers



Anass Kabil

v1.0

https://github.com/anassk01/higher-order-functions-js-book

January 19, 2026

Contents

1 What Are Higher-Order Functions? . . . 3
1.1 The Big Idea . . . 3
1.2 Functions as Values . . . 3
1.2.1 Arrow Functions: The Modern Syntax . . . 4
1.3 Functions That Take Functions . . . 4
1.3.1 The Callback Pattern . . . 4
1.3.2 Real Web Dev Example: Event Handlers . . . 5
1.4 Functions That Return Functions . . . 5
1.4.1 The Factory Pattern . . . 5
1.4.2 With Arrow Functions . . . 6
1.4.3 Real Web Dev Example: Configured Functions . . . 6
1.5 Both Patterns Combined . . . 7
1.6 Mental Model: Functions as Values . . . 7
1.7 Why Higher-Order Functions Matter in Web Dev . . . 8
1.7.1 Problem: Repetitive Patterns . . . 8
1.7.2 Solution: Abstract the Pattern . . . 8
1.8 The Three Questions . . . 8
1.9 Practice Exercises . . . 9
1.10 Chapter Summary . . . 12

2 The Core Three — map, filter, reduce . . . 13
2.1 map: Transform Every Element . . . 13
2.1.1 Basic Examples . . . 13
2.1.2 The Transformation Function . . . 13
2.1.3 Web Dev Example: Processing API Data . . . 14
2.1.4 Web Dev Example: Rendering Lists . . . 14
2.2 filter: Select Elements That Pass a Test . . . 15
2.2.1 Basic Examples . . . 15
2.2.2 The Predicate Function . . . 15
2.2.3 Web Dev Example: Filtering Products . . . 16
2.2.4 Web Dev Example: Form Validation . . . 16
2.2.5 Filtering Out Falsy Values . . . 17
2.3 reduce: Accumulate to a Single Value . . . 17
2.3.1 Basic Example: Sum . . . 17
2.3.2 Common Reductions . . . 18
2.3.3 reduce to Object . . . 19
2.3.4 reduce to Count Occurrences . . . 19
2.3.5 reduce to Group By . . . 19
2.3.6 Web Dev Example: Shopping Cart Total . . . 20
2.3.7 Web Dev Example: Flatten Query Parameters . . . 20
2.4 Method Chaining: Combining map, filter, reduce . . . 20
2.4.1 Reading Chains: Top to Bottom . . . 21
2.4.2 Web Dev Example: API Response Processing . . . 21
2.5 Performance Consideration . . . 21
2.6 When to Use Each . . . 22
2.7 Practice Exercises . . . 22
2.8 Chapter Summary . . . 24

3 Beyond the Core — find, some, every, flatMap **25**
3.1 find: Get the First Match 25
3.1.1 Basic Example 25
3.1.2 find vs filter 25
3.1.3 Performance: find Stops Early 26
3.1.4 Web Dev Example: Find User by ID 26
3.1.5 Handling undefined 27
3.1.6 findIndex: Get Position Instead of Element 27
3.2 some: Does Any Element Pass? 27
3.2.1 Basic Examples 28
3.2.2 some Stops Early 28
3.2.3 Web Dev Example: Permission Check 28
3.2.4 Web Dev Example: Form Has Errors 28
3.3 every: Do All Elements Pass? 29
3.3.1 Basic Examples 29
3.3.2 every Stops Early (On First Failure) 29
3.3.3 Web Dev Example: Form Validation 29
3.3.4 Web Dev Example: All Items Selected 30
3.4 some vs every: Quick Reference 30
3.5 flatMap: Map and Flatten in One Step 30
3.5.1 The Problem flatMap Solves 30
3.5.2 When Your Transform Returns an Array 31
3.5.3 flatMap for Conditional Inclusion 31
3.5.4 Web Dev Example: Nested Data Extraction 32
3.5.5 Web Dev Example: Processing Nested API Response 32
3.6 Combining Everything: Real Patterns 32
3.6.1 Pattern: Find and Transform 32
3.6.2 Pattern: Validate Then Process 33
3.6.3 Pattern: Extract from Nested Structure 33
3.7 Decision Guide: Which Method to Use? 34
3.8 Practice Exercises 34
3.9 Chapter Summary 36

Part I Summary **37**

4 Function Factories & Closures **43**
4.1 The Factory Pattern 43
4.1.1 Why This Matters 43
4.2 Understanding Closures 44
4.2.1 The Mechanism 44
4.2.2 Visualizing Closure 45
4.2.3 Each Call Creates a New Closure 45
4.3 Private State with Closures 45
4.4 Practical Factory Patterns 46
4.4.1 Pattern 1: Configured API Fetchers 46
4.4.2 Pattern 2: Event Handler Factories 47
4.4.3 Pattern 3: Validator Factories 48
4.4.4 Pattern 4: Memoization Factory 48
4.4.5 Pattern 5: Rate Limiter Factory 49
4.5 Closure Memory Considerations 49
4.5.1 The Problem: Unintended Retention 50
4.5.2 The Fix: Minimize Closure Scope 50
4.5.3 The Shared Context Trap 50
4.5.4 Best Practice: Factory Function Hygiene 51
4.6 The Module Pattern 51

4.7 Common Closure Pitfalls . . . 52
4.7.1 Pitfall 1: Loop Variables . . . 52
4.7.2 Pitfall 2: this Binding in Closures . . . 53
4.7.3 Pitfall 3: Stale Closures (React) . . . 54
4.8 Chapter Summary . . . 54
4.9 Practice Exercises . . . 55

5 Composition & Pipelines . . . 58
5.1 The Composition Concept . . . 58
5.1.1 Mathematical Composition . . . 58
5.1.2 In JavaScript . . . 58
5.1.3 The Problem with Nesting . . . 58
5.2 compose: Right-to-Left Composition . . . 58
5.2.1 How compose Works . . . 59
5.3 pipe: Left-to-Right Composition . . . 59
5.3.1 pipe is the Web Dev Standard . . . 60
5.4 Building Pipelines . . . 60
5.4.1 Data Transformation Pipeline . . . 60
5.4.2 Request Processing Pipeline . . . 61
5.4.3 Array Processing Pipeline . . . 61
5.5 The tap Utility: Debugging Pipelines . . . 62
5.5.1 tap Variations . . . 62
5.6 Point-Free Style . . . 63
5.6.1 Point-Free with Higher-Order Utilities . . . 63
5.6.2 When to Use Point-Free . . . 63
5.7 Composing with Multiple Arguments . . . 64
5.8 Real-World Composition Patterns . . . 64
5.8.1 Pattern 1: Validation Pipeline . . . 64
5.8.2 Pattern 2: Async Pipeline . . . 65
5.9 Chapter Summary . . . 66
5.10 Practice Exercises . . . 66

6 Partial Application & Currying . . . 68
6.1 The Problem: Repeated Arguments . . . 68
6.2 Partial Application . . . 68
6.2.1 Using bind for Partial Application . . . 68
6.2.2 A Generic partial Function . . . 69
6.2.3 Web Dev Example: API Client . . . 69
6.3 Currying . . . 70
6.3.1 Arrow Function Syntax . . . 70
6.3.2 The Difference: Partial vs Curry . . . 70
6.4 Auto-Curry: Flexible Currying . . . 71
6.4.1 How Auto-Curry Works . . . 71
6.5 Currying in Practice . . . 71
6.5.1 Data-Last for Piping . . . 72
6.5.2 Web Dev Example: Validation . . . 72
6.6 Partial Application vs Currying: When to Use Each . . . 73
6.6.1 Use Partial Application When: . . . 73
6.6.2 Use Currying When: . . . 73
6.7 Right-to-Left Partial Application . . . 74
6.8 Chapter Summary . . . 74
6.9 Practice Exercises . . . 74

7 The Pipeline Injection Protocol . . . 76
7.1 The 4-Phase Protocol . . . 76
7.2 Phase 1: Isolate State . . . 76
7.2.1 Pattern Recognition . . . 76
7.2.2 Example . . . 77
7.2.3 State Types and Their HOF Equivalents . . . 77
7.3 Phase 2: Extract Predicates . . . 77
7.3.1 Pattern Recognition . . . 77
7.3.2 Example Continued . . . 78
7.3.3 Combining Predicates . . . 78
7.3.4 Common Predicate Patterns . . . 78
7.4 Phase 3: Decouple Transformations . . . 78
7.4.1 Pattern Recognition . . . 78
7.4.2 Example Continued . . . 79
7.4.3 Common Transformation Patterns . . . 79
7.5 Phase 4: Compose Pipeline . . . 79
7.5.1 The Mapping . . . 80
7.5.2 Example Completed . . . 80
7.6 Complete Worked Examples . . . 80
7.6.1 Example 1: Sum of Squares of Evens . . . 81
7.6.2 Example 2: Find First Admin . . . 81
7.6.3 Example 3: Validate All Fields . . . 82
7.6.4 Example 4: Group By Category . . . 82
7.6.5 Example 5: Nested Loop Flattening . . . 83
7.7 Decision Flowchart . . . 84
7.8 Anti-Patterns to Avoid . . . 84
7.8.1 Anti-Pattern 1: Reduce-Spread ($O(n^2)$) . . . 84
7.8.2 Anti-Pattern 2: Filter Then Length . . . 85
7.8.3 Anti-Pattern 3: Map for Side Effects . . . 85
7.9 When NOT to Convert . . . 85
7.9.1 1. Performance-Critical Code . . . 85
7.9.2 2. Complex Control Flow . . . 86
7.9.3 3. Early Exit on Find . . . 86
7.9.4 4. Mutation is Intentional . . . 86
7.10 Chapter Summary . . . 86
7.10.1 The 4-Phase Protocol . . . 87
7.10.2 Quick Reference . . . 87
7.11 Practice Exercises . . . 87

Part II Summary . . . 89

8 HOFs in Async JavaScript . . . 93
8.1 Promises Are Built on HOFs . . . 93
8.1.1 The Promise Chain as a Pipeline . . . 93
8.1.2 Passing Functions Directly . . . 93
8.2 Promise.all with map . . . 94
8.2.1 Step by Step . . . 94
8.2.2 Real-World Example: Enriching Data . . . 95
8.2.3 Handling Partial Failures . . . 95
8.3 Sequential vs Parallel Execution . . . 95
8.3.1 Parallel: Promise.all + map . . . 95
8.3.2 Sequential: for...of with await . . . 96
8.3.3 Sequential: reduce with async . . . 96
8.3.4 When to Use Each . . . 96
8.4 Controlled Concurrency . . . 96

8.5 Async Higher-Order Functions . . . 97
8.5.1 Async Map . . . 97
8.5.2 Async Filter . . . 97
8.5.3 Async Find . . . 98
8.5.4 Async Reduce . . . 98
8.6 Retry Logic as HOF . . . 99
8.6.1 Exponential Backoff . . . 99
8.6.2 Conditional Retry . . . 100
8.7 Debounce and Throttle . . . 100
8.7.1 Debounce: Wait for Pause . . . 100
8.7.2 Debounce with Promise . . . 101
8.7.3 Throttle: Rate Limit . . . 102
8.8 Timeout Wrapper . . . 102
8.8.1 With AbortController . . . 103
8.9 Async Pipeline . . . 103
8.9.1 With Error Handling . . . 104
8.10 Event Handling Patterns . . . 104
8.10.1 Once: Execute Only First Time . . . 104
8.10.2 After: Execute After N Calls . . . 105
8.10.3 Before: Execute Only First N Times . . . 105
8.11 Practice Exercises . . . 106
8.12 Chapter Summary . . . 107

9 HOFs in React . . . 108
9.1 React Hooks Are HOF-Based . . . 108
9.2 useCallback: Memoizing Functions . . . 108
9.2.1 The Problem . . . 108
9.2.2 The Solution . . . 109
9.2.3 When Dependencies Change . . . 109
9.3 useMemo: Memoizing Values . . . 110
9.3.1 useMemo vs useCallback . . . 110
9.3.2 When to Use useMemo . . . 110
9.4 The Stale Closure Problem . . . 111
9.4.1 The Problem . . . 111
9.4.2 Fix 1: Functional Updates . . . 112
9.4.3 Fix 2: Include in Dependencies . . . 112
9.4.4 Fix 3: useRef for Mutable Reference . . . 113
9.5 Custom Hooks as HOF Patterns . . . 113
9.5.1 useToggle . . . 113
9.5.2 useDebounce . . . 114
9.5.3 useAsync . . . 114
9.6 Higher-Order Components (HOCs) . . . 115
9.6.1 HOC with Configuration . . . 115
9.6.2 Composing Multiple HOCs . . . 116
9.7 Practice Exercises . . . 116
9.8 Chapter Summary . . . 117

10 HOFs in State Management . . . 119
10.1 Redux: HOFs Everywhere . . . 119
10.2 The Reducer Pattern . . . 119
10.2.1 Higher-Order Reducers . . . 120
10.2.2 Resettable Reducer HOF . . . 121
10.3 Redux Middleware . . . 121
10.3.1 Logger Middleware . . . 121

10.3.2 Thunk Middleware . . . 122
10.3.3 Analytics Middleware . . . 122
10.4 Selectors as HOFs . . . 123
10.4.1 Memoized Selectors with Reselect . . . 123
10.5 Practice Exercises . . . 124
10.6 Chapter Summary . . . 124

11 HOFs in Node.js & APIs . . . 126
11.1 Express Middleware Pattern . . . 126
11.1.1 Creating Middleware Factories . . . 126
11.1.2 Authentication Middleware Factory . . . 127
11.1.3 Role-Based Authorization . . . 127
11.2 Middleware Composition . . . 128
11.2.1 Compose Multiple Middleware . . . 128
11.2.2 Conditional Middleware . . . 128
11.2.3 Async Middleware Wrapper . . . 129
11.3 Request Validation . . . 129
11.3.1 Validation Middleware Factory . . . 129
11.4 Error Handling Patterns . . . 130
11.4.1 Error Handler Factory . . . 130
11.5 Practice Exercises . . . 130
11.6 Chapter Summary . . . 131

Part III Summary . . . 133

12 Performance & Optimization . . . 137
12.1 Understanding the Cost of HOFs . . . 137
12.1.1 Measuring Performance . . . 137
12.1.2 The Reality: It Usually Doesn't Matter . . . 138
12.2 The Chain Problem . . . 138
12.2.1 Measuring Chain Overhead . . . 138
12.3 Transducers: Single-Pass Processing . . . 139
12.3.1 The Concept . . . 139
12.3.2 Building Transducers . . . 140
12.3.3 Using Transducers . . . 140
12.3.4 Performance Comparison . . . 141
12.4 Lazy Evaluation with Generators . . . 141
12.4.1 Using Lazy Pipelines . . . 142
12.4.2 Lazy Class Implementation . . . 142
12.5 Iterator Helpers (ES2025) . . . 144
12.5.1 Available Iterator Helpers . . . 145
12.6 Memoization Strategies . . . 145
12.6.1 Basic Memoization . . . 145
12.6.2 Memoization with LRU Cache . . . 146
12.6.3 Memoization with TTL . . . 146
12.7 V8 Optimization Insights . . . 147
12.7.1 Monomorphism: Keep Types Consistent . . . 147
12.7.2 Hidden Classes: Consistent Object Shapes . . . 147
12.8 Performance Decision Matrix . . . 148
12.8.1 Quick Optimization Checklist . . . 148
12.9 Practice Exercises . . . 149
12.10 Chapter Summary . . . 149

13 Functional Error Handling **151**
13.1 The Problem with Exceptions 151
13.1.1 Exceptions Are Side Effects 151
13.2 The Either Pattern 152
13.2.1 Using Either 152
13.3 Railway Oriented Programming 153
13.4 Collecting All Errors 154
13.5 Maybe/Option Pattern 155
13.5.1 Safe Property Access 155
13.6 Result Type (Rust-Inspired) 156
13.7 Async Either (TaskEither) 157
13.8 Practice Exercises 158
13.9 Chapter Summary 159

14 Working with Complex Data **160**
14.1 The Problem with Deep Updates 160
14.2 What is a Lens? 160
14.2.1 Using Lenses 161
14.2.2 Composing Lenses 161
14.3 Lens Utilities 162
14.4 Prisms: For Optional/Union Types 162
14.5 Traversals: For Multiple Focuses 163
14.6 Practical Lens Patterns 163
14.6.1 Pattern 1: Redux-Style State Updates 164
14.6.2 Pattern 2: Form State Management 164
14.7 Immer as a Lens Alternative 164
14.7.1 When to Use Lenses vs Immer 165
14.8 Practice Exercises 165
14.9 Chapter Summary 166

15 Metaprogramming with Proxies + HOFs **167**
15.1 Proxy Fundamentals 167
15.2 HOF-Based Proxy Factories 167
15.2.1 Observable Objects 167
15.2.2 Deep Observable 168
15.3 Validation Proxy 169
15.4 Lazy Initialization 170
15.5 Method Chaining Proxy 171
15.6 Negative Array Indices 172
15.7 Immutable Proxy 173
15.8 Method Interception (AOP) 173
15.9 Virtual Properties 175
15.10 Fluent API Builder 176
15.11 Practice Exercises 176
15.12 Chapter Summary 177

Part IV Summary **179**

16 Debugging & Testing Higher-Order Functions **183**
16.1 The Challenge of Debugging HOFs 183
16.2 The tap Pattern: Non-Invasive Debugging 183
16.2.1 Tap Variations 184
16.2.2 Pipeline Tap for Arrays 184

16.3 Tracing Function Calls . . . 185
16.3.1 Wrapper for Tracing . . . 185
16.3.2 Tracing with Timing . . . 185
16.3.3 Accumulating Trace . . . 186
16.4 Debugging Closures . . . 187
16.4.1 Inspecting Closed-Over Variables . . . 187
16.4.2 The Closure Snapshot Pattern . . . 187
16.4.3 Detecting Stale Closures . . . 188
16.5 Debugging Async HOFs . . . 188
16.5.1 Tracing Async Operations . . . 189
16.5.2 Debugging Promise Chains . . . 189
16.5.3 Async Pipeline Debugger . . . 189
16.6 Testing Higher-Order Functions . . . 190
16.6.1 Testing Function Factories . . . 190
16.6.2 Testing Closures . . . 191
16.6.3 Testing Wrapped Functions . . . 192
16.6.4 Testing Debounce/Throttle . . . 192
16.6.5 Testing Compose/Pipe . . . 193
16.7 Testing Strategies for HOFs . . . 194
16.7.1 Strategy 1: Test the Factory and Product Separately . . . 194
16.7.2 Strategy 2: Test Composition Behavior . . . 195
16.7.3 Strategy 3: Property-Based Testing . . . 196
16.8 Common HOF Bugs and Fixes . . . 197
16.8.1 Bug 1: Mutating in map/filter . . . 197
16.8.2 Bug 2: Forgetting Return in reduce . . . 197
16.8.3 Bug 3: Wrong reduce Initial Value . . . 198
16.8.4 Bug 4: Async in forEach . . . 198
16.8.5 Bug 5: Filter with Async Predicate . . . 198
16.9 Debug Utilities Library . . . 199
16.10 Chapter 16 Practice Exercises . . . 201
16.11 Chapter 16 Summary . . . 203

17 Quick Reference & Decision Guides . . . 205
17.1 Method Quick Reference . . . 205
17.1.1 Array Methods . . . 205
17.1.2 Early Exit Methods . . . 205
17.2 Decision Trees . . . 205
17.2.1 Which Array Method? . . . 206
17.2.2 Which Composition Pattern? . . . 206
17.2.3 Which Wrapper Pattern? . . . 206
17.3 Pattern Quick Reference . . . 206
17.3.1 Factory Pattern . . . 206
17.3.2 Memoization . . . 207
17.3.3 Debounce . . . 207
17.3.4 Throttle . . . 207
17.3.5 Pipe . . . 207
17.3.6 Compose . . . 207
17.3.7 Curry . . . 208
17.3.8 Partial Application . . . 208
17.4 Common Transformations . . . 208
17.4.1 Array to Object (by key) . . . 208
17.4.2 Group By . . . 208
17.4.3 Count Occurrences . . . 208
17.4.4 Unique Values . . . 209
17.4.5 Flatten . . . 209

17.4.6 Partition . . . 209
17.4.7 Zip . . . 209
17.4.8 Chunk . . . 209
17.5 Performance Quick Guide . . . 209
17.5.1 Optimization Checklist . . . 210
17.6 Functional Error Handling . . . 210
17.6.1 Either Pattern . . . 210
17.6.2 Maybe Pattern . . . 211
17.7 Lens Quick Reference . . . 211
17.8 Async Patterns . . . 212
17.8.1 Sequential Processing . . . 212
17.8.2 Parallel Processing . . . 212
17.8.3 Async Pipe . . . 213
17.9 React HOF Patterns . . . 213
17.9.1 Higher-Order Components (HOC) . . . 213
17.9.2 Custom Hooks with HOFs . . . 213
17.9.3 Event Handler Factories . . . 214
17.10 Common Mistakes Quick Reference . . . 214
17.11 Library Quick Reference . . . 214
17.12 ES2025+ Features . . . 214
17.12.1 Iterator Helpers (Lazy) . . . 214
17.12.2 Array.fromAsync . . . 216
17.12.3 Set Methods . . . 216
17.12.4 Object.groupBy . . . 216
17.13 Chapter 17 Summary . . . 216

Part V Summary: Mastery Achieved . . . 217
Final Practice Project . . . 217

Part I

Foundations

Chapter 1

What Are Higher-Order Functions?

> *"Functions are first-class citizens in JavaScript—they can go anywhere any other value can go."*
> — Douglas Crockford

1.1 The Big Idea

In JavaScript, functions are **first-class citizens**. This means functions can be:

- Stored in variables
- Passed as arguments to other functions
- Returned from other functions
- Stored in data structures

Definition: Higher-Order Function

A **Higher-Order Function (HOF)** is a function that does at least one of these:

1. Takes one or more functions as arguments
2. Returns a function as its result

That's it. No magic, no complex theory. Just functions working with functions.

1.2 Functions as Values

Before we dive into HOFs, let's cement the foundation: functions are values, just like numbers or strings.

```
// A number stored in a variable
const age = 25;

// A string stored in a variable
const name = "Alice";

// A function stored in a variable
const greet = function(person) {
    return `Hello, ${person}!`;
};

// All three are just values
console.log(age);         // 25
console.log(name);        // Alice
```

```
console.log(greet);       // [Function: greet]
console.log(greet(name)); // Hello, Alice!
```

Key Insight

The function `greet` is not special—it's a value that happens to be callable.

1.2.1 Arrow Functions: The Modern Syntax

```
// Traditional function expression
const add = function(a, b) {
    return a + b;
};

// Arrow function (equivalent)
const addArrow = (a, b) => a + b;

// Single parameter: parentheses optional
const double = x => x * 2;

// No parameters: empty parentheses required
const sayHi = () => "Hi!";
```

Arrow functions are not just shorter syntax—they also don't have their own `this` binding, which matters in web development (we'll cover this in Chapter 9).

1.3 Functions That Take Functions

The most common HOF pattern: passing a function as an argument.

1.3.1 The Callback Pattern

```
// A higher-order function: takes a function as argument
function processUser(user, callback) {
    // Do some processing
    const processed = {
        ...user,
        processedAt: Date.now()
    };

    // Call the passed-in function with the result
    callback(processed);
}

// Usage: we pass a function as the second argument
processUser({ name: "Bob" }, function(result) {
    console.log(result);
});
```

```
// With arrow function (cleaner)
processUser({ name: "Bob" }, result => console.log(result));
```

Why is `processUser` a higher-order function? Because it takes `callback` as a parameter, and `callback` is a function.

1.3.2 Real Web Dev Example: Event Handlers

Every time you add an event listener, you're using a higher-order function:

```
// addEventListener is a HOF--it takes a function as its second argument
button.addEventListener('click', function(event) {
    console.log('Button clicked!');
});

// With arrow function
button.addEventListener('click', (event) => {
    console.log('Button clicked!');
});

// Or pass a named function
function handleClick(event) {
    console.log('Button clicked!');
}
button.addEventListener('click', handleClick);
```

`addEventListener` doesn't know or care what your function does—it just knows to call it when the event occurs.

1.4 Functions That Return Functions

The second HOF pattern: a function that creates and returns another function.

1.4.1 The Factory Pattern

```
// A function that returns a function
function createMultiplier(factor) {
    // This inner function is returned
    return function(number) {
        return number * factor;
    };
}

// Create specific multiplier functions
const double = createMultiplier(2);
```

```
const triple = createMultiplier(3);
const tenX = createMultiplier(10);

// Use them
console.log(double(5));  // 10
console.log(triple(5));  // 15
console.log(tenX(5));    // 50
```

1. `createMultiplier(2)` runs and returns a new function
2. That returned function "remembers" that `factor` is `2`
3. We store that function in `double`
4. When we call `double(5)`, it multiplies `5 * 2`

Key Insight

This "remembering" is called a **closure**—the inner function closes over the variable `factor` from its outer scope.

1.4.2 With Arrow Functions

```
// Same thing, more concise
const createMultiplier = factor => number => number * factor;

// Reading this:
// createMultiplier is a function that takes `factor`
// and returns a function that takes `number`
// and returns `number * factor`

const double = createMultiplier(2);
console.log(double(5)); // 10
```

1.4.3 Real Web Dev Example: Configured Functions

API Fetcher Factory

```
// Create a fetcher for a specific API
function createApiFetcher(baseUrl) {
    return function(endpoint) {
        return fetch(`${baseUrl}${endpoint}`)
            .then(response => response.json());
    };
}

// Create fetchers for different APIs
const githubApi = createApiFetcher('https://api.github.com');
const myApi = createApiFetcher('https://api.myapp.com');

// Use them
```

```
githubApi('/users/octocat').then(data => console.log(data));
myApi('/products').then(data => console.log(data));
```

1.5 Both Patterns Combined

Many HOFs both take AND return functions:

```
// Takes a function, returns a function
function withLogging(fn) {
    return function(...args) {
        console.log(`Calling with args:`, args);
        const result = fn(...args);
        console.log(`Result:`, result);
        return result;
    };
}

// Original function
const add = (a, b) => a + b;

// Enhanced function
const addWithLogging = withLogging(add);

addWithLogging(2, 3);
// Logs: Calling with args: [2, 3]
// Logs: Result: 5
// Returns: 5
```

Key Insight

This pattern—wrapping a function to add behavior—is foundational to middleware, decorators, and much of web development.

1.6 Mental Model: Functions as Values

Here's the mental shift that makes HOFs intuitive:

Think of functions like you think of objects.

With Objects	With Functions
`const user = { name: "Bob" }`	`const greet = () => "Hi"`
`doSomething(user)`	`doSomething(greet)`
`return user`	`return greet`
`users.push(user)`	`handlers.push(greet)`

When someone says "pass a function," think "pass a value that happens to be callable."

1.7 Why Higher-Order Functions Matter in Web Dev

1.7.1 Problem: Repetitive Patterns

× Bad

```
// Without HOFs: repetitive code
const numbers = [1, 2, 3, 4, 5];

// Double each number
const doubled = [];
for (let i = 0; i < numbers.length; i++) {
    doubled.push(numbers[i] * 2);
}

// Triple each number
const tripled = [];
for (let i = 0; i < numbers.length; i++) {
    tripled.push(numbers[i] * 3);
}

// Get string versions
const strings = [];
for (let i = 0; i < numbers.length; i++) {
    strings.push(String(numbers[i]));
}
```

1.7.2 Solution: Abstract the Pattern

Good

```
// With HOFs: the pattern is abstracted
const numbers = [1, 2, 3, 4, 5];

const doubled = numbers.map(n => n * 2);
const tripled = numbers.map(n => n * 3);
const strings = numbers.map(n => String(n));
```

Key Insight

`map` is a HOF that abstracts "do something to each element." We just tell it *what* to do.

1.8 The Three Questions

When you see a function, ask:

1. **Does it take a function as an argument?** → It's a HOF
2. **Does it return a function?** → It's a HOF
3. **Does it do both?** → It's a HOF

```
// Takes function: YES (the callback)
// Returns function: NO
array.forEach(callback);

// Takes function: NO
// Returns function: YES
function createHandler() {
    return () => console.log('handled');
}

// Takes function: YES (fn)
// Returns function: YES (the wrapper)
function memoize(fn) {
    const cache = {};
    return (arg) => {
        if (!(arg in cache)) {
            cache[arg] = fn(arg);
        }
        return cache[arg];
    };
}
```

1.9 Practice Exercises

Exercise 1.1: Identify the HOFs

Which of these are higher-order functions? Why?

Exercise

```
// A
function add(a, b) {
    return a + b;
}

// B
function runTwice(fn) {
    fn();
    fn();
}

// C
const numbers = [1, 2, 3];
numbers.push(4);

// D
function createCounter() {
    let count = 0;
    return function() {
        return ++count;
    };
}

// E
setTimeout(function() {
    console.log('delayed');
}, 1000);
```

Answers:

- **A:** Not a HOF (doesn't take or return functions)
- **B:** HOF (takes a function as argument)
- **C:** Not a HOF (`push` doesn't involve functions here)
- **D:** HOF (returns a function)
- **E:** `setTimeout` is a HOF (takes a function as argument)

Exercise 1.2: Convert to HOF

Convert this repetitive code using a higher-order function:

Exercise

```
// Current code: three similar functions
function logError(message) {
    console.log(`[ERROR] ${message}`);
}

function logWarning(message) {
    console.log(`[WARNING] ${message}`);
}

function logInfo(message) {
    console.log(`[INFO] ${message}`);
}
```

Solution:

```
function createLogger(level) {
    return function(message) {
        console.log(`[${level}] ${message}`);
    };
}

const logError = createLogger('ERROR');
const logWarning = createLogger('WARNING');
const logInfo = createLogger('INFO');
```

Exercise 1.3: Event Handler Factory

Create a HOF called `createClickHandler` that:

- Takes an `action` string
- Returns a function suitable for use as a click handler
- The returned function should log: `"Button clicked: {action}"`

Exercise

```
// Your solution:
const createClickHandler = action => () => {
    console.log(`Button clicked: ${action}`);
};

// Usage:
const saveHandler = createClickHandler('save');
const deleteHandler = createClickHandler('delete');

button1.addEventListener('click', saveHandler);
button2.addEventListener('click', deleteHandler);
```

1.10 Chapter Summary

Chapter Summary

Pattern	Description
First-class function	A function treated as a value
Higher-order function	A function that takes/returns functions
Callback	A function passed to another function
Factory function	A function that returns a function
Closure	A function that remembers its outer scope

Key Insight

HOFs let us abstract *what varies* (the operation) from *what stays the same* (the pattern/structure).

Chapter 2

The Core Three — map, filter, reduce

> *"Give me a lever long enough and a fulcrum on which to place it, and I shall move the world."*
> — Archimedes

These three array methods are the workhorses of functional JavaScript. Master them, and you'll write cleaner, more expressive code.

2.1 map: Transform Every Element

`map` creates a new array by applying a function to every element.

Definition: map

```
const newArray = originalArray.map(transformFunction);
```

2.1.1 Basic Examples

```
const numbers = [1, 2, 3, 4, 5];

// Double each number
const doubled = numbers.map(n => n * 2);
// [2, 4, 6, 8, 10]

// Square each number
const squared = numbers.map(n => n * n);
// [1, 4, 9, 16, 25]

// Convert to strings
const strings = numbers.map(n => String(n));
// ['1', '2', '3', '4', '5']
```

2.1.2 The Transformation Function

The function you pass to `map` receives three arguments:

```
array.map((element, index, originalArray) => {
    // element: the current item
    // index: its position (0, 1, 2...)
    // originalArray: the whole array (rarely used)
    return transformedValue;
```

```
});
```

Usually, you only need `element`:

```
const prices = [10, 20, 30];
const withTax = prices.map(price => price * 1.2);
// [12, 24, 36]
```

Sometimes you need the index:

```
const letters = ['a', 'b', 'c'];
const numbered = letters.map((letter, index) => `${index + 1}. ${letter}`);
// ['1. a', '2. b', '3. c']
```

2.1.3 Web Dev Example: Processing API Data

API Response Transformation

```
// API returns array of user objects
const apiResponse = [
    { id: 1, first_name: 'John', last_name: 'Doe', email: 'john@example.com' },
    { id: 2, first_name: 'Jane', last_name: 'Smith', email: 'jane@example.com' }
];

// Transform to the shape your UI needs
const users = apiResponse.map(user => ({
    id: user.id,
    fullName: `${user.first_name} ${user.last_name}`,
    email: user.email.toLowerCase()
}));

// [
//   { id: 1, fullName: 'John Doe', email: 'john@example.com' },
//   { id: 2, fullName: 'Jane Smith', email: 'jane@example.com' }
// ]
```

2.1.4 Web Dev Example: Rendering Lists

```
// React pattern
const users = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' }
];

const userElements = users.map(user => (
    <li key={user.id}>{user.name}</li>
));

// Vanilla JS pattern
const userHTML = users.map(user => `<li>${user.name}</li>`).join('');
```

```
document.querySelector('ul').innerHTML = userHTML;
```

Critical Rule: `map` always returns an array with the same number of elements:

- Input: 5 elements → Output: 5 elements
- Input: 0 elements → Output: 0 elements

If you need fewer elements, that's `filter`. If you need more or fewer, that might be `flatMap` or `reduce`.

2.2 filter: Select Elements That Pass a Test

`filter` creates a new array containing only elements that pass a test.

Definition: filter

`const newArray = originalArray.filter(testFunction);`

The test function (predicate) must return `true` or `false`:

- Return `true` → element is included
- Return `false` → element is excluded

2.2.1 Basic Examples

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

// Keep only even numbers
const evens = numbers.filter(n => n % 2 === 0);
// [2, 4, 6, 8, 10]

// Keep only numbers greater than 5
const big = numbers.filter(n => n > 5);
// [6, 7, 8, 9, 10]

// Keep only numbers between 3 and 7
const middle = numbers.filter(n => n >= 3 && n <= 7);
// [3, 4, 5, 6, 7]
```

2.2.2 The Predicate Function

A **predicate** is a function that returns `true` or `false`:

```
// These are predicates
const isEven = n => n % 2 === 0;
const isPositive = n => n > 0;
const isLongEnough = str => str.length >= 3;
const hasEmail = user => user.email !== undefined;
```

Using named predicates makes code self-documenting:

× Bad

```
const numbers = [-2, -1, 0, 1, 2, 3, 4];

// Less clear
const result = numbers.filter(n => n > 0 && n % 2 === 0);
```

Good

```
// More clear
const isPositive = n => n > 0;
const isEven = n => n % 2 === 0;
const isPositiveAndEven = n => isPositive(n) && isEven(n);

const result = numbers.filter(isPositiveAndEven);
// [2, 4]
```

2.2.3 Web Dev Example: Filtering Products

```
const products = [
    { id: 1, name: 'Laptop', price: 999, inStock: true, category: 'electronics' },
    { id: 2, name: 'Shirt', price: 29, inStock: true, category: 'clothing' },
    { id: 3, name: 'Phone', price: 699, inStock: false, category: 'electronics' },
    { id: 4, name: 'Pants', price: 59, inStock: true, category: 'clothing' }
];

// Only in-stock items
const available = products.filter(p => p.inStock);

// Only electronics
const electronics = products.filter(p => p.category === 'electronics');

// Affordable and available
const affordable = products.filter(p => p.price < 100 && p.inStock);
// [{ id: 2, ... }, { id: 4, ... }]
```

2.2.4 Web Dev Example: Form Validation

```
const formFields = [
    { name: 'email', value: 'test@example.com', required: true },
    { name: 'phone', value: '', required: false },
    { name: 'name', value: '', required: true }
];

// Find all invalid required fields
const invalidFields = formFields.filter(field =>
    field.required && field.value.trim() === ''
);

// [{ name: 'name', value: '', required: true }]

if (invalidFields.length > 0) {
```

```
    console.log('Please fill in:', invalidFields.map(f => f.name));
}
```

2.2.5 Filtering Out Falsy Values

A common pattern to clean up arrays:

```
const messyArray = [0, 'hello', '', null, 'world', undefined, 42, false];

// Remove all falsy values
const clean = messyArray.filter(Boolean);
// ['hello', 'world', 42]

// This works because Boolean(value) returns true/false
// filter keeps elements where Boolean(element) is true
```

The `Boolean` function is a handy predicate for removing falsy values like `null`, `undefined`, `''`, `0`, and `false`.

2.3 reduce: Accumulate to a Single Value

`reduce` processes an array and accumulates it into a single value. That value can be a number, string, object, or even another array.

Definition: reduce

`const result = array.reduce(reducerFunction, initialValue);`

The reducer function receives four parameters: `accumulator` (the running total/result), `currentElement` (current item being processed), `index`, and `array`. Return the new accumulator value.

```
array.reduce((accumulator, currentElement, index, array) => {
    // accumulator: the running total/result
    // currentElement: the current item being processed
    // Return the new accumulator value
    return newAccumulator;
}, initialValue);
```

2.3.1 Basic Example: Sum

```
const numbers = [1, 2, 3, 4, 5];

const sum = numbers.reduce((total, num) => total + num, 0);
// Step by step:
// total=0, num=1 -> return 1
// total=1, num=2 -> return 3
```

```
// total=3, num=3 -> return 6
// total=6, num=4 -> return 10
// total=10, num=5 -> return 15
// Result: 15
```

```
Step-by-step execution of reduce:
Initial: accumulator = 0
Step 1: 0 + 1 = 1 (accumulator is now 1) Step 2: 1 + 2 = 3 (accumulator is now 3)
Step 3: 3 + 3 = 6 (accumulator is now 6) Step 4: 6 + 4 = 10 (accumulator is now 10)
Step 5: 10 + 5 = 15 (accumulator is now 15)
Final: 15
```

Always Provide Initial Value! Without an initial value, `reduce` uses the first element as the accumulator—but this fails on empty arrays!

```
// GOOD: explicit initial value
const sum = numbers.reduce((acc, n) => acc + n, 0);

// RISKY: no initial value (uses first element)
const sum = numbers.reduce((acc, n) => acc + n);
// Works, but fails on empty array!

// This throws an error:
[].reduce((acc, n) => acc + n); // TypeError!

// This returns 0 safely:
[].reduce((acc, n) => acc + n, 0); // 0
```

2.3.2 Common Reductions

```
const numbers = [1, 2, 3, 4, 5];

// Sum
const sum = numbers.reduce((acc, n) => acc + n, 0);
// 15

// Product
const product = numbers.reduce((acc, n) => acc * n, 1);
// 120

// Maximum
const max = numbers.reduce((acc, n) => n > acc ? n : acc, -Infinity);
// 5

// Minimum
const min = numbers.reduce((acc, n) => n < acc ? n : acc, Infinity);
// 1
```

2.3.3 reduce to Object

Creating a Lookup Object

```
const users = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' },
    { id: 3, name: 'Charlie' }
];

// Create a lookup object by ID
const usersById = users.reduce((acc, user) => {
    acc[user.id] = user;
    return acc;
}, {});

// {
//   1: { id: 1, name: 'Alice' },
//   2: { id: 2, name: 'Bob' },
//   3: { id: 3, name: 'Charlie' }
// }

// Now O(1) lookup instead of O(n) find:
const user = usersById[2]; // { id: 2, name: 'Bob' }
```

2.3.4 reduce to Count Occurrences

```
const fruits = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple'];

const counts = fruits.reduce((acc, fruit) => {
    acc[fruit] = (acc[fruit] || 0) + 1;
    return acc;
}, {});

// { apple: 3, banana: 2, orange: 1 }
```

2.3.5 reduce to Group By

```
const people = [
    { name: 'Alice', department: 'Engineering' },
    { name: 'Bob', department: 'Sales' },
    { name: 'Charlie', department: 'Engineering' },
    { name: 'Diana', department: 'Sales' }
];

const byDepartment = people.reduce((acc, person) => {
    const dept = person.department;
    if (!acc[dept]) {
        acc[dept] = [];
    }
    acc[dept].push(person);
    return acc;
}, {});
```

```
// {
//   Engineering: [{ name: 'Alice', ... }, { name: 'Charlie', ... }],
//   Sales: [{ name: 'Bob', ... }, { name: 'Diana', ... }]
// }
```

2.3.6 Web Dev Example: Shopping Cart Total

```
const cart = [
    { name: 'Laptop', price: 999, quantity: 1 },
    { name: 'Mouse', price: 29, quantity: 2 },
    { name: 'Keyboard', price: 79, quantity: 1 }
];

const total = cart.reduce((sum, item) => {
    return sum + (item.price * item.quantity);
}, 0);

// 999 + 58 + 79 = 1136
```

2.3.7 Web Dev Example: Flatten Query Parameters

```
const params = [
    { key: 'page', value: '1' },
    { key: 'sort', value: 'name' },
    { key: 'order', value: 'asc' }
];

const queryString = params.reduce((acc, param, index) => {
    const prefix = index === 0 ? '?' : '&';
    return `${acc}${prefix}${param.key}=${param.value}`;
}, '');

// '?page=1&sort=name&order=asc'
```

2.4 Method Chaining: Combining map, filter, reduce

The real power comes from combining these methods:

```
const users = [
    { name: 'Alice', age: 25, active: true },
    { name: 'Bob', age: 17, active: true },
    { name: 'Charlie', age: 30, active: false },
    { name: 'Diana', age: 22, active: true }
];

// Get names of active adults
const activeAdultNames = users
    .filter(user => user.active)          // Keep active users
    .filter(user => user.age >= 18)       // Keep adults
    .map(user => user.name);              // Extract names
```

```
// ['Alice', 'Diana']
```

2.4.1 Reading Chains: Top to Bottom

```
const result = data
    .filter(...)    // First: remove unwanted items
    .map(...)       // Second: transform remaining items
    .reduce(...);   // Third: combine into final result

// Read it as a story:
// "Take data, keep only X, transform each to Y, combine into Z"
```

Key Insight

Think of data processing as a **pipeline**—data flows through transformations, filters, and accumulations.

2.4.2 Web Dev Example: API Response Processing

```
// Raw API response
const apiResponse = {
    data: [
        { id: 1, type: 'user', attributes: { name: 'Alice', email: 'alice@test.com',
          ↪ role: 'admin' }},
        { id: 2, type: 'user', attributes: { name: 'Bob', email: 'bob@test.com',
          ↪ role: 'user' }},
        { id: 3, type: 'user', attributes: { name: 'Charlie', email: null, role:
          ↪ 'user' }},
        { id: 4, type: 'user', attributes: { name: 'Diana', email: 'diana@test.com',
          ↪ role: 'admin' }}
    ]
};

// Process: get admin emails for notification
const adminEmails = apiResponse.data
    .filter(item => item.attributes.role === 'admin')  // Only admins
    .filter(item => item.attributes.email !== null)    // Has email
    .map(item => item.attributes.email);               // Extract email

// ['alice@test.com', 'diana@test.com']
```

2.5 Performance Consideration

Each `map` and `filter` creates a new array. **For small arrays (< 10,000 items):** Don't worry—the clarity is worth it. **For large arrays:** We'll cover optimization in Chapter 12 (transducers, lazy evaluation).

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

const result = numbers
    .filter(n => n % 2 === 0)  // Creates [2, 4, 6, 8, 10]
    .map(n => n * 2)           // Creates [4, 8, 12, 16, 20]
    .filter(n => n > 10);      // Creates [12, 16, 20]
```

2.6 When to Use Each

Goal	Method	Example
Transform each element	`map`	Convert prices to include tax
Remove some elements	`filter`	Keep only in-stock items
Combine into one value	`reduce`	Calculate total price
Find one element	`find`	Get user by ID (Chapter 3)
Check if any pass	`some`	Are any items on sale? (Chapter 3)
Check if all pass	`every`	Are all fields valid? (Chapter 3)

2.7 Practice Exercises

Exercise 2.1: Data Transformation

Given this API response, extract an array of formatted strings:

Exercise

```
const products = [
    { id: 1, name: 'Laptop', price: 999.99, currency: 'USD' },
    { id: 2, name: 'Mouse', price: 29.99, currency: 'USD' },
    { id: 3, name: 'Keyboard', price: 79.99, currency: 'USD' }
];

// Solution:
const formatted = products.map(p => `${p.name}: $${p.price}`);
// ['Laptop: $999.99', 'Mouse: $29.99', 'Keyboard: $79.99']
```

Exercise 2.2: Filter Chain

Given this data, find all active premium users from the US:

Exercise

```
const users = [
    { id: 1, name: 'Alice', country: 'US', plan: 'premium', active: true },
    { id: 2, name: 'Bob', country: 'UK', plan: 'premium', active: true },
    { id: 3, name: 'Charlie', country: 'US', plan: 'free', active: true },
    { id: 4, name: 'Diana', country: 'US', plan: 'premium', active: false },
    { id: 5, name: 'Eve', country: 'US', plan: 'premium', active: true }
];

// Solution:
const result = users
    .filter(u => u.active)
    .filter(u => u.plan === 'premium')
    .filter(u => u.country === 'US');
// [{ id: 1, ... }, { id: 5, ... }]
```

Exercise 2.3: Reduce to Object

Convert this array to an object grouped by category:

Exercise

```
const items = [
    { name: 'Apple', category: 'fruit' },
    { name: 'Carrot', category: 'vegetable' },
    { name: 'Banana', category: 'fruit' },
    { name: 'Broccoli', category: 'vegetable' }
];

// Solution:
const grouped = items.reduce((acc, item) => {
    if (!acc[item.category]) {
        acc[item.category] = [];
    }
    acc[item.category].push(item.name);
    return acc;
}, {});
// { fruit: ['Apple', 'Banana'], vegetable: ['Carrot', 'Broccoli'] }
```

Exercise 2.4: Complete Pipeline

Build a data processing pipeline for this e-commerce scenario:

Exercise

```
const orders = [
    { id: 1, customer: 'Alice', items: [{ price: 10 }, { price: 20 }], status:
      ↪ 'completed' },
    { id: 2, customer: 'Bob', items: [{ price: 15 }], status: 'pending' },
    { id: 3, customer: 'Alice', items: [{ price: 30 }, { price: 40 }], status:
      ↪ 'completed' },
    { id: 4, customer: 'Charlie', items: [{ price: 25 }], status: 'completed' }
];

// Task: Calculate total revenue from completed orders
// Solution:
const revenue = orders
    .filter(order => order.status === 'completed')
    .flatMap(order => order.items)
    .reduce((sum, item) => sum + item.price, 0);
// 10 + 20 + 30 + 40 + 25 = 125
```

2.8 Chapter Summary

Chapter Summary

Method	Input	Output	Purpose
`map`	Array of N items	Array of N items	Transform each element
`filter`	Array of N items	Array of 0 to N items	Keep elements that pass test
`reduce`	Array of N items	Single value (any type)	Accumulate to one result

Key Insight

Think of data processing as a pipeline—data flows through transformations, filters, and accumulations.

Chapter 3

Beyond the Core — find, some, every, flatMap

> *"Perfection is achieved not when there is nothing more to add, but when there is nothing left to take away."*
> — Antoine de Saint-Exupéry

The core three (`map`, `filter`, `reduce`) handle most cases, but these additional methods make specific patterns cleaner and more efficient.

3.1 find: Get the First Match

`find` returns the **first element** that passes a test, or `undefined` if none match.

Definition: find

`const element = array.find(testFunction);`
Returns the element itself, or `undefined` if no match.

3.1.1 Basic Example

```
const numbers = [1, 5, 10, 15, 20];

const firstBigNumber = numbers.find(n => n > 8);
// 10 (not [10, 15, 20] -- just the first one)

const notFound = numbers.find(n => n > 100);
// undefined
```

3.1.2 find vs filter

Method	Returns	Use When
`filter`	Array of all matches	You need all matching items
`find`	First match or `undefined`	You need just one item

```
const users = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' },
    { id: 3, name: 'Alice' }  // Duplicate name
];
```

```
// filter: all users named Alice
users.filter(u => u.name === 'Alice');
// [{ id: 1, name: 'Alice' }, { id: 3, name: 'Alice' }]

// find: first user named Alice
users.find(u => u.name === 'Alice');
// { id: 1, name: 'Alice' }
```

3.1.3 Performance: find Stops Early

`find` stops iterating as soon as it finds a match:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

// find stops at 3
numbers.find(n => {
    console.log('Checking', n);
    return n > 2;
});
// Logs: Checking 1, Checking 2, Checking 3
// Returns: 3

// filter checks everything
numbers.filter(n => {
    console.log('Checking', n);
    return n > 2;
});
// Logs: Checking 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
// Returns: [3, 4, 5, 6, 7, 8, 9, 10]
```

Use `find` when you only need one item—it's more efficient than `filter` because it stops at the first match.

3.1.4 Web Dev Example: Find User by ID

```
const users = [
    { id: 1, name: 'Alice', email: 'alice@test.com' },
    { id: 2, name: 'Bob', email: 'bob@test.com' },
    { id: 3, name: 'Charlie', email: 'charlie@test.com' }
];

function getUserById(id) {
    return users.find(user => user.id === id);
}

const user = getUserById(2);
// { id: 2, name: 'Bob', email: 'bob@test.com' }

const notFound = getUserById(999);
// undefined
```

3.1.5 Handling undefined

```
const user = users.find(u => u.id === 999);

// WRONG: might crash
console.log(user.name); // TypeError: Cannot read property 'name' of undefined

// RIGHT: check first
if (user) {
    console.log(user.name);
}

// Or use optional chaining
console.log(user?.name); // undefined (no crash)

// Or provide default
const name = user?.name ?? 'Unknown';
```

3.1.6 findIndex: Get Position Instead of Element

```
const numbers = [10, 20, 30, 40];

const index = numbers.findIndex(n => n > 25);
// 2 (position of 30)

const notFoundIndex = numbers.findIndex(n => n > 100);
// -1 (not found)
```

Useful when you need to modify or remove:

```
const todos = [
    { id: 1, text: 'Learn JS', done: false },
    { id: 2, text: 'Build app', done: false }
];

// Find and update
const index = todos.findIndex(t => t.id === 1);
if (index !== -1) {
    todos[index] = { ...todos[index], done: true };
}
```

3.2 some: Does Any Element Pass?

`some` returns `true` if **at least one** element passes the test.

Definition: some

`const hasMatch = array.some(testFunction);`

Returns `true` or `false`.

3.2.1 Basic Examples

```
const numbers = [1, 2, 3, 4, 5];

numbers.some(n => n > 4);     // true (5 is > 4)
numbers.some(n => n > 10);    // false (none are > 10)
numbers.some(n => n === 3);   // true (3 exists)
```

3.2.2 some Stops Early

Like `find`, `some` stops as soon as it finds a passing element:

```
const numbers = [1, 2, 3, 4, 5];

numbers.some(n => {
    console.log('Checking', n);
    return n === 2;
});
// Logs: Checking 1, Checking 2
// Returns: true (stopped early)
```

3.2.3 Web Dev Example: Permission Check

```
const user = {
    name: 'Alice',
    roles: ['editor', 'viewer']
};

const adminRoles = ['admin', 'superadmin'];

const isAdmin = user.roles.some(role => adminRoles.includes(role));
// false

// Or check for specific permission
const canEdit = user.roles.some(role => role === 'editor' || role === 'admin');
// true
```

3.2.4 Web Dev Example: Form Has Errors

```
const formFields = [
    { name: 'email', error: null },
    { name: 'password', error: 'Too short' },
    { name: 'username', error: null }
];

const hasErrors = formFields.some(field => field.error !== null);
// true

if (hasErrors) {
    console.log('Please fix errors before submitting');
}
```

3.3 every: Do All Elements Pass?

every returns `true` only if **all elements** pass the test.

Definition: every

`const allPass = array.every(testFunction);`
Returns `true` or `false`.

3.3.1 Basic Examples

```
const numbers = [2, 4, 6, 8];

numbers.every(n => n % 2 === 0);  // true (all even)
numbers.every(n => n > 5);        // false (2 and 4 fail)
numbers.every(n => n > 0);        // true (all positive)
```

3.3.2 every Stops Early (On First Failure)

```
const numbers = [2, 4, 5, 6, 8];  // 5 is odd

numbers.every(n => {
    console.log('Checking', n);
    return n % 2 === 0;
});
// Logs: Checking 2, Checking 4, Checking 5
// Returns: false (stopped at 5)
```

Empty Array: every returns `true` for empty arrays: `[].every(n => n > 0)` returns `true`. This is "vacuous truth"—logically, "all elements pass" is true when there are no elements to fail.

3.3.3 Web Dev Example: Form Validation

```
const formFields = [
    { name: 'email', value: 'test@example.com', valid: true },
    { name: 'password', value: '12345678', valid: true },
    { name: 'username', value: 'alice', valid: true }
];

const isFormValid = formFields.every(field => field.valid);
// true

// With validation functions
const validators = {
    email: value => value.includes('@'),
    password: value => value.length >= 8,
```

```
    username: value => value.length >= 3
};

const isValid = formFields.every(field =>
    validators[field.name](field.value)
);
```

3.3.4 Web Dev Example: All Items Selected

```
const items = [
    { id: 1, selected: true },
    { id: 2, selected: true },
    { id: 3, selected: false }
];

const allSelected = items.every(item => item.selected);
// false

// Toggle "select all" checkbox state
selectAllCheckbox.checked = allSelected;
```

3.4 some vs every: Quick Reference

Method	Returns true when	Stops when	Empty array
`some`	At least one passes	First pass	`false`
`every`	All pass	First fail	`true`

```
const numbers = [1, 2, 3, 4, 5];

// "Is there ANY number greater than 3?"
numbers.some(n => n > 3);   // true

// "Are ALL numbers greater than 3?"
numbers.every(n => n > 3);  // false
```

3.5 flatMap: Map and Flatten in One Step

`flatMap` combines `map` and `flat(1)`—it transforms each element and flattens one level.

3.5.1 The Problem flatMap Solves

```
const sentences = ['Hello world', 'How are you'];

// map gives nested arrays
const words = sentences.map(s => s.split(' '));
// [['Hello', 'world'], ['How', 'are', 'you']]
```

```
// We need to flatten
const flatWords = sentences.map(s => s.split(' ')).flat();
// ['Hello', 'world', 'How', 'are', 'you']

// flatMap does both in one step
const words2 = sentences.flatMap(s => s.split(' '));
// ['Hello', 'world', 'How', 'are', 'you']
```

Definition: flatMap

`array.flatMap(fn)` is equivalent to `array.map(fn).flat(1)`.
Use `flatMap` when your transformation function returns an array and you want all results in a single flat array.

3.5.2 When Your Transform Returns an Array

```
const users = [
    { name: 'Alice', pets: ['cat', 'dog'] },
    { name: 'Bob', pets: ['fish'] },
    { name: 'Charlie', pets: [] }
];

// Get all pets
const allPets = users.flatMap(user => user.pets);
// ['cat', 'dog', 'fish']

// With map, you'd get nested arrays:
const nested = users.map(user => user.pets);
// [['cat', 'dog'], ['fish'], []]
```

3.5.3 flatMap for Conditional Inclusion

Return empty array `[]` to exclude items, array with item `[item]` to include:

```
const numbers = [1, 2, 3, 4, 5, 6];

// Double only even numbers, remove odds
const doubledEvens = numbers.flatMap(n =>
    n % 2 === 0 ? [n * 2] : []
);
// [4, 8, 12]

// Equivalent to filter + map:
const same = numbers.filter(n => n % 2 === 0).map(n => n * 2);
```

3.5.4 Web Dev Example: Nested Data Extraction

```
const departments = [
    {
        name: 'Engineering',
        teams: [
            { name: 'Frontend', members: ['Alice', 'Bob'] },
            { name: 'Backend', members: ['Charlie'] }
        ]
    },
    {
        name: 'Design',
        teams: [
            { name: 'UX', members: ['Diana', 'Eve'] }
        ]
    }
];

// Get all team names
const teamNames = departments.flatMap(dept =>
    dept.teams.map(team => team.name)
);
// ['Frontend', 'Backend', 'UX']

// Get all members
const allMembers = departments.flatMap(dept =>
    dept.teams.flatMap(team => team.members)
);
// ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve']
```

3.5.5 Web Dev Example: Processing Nested API Response

```
const apiResponse = {
    pages: [
        { items: [{ id: 1 }, { id: 2 }] },
        { items: [{ id: 3 }] },
        { items: [{ id: 4 }, { id: 5 }] }
    ]
};

// Flatten all items from all pages
const allItems = apiResponse.pages.flatMap(page => page.items);
// [{ id: 1 }, { id: 2 }, { id: 3 }, { id: 4 }, { id: 5 }]
```

3.6 Combining Everything: Real Patterns

3.6.1 Pattern: Find and Transform

```
const products = [
    { id: 1, name: 'Laptop', price: 999 },
    { id: 2, name: 'Phone', price: 699 },
    { id: 3, name: 'Tablet', price: 499 }
];
```

```
// Find product and get just the name
const productName = products.find(p => p.id === 2)?.name;
// 'Phone'

// Find product and transform it
const productCard = products.find(p => p.id === 2);
const formatted = productCard
    ? `${productCard.name}: $${productCard.price}`
    : 'Not found';
// 'Phone: $699'
```

3.6.2 Pattern: Validate Then Process

```
const items = [
    { name: 'A', quantity: 5, price: 10 },
    { name: 'B', quantity: 0, price: 20 },
    { name: 'C', quantity: 3, price: 15 }
];

// Check if all items have valid quantities before calculating
const allValid = items.every(item => item.quantity >= 0);

if (allValid) {
    const total = items
        .filter(item => item.quantity > 0)  // Only items with quantity
        .map(item => item.quantity * item.price)  // Calculate line totals
        .reduce((sum, lineTotal) => sum + lineTotal, 0);  // Sum up

    console.log(`Total: $${total}`);  // Total: $95
}
```

3.6.3 Pattern: Extract from Nested Structure

```
const organization = {
    departments: [
        {
            name: 'Engineering',
            employees: [
                { name: 'Alice', skills: ['JS', 'React'] },
                { name: 'Bob', skills: ['Python', 'JS'] }
            ]
        },
        {
            name: 'Design',
            employees: [
                { name: 'Charlie', skills: ['Figma', 'CSS'] }
            ]
        }
    ]
};

// Find all unique skills in the organization
```

```
const allSkills = organization.departments
    .flatMap(dept => dept.employees)
    .flatMap(emp => emp.skills);

const uniqueSkills = [...new Set(allSkills)];
// ['JS', 'React', 'Python', 'Figma', 'CSS']

// Find if anyone knows React
const hasReactDev = organization.departments
    .flatMap(dept => dept.employees)
    .some(emp => emp.skills.includes('React'));
// true
```

3.7 Decision Guide: Which Method to Use?

```
Do you need...
+- All items that match? | - filter() | +- Just the first match? | - find() | +-
The position of first match? | - findIndex() | +- To know if ANY item matches? |
- some() | +- To know if ALL items match? | - every() | +- To transform each item?
| +- One-to-one transformation? | | - map() | - One-to-many (nested results)? | -
flatMap() | - To combine into single value? - reduce()
```

3.8 Practice Exercises

Exercise 3.1: User Lookup

Implement these functions using the appropriate methods:

Exercise

```
const users = [
    { id: 1, name: 'Alice', role: 'admin', active: true },
    { id: 2, name: 'Bob', role: 'user', active: false },
    { id: 3, name: 'Charlie', role: 'user', active: true },
    { id: 4, name: 'Diana', role: 'admin', active: true }
];

// 1. Find user by ID
const findById = id => users.find(u => u.id === id);

// 2. Check if any user is inactive
const hasInactiveUsers = () => users.some(u => !u.active);

// 3. Check if all admins are active
const allAdminsActive = () => users
    .filter(u => u.role === 'admin')
    .every(u => u.active);

// 4. Get index of first inactive user
const findFirstInactiveIndex = () => users.findIndex(u => !u.active);
```

Exercise 3.2: Nested Data Extraction

Given this data structure, extract all product names across all categories:

Exercise

```
const store = {
    categories: [
        {
            name: 'Electronics',
            products: [
                { name: 'Laptop', price: 999 },
                { name: 'Phone', price: 699 }
            ]
        },
        {
            name: 'Clothing',
            products: [
                { name: 'Shirt', price: 29 },
                { name: 'Pants', price: 49 },
                { name: 'Hat', price: 19 }
            ]
        }
    ]
};

// Solution:
const productNames = store.categories
    .flatMap(cat => cat.products)
    .map(p => p.name);
// ['Laptop', 'Phone', 'Shirt', 'Pants', 'Hat']
```

Exercise 3.3: Shopping Cart Operations

Implement these cart operations:

Exercise

```
const cart = [
    { id: 1, name: 'Laptop', price: 999, quantity: 1 },
    { id: 2, name: 'Mouse', price: 29, quantity: 2 },
    { id: 3, name: 'Keyboard', price: 79, quantity: 1 },
    { id: 4, name: 'Monitor', price: 299, quantity: 0 }  // Out of stock
];

// 1. Find item by ID
const findItem = id => cart.find(item => item.id === id);

// 2. Check if cart has any out-of-stock items
const hasOutOfStock = () => cart.some(item => item.quantity === 0);

// 3. Check if all items are available
const allAvailable = () => cart.every(item => item.quantity > 0);

// 4. Get available items summary
const getAvailableItemsSummary = () => cart
    .filter(item => item.quantity > 0)
    .map(item => `${item.name} (x${item.quantity})`);
// ['Laptop (x1)', 'Mouse (x2)', 'Keyboard (x1)']
```

3.9 Chapter Summary

Chapter Summary

Method	Returns	Stops Early?	Use Case
`find`	First match or `undefined`	Yes (first match)	Get single item by condition
`findIndex`	Index or `-1`	Yes (first match)	Get position for updates
`some`	`true` / `false`	Yes (first `true`)	Check if any item matches
`every`	`true` / `false`	Yes (first `false`)	Validate all items
`flatMap`	Flattened array	No	Transform + flatten nested results

Key Insight

`find`, `some`, and `every` are optimized for early exit—use them instead of `filter` when you only need to know about existence or validity.

Part I Summary: Building Blocks Mastered

You now have the foundation:

Concept	What You Learned
First-Class Functions	Functions are values—store, pass, return them
Higher-Order Functions	Functions that take/return functions
Closures	Inner functions remember outer scope
`map`	Transform every element (1:1)
`filter`	Keep elements that pass test
`reduce`	Accumulate to single value
`find` / `findIndex`	Get first match (with early exit)
`some` / `every`	Boolean checks (with early exit)
`flatMap`	Transform and flatten nested results
Method Chaining	Combine operations in readable pipelines

What's Next

In **Part II: Patterns & Techniques**, you'll learn:

- Creating powerful function factories (Chapter 4)
- Building data pipelines with `pipe` and `compose` (Chapter 5)
- Partial application and currying (Chapter 6)
- The systematic protocol for converting imperative code (Chapter 7)

Part I Practice Project: User Dashboard Data Pipeline

Practice Project: User Dashboard Data Pipeline

Apply everything from Part I to build a complete data processing solution using the raw data shown below.

Tasks:

1. Get all active users with normalized emails (lowercase)
2. Calculate total salary expense for active employees
3. Group active users by department
4. Find the highest paid active employee
5. Check if all Engineering employees are active
6. Get list of departments that have at least one active employee

7. Calculate average salary per department (active employees only)
8. Create a summary string for each active user

Raw Data:

```
const rawData = {
    users: [
        { id: 1, name: 'Alice Johnson', email: 'ALICE@COMPANY.COM',
          department: 'Engineering', salary: 95000, active: true,
          startDate: '2020-03-15' },
        { id: 2, name: 'Bob Smith', email: 'bob@company.com',
          department: 'Sales', salary: 75000, active: true,
          startDate: '2019-07-22' },
        { id: 3, name: 'Charlie Brown', email: 'CHARLIE@COMPANY.COM',
          department: 'Engineering', salary: 105000, active: false,
          startDate: '2018-01-10' },
        { id: 4, name: 'Diana Ross', email: 'diana@company.com',
          department: 'Engineering', salary: 115000, active: true,
          startDate: '2017-11-30' },
        { id: 5, name: 'Eve Wilson', email: 'eve@company.com',
          department: 'Sales', salary: 80000, active: true,
          startDate: '2021-02-14' }
    ],
    departments: [
        { name: 'Engineering', budget: 500000 },
        { name: 'Sales', budget: 300000 }
    ]
};
```

Solutions:

```
// 1. Active users with normalized emails
const activeUsers = rawData.users
    .filter(u => u.active)
    .map(u => ({ ...u, email: u.email.toLowerCase() }));

// 2. Total salary expense
const totalSalary = rawData.users
    .filter(u => u.active)
    .reduce((sum, u) => sum + u.salary, 0);

// 3. Group by department
const byDepartment = rawData.users
    .filter(u => u.active)
    .reduce((acc, u) => {
        if (!acc[u.department]) acc[u.department] = [];
        acc[u.department].push(u);
        return acc;
    }, {});

// 4. Highest paid active employee
const highestPaid = rawData.users
```

```
    .filter(u => u.active)
    .reduce((max, u) => u.salary > max.salary ? u : max);

// 5. All Engineering employees active?
const allEngActive = rawData.users
    .filter(u => u.department === 'Engineering')
    .every(u => u.active);

// 6. Departments with active employees
const activeDepts = [...new Set(
    rawData.users
        .filter(u => u.active)
        .map(u => u.department)
)];

// 7. Average salary per department
const avgByDept = rawData.users
    .filter(u => u.active)
    .reduce((acc, u) => {
        if (!acc[u.department]) {
            acc[u.department] = { total: 0, count: 0 };
        }
        acc[u.department].total += u.salary;
        acc[u.department].count += 1;
        return acc;
    }, {});
// Then: Object.entries(avgByDept).map(([k, v]) => [k, v.total / v.count])

// 8. Summary strings
const summaries = rawData.users
    .filter(u => u.active)
    .map(u => `${u.name} (${u.department}) - $${u.salary}`);
```

Part II

Patterns & Techniques

Chapter 4

Function Factories & Closures

Function factories are HOFs that create and return new functions. They're the foundation of many powerful patterns in JavaScript—from event handlers to middleware to React hooks.

4.1 The Factory Pattern

A function factory creates specialized functions based on configuration:

```
// Factory: creates greeting functions
function createGreeter(greeting) {
    return function(name) {
        return `${greeting}, ${name}!`;
    };
}

// Create specialized greeters
const sayHello = createGreeter('Hello');
const sayHi = createGreeter('Hi');
const sayHey = createGreeter('Hey');

// Use them
sayHello('Alice');  // 'Hello, Alice!'
sayHi('Bob');       // 'Hi, Bob!'
sayHey('Charlie');  // 'Hey, Charlie!'
```

4.1.1 Why This Matters

Instead of writing repetitive code:

× **Bad**

```
// Without factory: repetitive, coupled code
function greetWithHello(name) { return `Hello, ${name}!`; }
function greetWithHi(name) { return `Hi, ${name}!`; }
function greetWithHey(name) { return `Hey, ${name}!`; }
```

You get configurable, DRY code:

Good

```
// With factory: configurable, DRY
const sayHello = createGreeter('Hello');
const sayHi = createGreeter('Hi');
const sayHey = createGreeter('Hey');
```

4.2 Understanding Closures

Definition: Closure

A **closure** is a function that "remembers" variables from its outer scope, even after that outer function has finished executing.

4.2.1 The Mechanism

```
function outer() {
    const secret = 'I am hidden';  // Local variable

    function inner() {
        console.log(secret);  // Inner function accesses outer's variable
    }

    return inner;  // Return the inner function
}

const myFunction = outer();  // outer() runs and returns inner
// At this point, outer() is done executing
// But...

myFunction();  // 'I am hidden' — inner still has access to secret!
```

The closure "closes over" the variable `secret`, keeping it alive in memory even after `outer()` has returned.

What happened?

1. `outer()` creates `secret` and `inner`
2. `inner` references `secret`
3. `outer()` returns `inner` and finishes
4. Normally, `secret` would be garbage collected
5. But `inner` still needs it, so JavaScript keeps `secret` alive
6. This is the closure— `inner` "closes over" `secret`

4.2.2 Visualizing Closure

```
┌──────────────────────────────────────────────┐ │ outer() Scope │
│ ┌──────────────────────────────────────────┐ │ │ │ secret = 'I am
hidden' │ │ │ │ │ │ │ │ ┌──────────────────────────────┐ │ │ │ │ │
inner() Scope │ │ │ │ │ │ • Has access to: secret │ │ │ │ │ │ • This
access persists │ │ │ │ │ └──────────────────────────────┘ │ │ │
└──────────────────────────────────────────┘ │ └──────────────────────────────────────┘
After outer() returns: - outer's scope would normally be garbage collected - But
inner() still references secret - So the binding is preserved (the closure)
```

4.2.3 Each Call Creates a New Closure

```
function createCounter() {
    let count = 0;  // Each call gets its own count
    return function() {
        count++;
        return count;
    };
}

const counterA = createCounter();
const counterB = createCounter();

counterA();  // 1
counterA();  // 2
counterA();  // 3

counterB();  // 1 (separate count!)
counterB();  // 2
```

`counterA` and `counterB` each have their own `count` variable—they don't share.

4.3 Private State with Closures

Closures let you create truly private data—inaccessible from outside:

Bank Account with Private Balance

```
function createBankAccount(initialBalance) {
    let balance = initialBalance;  // Private!

    return {
        deposit(amount) {
            if (amount > 0) {
                balance += amount;
                return balance;
            }
            throw new Error('Deposit amount must be positive');
        },

        withdraw(amount) {
```

```
            if (amount > 0 && amount ≤ balance) {
                balance -= amount;
                return balance;
            }
            throw new Error('Invalid withdrawal');
        },

        getBalance() {
            return balance;
        }
    };
}

const account = createBankAccount(100);

account.getBalance();   // 100
account.deposit(50);    // 150
account.withdraw(30);   // 120

// Cannot access balance directly!
account.balance;        // undefined
```

Key Insight

This is **real** privacy—not convention (like `_balance`), not symbols, actual inaccessibility. The `balance` variable cannot be accessed from outside the closure.

4.4 Practical Factory Patterns

4.4.1 Pattern 1: Configured API Fetchers

```
function createApiClient(baseUrl, defaultHeaders = {}) {
    return {
        async get(endpoint) {
            const response = await fetch(`${baseUrl}${endpoint}`, {
                method: 'GET',
                headers: defaultHeaders
            });
            return response.json();
        },

        async post(endpoint, data) {
            const response = await fetch(`${baseUrl}${endpoint}`, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                    ...defaultHeaders
                },
                body: JSON.stringify(data)
            });
            return response.json();
        }
    };
```

```
}

// Create clients for different APIs
const github = createApiClient('https://api.github.com', {
    'Authorization': 'token xxx'
});

const myApi = createApiClient('https://api.myapp.com', {
    'X-API-Key': 'my-key'
});

// Use them
const user = await github.get('/users/octocat');
const products = await myApi.get('/products');
```

4.4.2 Pattern 2: Event Handler Factories

```
function createClickHandler(action, options = {}) {
    const { preventDefault = true, stopPropagation = false, log = false } = options;

    return function(event) {
        if (preventDefault) event.preventDefault();
        if (stopPropagation) event.stopPropagation();
        if (log) console.log(`Action: ${action}`, event);

        // Perform the action
        switch (action) {
            case 'save':
                saveForm();
                break;
            case 'delete':
                deleteItem();
                break;
            case 'toggle':
                toggleState();
                break;
        }
    };
}

// Create handlers
const saveHandler = createClickHandler('save', { log: true });
const deleteHandler = createClickHandler('delete', { stopPropagation: true });

// Attach to elements
saveButton.addEventListener('click', saveHandler);
deleteButton.addEventListener('click', deleteHandler);
```

4.4.3 Pattern 3: Validator Factories

```
function createValidator(rules) {
    return function(value) {
        const errors = [];

        for (const rule of rules) {
            if (!rule.test(value)) {
                errors.push(rule.message);
            }
        }

        return {
            valid: errors.length === 0,
            errors
        };
    };
}

// Create specific validators
const validateEmail = createValidator([
    { test: v => v.length > 0, message: 'Email is required' },
    { test: v => v.includes('@'), message: 'Email must contain @' },
    { test: v => v.includes('.'), message: 'Email must contain a domain' }
]);

const validatePassword = createValidator([
    { test: v => v.length >= 8, message: 'Password must be at least 8 characters' },
    { test: v => /[A-Z]/.test(v), message: 'Password must contain uppercase' },
    { test: v => /[0-9]/.test(v), message: 'Password must contain a number' }
]);

// Use them
validateEmail('test@example.com');  // { valid: true, errors: [] }
validateEmail('invalid');           // { valid: false, errors: [...] }

validatePassword('weak');           // { valid: false, errors: [...] }
validatePassword('Strong1234');     // { valid: true, errors: [] }
```

4.4.4 Pattern 4: Memoization Factory

```
function createMemoized(fn) {
    const cache = new Map();

    return function(...args) {
        const key = JSON.stringify(args);

        if (cache.has(key)) {
            console.log('Cache hit for:', key);
            return cache.get(key);
        }

        console.log('Cache miss for:', key);
        const result = fn(...args);
        cache.set(key, result);
```

```
        return result;
    };
}

// Memoized Fibonacci
const fastFib = createMemoized(function(n) {
    if (n ≤ 1) return n;
    return fastFib(n - 1) + fastFib(n - 2);
});

fastFib(40);  // Fast! Uses cache for repeated calculations
```

4.4.5 Pattern 5: Rate Limiter Factory

```
function createRateLimiter(fn, limit, windowMs) {
    let calls = 0;
    let windowStart = Date.now();

    return async function(...args) {
        const now = Date.now();

        // Reset window if expired
        if (now - windowStart ≥ windowMs) {
            calls = 0;
            windowStart = now;
        }

        // Check limit
        if (calls ≥ limit) {
            throw new Error(`Rate limit exceeded. Max ${limit} calls per
              ↪  ${windowMs}ms`);
        }

        calls++;
        return fn(...args);
    };
}

// Allow max 5 API calls per second
const limitedFetch = createRateLimiter(fetch, 5, 1000);

// Use it
try {
    await limitedFetch('/api/data');  // Works
    await limitedFetch('/api/data');  // Works
    // ... after 5 calls in < 1 second:
    await limitedFetch('/api/data');  // Throws!
} catch (e) {
    console.log(e.message);
}
```

4.5 Closure Memory Considerations

Closures keep references alive. This is powerful but can cause memory issues.

4.5.1 The Problem: Unintended Retention

```
function processData() {
    const hugeData = new Array(1000000).fill('x');  // ~8MB

    // We only need a small piece
    const summary = hugeData.length;

    // This closure captures the entire scope
    return function() {
        return summary;  // Only uses summary
    };
}

const getSummary = processData();
// hugeData is STILL in memory because the closure exists!
```

4.5.2 The Fix: Minimize Closure Scope

```
function processData() {
    let summary;

    {
        // Block scope limits what the closure can capture
        const hugeData = new Array(1000000).fill('x');
        summary = hugeData.length;
        // hugeData goes out of scope here
    }

    return function() {
        return summary;  // Only captures summary
    };
}

// Or explicitly nullify
function processData() {
    let hugeData = new Array(1000000).fill('x');
    const summary = hugeData.length;

    hugeData = null;  // Release the reference

    return function() {
        return summary;
    };
}
```

4.5.3 The Shared Context Trap

All closures in the same scope share one context object:

```
function createHandlers() {
    const sharedData = { /* large object */ };

    // BOTH closures keep sharedData alive
```

```
    const handler1 = () => console.log('handler1');  // Doesn't use sharedData
    const handler2 = () => console.log(sharedData);  // Uses sharedData

    return { handler1, handler2 };
}

const { handler1 } = createHandlers();
// Even though we only kept handler1, sharedData is retained
// because handler1's closure context includes it
```

Closures keep references alive. Be careful not to accidentally retain large objects in memory.

4.5.4 Best Practice: Factory Function Hygiene

× Bad

```
// BAD: Accidentally capturing large data
function createLoggerBad(config) {
    // config might be huge, but we only need prefix
    return (message) => console.log(`[${config.prefix}] ${message}`);
}
```

Good

```
// GOOD: Only close over what you need
function createLogger(prefix) {
    // prefix is small, intentionally captured
    return (message) => console.log(`[${prefix}] ${message}`);
}

// Or extract what you need
function createLoggerFixed(config) {
    const prefix = config.prefix;  // Extract only what's needed
    return (message) => console.log(`[${prefix}] ${message}`);
}
```

4.6 The Module Pattern

Closures enable the classic module pattern—private state with public interface:

```
const UserModule = (function() {
    // Private
    let users = [];
    let nextId = 1;

    function generateId() {
        return nextId++;
    }
```

```
    // Public API (returned object)
    return {
        add(name) {
            const user = { id: generateId(), name };
            users.push(user);
            return user;
        },

        remove(id) {
            users = users.filter(u => u.id !== id);
        },

        getAll() {
            return [...users];  // Return copy to prevent mutation
        },

        findById(id) {
            return users.find(u => u.id === id);
        }
    };
})();  // Immediately invoked!

// Usage
UserModule.add('Alice');     // { id: 1, name: 'Alice' }
UserModule.add('Bob');       // { id: 2, name: 'Bob' }
UserModule.getAll();         // [{ id: 1, ... }, { id: 2, ... }]

// Cannot access internals
UserModule.users;            // undefined
UserModule.nextId;           // undefined
UserModule.generateId;       // undefined
```

4.7 Common Closure Pitfalls

4.7.1 Pitfall 1: Loop Variables

× **Bad**

```
// BROKEN: All handlers log 5
for (var i = 0; i < 5; i++) {
    buttons[i].addEventListener('click', function() {
        console.log(i);  // Always 5!
    });
}
```

Use `let` instead of `var` to create block-scoped variables that work correctly with closures in loops.

Good

```
// FIX 1: Use let (block-scoped)
for (let i = 0; i < 5; i++) {
    buttons[i].addEventListener('click', function() {
        console.log(i);  // Correct: 0, 1, 2, 3, 4
    });
}

// FIX 2: Create new scope with factory
for (var i = 0; i < 5; i++) {
    buttons[i].addEventListener('click', createHandler(i));
}

function createHandler(index) {
    return function() {
        console.log(index);  // Each has its own index
    };
}

// FIX 3: IIFE (Immediately Invoked Function Expression)
for (var i = 0; i < 5; i++) {
    (function(index) {
        buttons[index].addEventListener('click', function() {
            console.log(index);
        });
    })(i);
}
```

4.7.2 Pitfall 2: this Binding in Closures

```
const obj = {
    name: 'MyObject',

    // BROKEN: Regular function loses this
    delayedLog() {
        setTimeout(function() {
            console.log(this.name);  // undefined! this is window/global
        }, 1000);
    },

    // FIX 1: Arrow function (inherits this)
    delayedLogArrow() {
        setTimeout(() => {
            console.log(this.name);  // 'MyObject'
        }, 1000);
    },

    // FIX 2: Save this reference
    delayedLogSaved() {
        const self = this;
        setTimeout(function() {
            console.log(self.name);  // 'MyObject'
        }, 1000);
    },
```

```
    // FIX 3: Bind
    delayedLogBound() {
        setTimeout(function() {
            console.log(this.name);  // 'MyObject'
        }.bind(this), 1000);
    }
};
```

4.7.3 Pitfall 3: Stale Closures (React)

```
// React example - common bug
function Counter() {
    const [count, setCount] = useState(0);

    useEffect(() => {
        const interval = setInterval(() => {
            console.log(count);  // Always logs 0!
            setCount(count + 1); // Always sets to 1!
        }, 1000);

        return () => clearInterval(interval);
    }, []);  // Empty deps = closure captures initial count (0)

    return <div>{count}</div>;
}

// FIX: Use functional update
function CounterFixed() {
    const [count, setCount] = useState(0);

    useEffect(() => {
        const interval = setInterval(() => {
            setCount(c => c + 1);  // c is always current
        }, 1000);

        return () => clearInterval(interval);
    }, []);

    return <div>{count}</div>;
}
```

4.8 Chapter Summary

Chapter Summary

Pattern	Description
Function Factory	A function that creates and returns other functions
Closure	A function that retains access to its outer scope
Private State	Variables inaccessible from outside, protected by closure
Module Pattern	IIFE that returns public API, hides private implementation
Closure Scope	All closures in same function share one context object

Key Insight

Closures let you create functions with "memory"—they remember configuration, accumulate state, and encapsulate private data.

4.9 Practice Exercises

Exercise 4.1: Counter Factory

Create a `createCounter` factory that returns an object with:

- `increment()` – increases count by 1, returns new count
- `decrement()` – decreases count by 1, returns new count
- `reset()` – resets to initial value, returns it
- `getCount()` – returns current count

Exercise

```
function createCounter(initialValue = 0) {
    // Your code here
}

// Usage:
const counter = createCounter(10);
counter.increment();  // 11
counter.increment();  // 12
counter.decrement();  // 11
counter.getCount();   // 11
counter.reset();      // 10
```

Exercise 4.2: Once Factory

Create a `once` factory that ensures a function can only be called once:

Exercise

```
function once(fn) {
    // Your code here
}

// Usage:
const initialize = once(() => {
    console.log('Initializing...');
    return 'initialized';
});

initialize();  // Logs 'Initializing...', returns 'initialized'
initialize();  // Does nothing, returns 'initialized' (cached)
initialize();  // Does nothing, returns 'initialized' (cached)
```

Exercise 4.3: Debounce Factory

Create a `debounce` factory that delays function execution until after a wait period of inactivity:

Exercise

```
function debounce(fn, waitMs) {
    // Your code here
}

// Usage:
const debouncedSearch = debounce((query) => {
    console.log('Searching for:', query);
}, 300);

// Rapid calls
debouncedSearch('h');
debouncedSearch('he');
debouncedSearch('hel');
debouncedSearch('hell');
debouncedSearch('hello');
// Only logs: 'Searching for: hello' (after 300ms of no calls)
```

Exercise 4.4: Private Stack

Create a stack data structure using closures with true private data:

Exercise

```
function createStack() {
    // Your code here
}

// Usage:
const stack = createStack();
stack.push(1);
stack.push(2);
stack.push(3);
stack.peek();    // 3 (doesn't remove)
stack.pop();     // 3
stack.pop();     // 2
stack.size();    // 1
stack.isEmpty(); // false

// These should not work:
stack.items;     // undefined (private!)
```

Chapter 5

Composition & Pipelines

Composition is the art of combining simple functions to build complex behavior. Instead of one large function, you create small, focused functions and connect them.

5.1 The Composition Concept

5.1.1 Mathematical Composition

In math, $f \circ g$ means "apply g, then apply f":

$$(f \circ g)(x) = f(g(x))$$

Read right-to-left: first g, then f.

5.1.2 In JavaScript

```
// Two simple functions
const addOne = x => x + 1;
const double = x => x * 2;

// Manual composition
const addOneThenDouble = x => double(addOne(x));

addOneThenDouble(5);  // double(addOne(5)) = double(6) = 12
```

5.1.3 The Problem with Nesting

As you add more functions, nesting becomes unreadable:

```
// Hard to read: inside-out
const result = format(validate(parse(sanitize(input))));

// What's the order? You have to read inside-out:
// 1. sanitize
// 2. parse
// 3. validate
// 4. format
```

5.2 compose: Right-to-Left Composition

`compose` creates a function that applies functions right-to-left:

```
// compose implementation
const compose = (...fns) => x => fns.reduceRight((acc, fn) => fn(acc), x);

// Usage
const addOne = x => x + 1;
const double = x => x * 2;
const square = x => x * x;

const computed = compose(square, double, addOne);
// Reads: square after double after addOne
// Executes: addOne → double → square

computed(5);  // square(double(addOne(5))) = square(double(6)) = square(12) = 144
```

5.2.1 How compose Works

```
const compose = (...fns) => x => fns.reduceRight((acc, fn) => fn(acc), x);

// Let's trace compose(square, double, addOne)(5):

// fns = [square, double, addOne]
// x = 5

// reduceRight starts from the RIGHT:
// Step 1: acc=5, fn=addOne → addOne(5) = 6
// Step 2: acc=6, fn=double → double(6) = 12
// Step 3: acc=12, fn=square → square(12) = 144

// Result: 144
```

> `compose` follows mathematical convention—function composition reads right-to-left. This is preferred in academic functional programming.

5.3 pipe: Left-to-Right Composition

`pipe` is the more intuitive version—functions apply left-to-right:

```
// pipe implementation
const pipe = (...fns) => x => fns.reduce((acc, fn) => fn(acc), x);

// Usage
const addOne = x => x + 1;
const double = x => x * 2;
const square = x => x * x;

const computed = pipe(addOne, double, square);
// Reads: addOne then double then square
// Executes: addOne → double → square
```

```
computed(5);  // 144 (same result, but reads naturally)
```

5.3.1 pipe is the Web Dev Standard

Most JavaScript libraries and frameworks prefer `pipe`:

- RxJS uses `pipe()`
- fp-ts uses `pipe()`
- Effect-TS uses `pipe()`
- Redux middleware flows left-to-right

Why? It reads like English: "Do this, then this, then this."

```
// pipe reads top-to-bottom, left-to-right
const processUser = pipe(
    validateInput,
    normalizeEmail,
    hashPassword,
    saveToDatabase
);

// compose reads bottom-to-top, right-to-left
const processUser = compose(
    saveToDatabase,
    hashPassword,
    normalizeEmail,
    validateInput
);
```

5.4 Building Pipelines

5.4.1 Data Transformation Pipeline

```
const pipe = (...fns) => x => fns.reduce((acc, fn) => fn(acc), x);

// Small, focused functions
const trim = str => str.trim();
const toLowerCase = str => str.toLowerCase();
const replaceSpaces = str => str.replace(/\s+/g, '-');
const removeSpecialChars = str => str.replace(/[^a-z0-9-]/g, '');

// Combine into pipeline
const slugify = pipe(
    trim,
    toLowerCase,
    replaceSpaces,
    removeSpecialChars
);

slugify('  Hello World! This is a TEST  ');
// 'hello-world-this-is-a-test'
```

5.4.2 Request Processing Pipeline

```
// Each function transforms the request object
const addTimestamp = req => ({
    ...req,
    timestamp: Date.now()
});

const validateAuth = req => {
    if (!req.headers.authorization) {
        throw new Error('Unauthorized');
    }
    return req;
};

const parseBody = req => ({
    ...req,
    body: JSON.parse(req.rawBody)
});

const sanitizeInput = req => ({
    ...req,
    body: {
        ...req.body,
        email: req.body.email?.toLowerCase().trim()
    }
});

// Pipeline
const processRequest = pipe(
    addTimestamp,
    validateAuth,
    parseBody,
    sanitizeInput
);

// Usage
const result = processRequest({
    headers: { authorization: 'Bearer xxx' },
    rawBody: '{"email": " ALICE@TEST.COM "}'
});
```

5.4.3 Array Processing Pipeline

```
const users = [
    { name: 'Alice', age: 25, active: true },
    { name: 'Bob', age: 17, active: true },
    { name: 'Charlie', age: 30, active: false },
    { name: 'Diana', age: 22, active: true }
];

// Functions that work on arrays
const filterActive = users => users.filter(u => u.active);
const filterAdults = users => users.filter(u => u.age >= 18);
const sortByAge = users => [...users].sort((a, b) => a.age - b.age);
```

```
const extractNames = users => users.map(u => u.name);

// Pipeline
const getActiveAdultNames = pipe(
    filterActive,
    filterAdults,
    sortByAge,
    extractNames
);

getActiveAdultNames(users);  // ['Diana', 'Alice']
```

5.5 The tap Utility: Debugging Pipelines

`tap` lets you peek at values without affecting the pipeline:

```
const tap = fn => value => {
    fn(value);
    return value;  // Pass through unchanged
};

// Usage: debug a pipeline
const processData = pipe(
    filterActive,
    tap(data => console.log('After filter:', data.length)),
    sortByAge,
    tap(data => console.log('After sort:', data)),
    extractNames,
    tap(names => console.log('Final:', names))
);
```

5.5.1 tap Variations

```
// Log with label
const tapLog = label => tap(x => console.log(label, x));

// Debugger breakpoint
const tapDebug = tap(() => debugger);

// Conditional tap
const tapIf = (predicate, fn) => value => {
    if (predicate(value)) fn(value);
    return value;
};

// Usage
const processData = pipe(
    filterActive,
    tapLog('After filter:'),
    tapIf(arr => arr.length === 0, () => console.warn('No active users!')),
    sortByAge,
    extractNames
);
```

5.6 Point-Free Style

Point-free (or tacit) style means defining functions without explicitly mentioning their arguments:

```
// Pointed (explicit argument)
const getLength = str => str.length;
const isEven = n => n % 2 === 0;

// Point-free
const getLength = str => str.length;  // Can't really avoid the arg here

// But with composition:
// Pointed
const getLengthAndDouble = str => double(str.length);

// Point-free (no mention of str)
const getLengthAndDouble = pipe(
    str => str.length,
    double
);
```

5.6.1 Point-Free with Higher-Order Utilities

```
// Helper to get property
const prop = key => obj => obj[key];

// Helper to call method
const method = (name, ...args) => obj => obj[name](...args);

// Now we can go point-free
const getName = prop('name');
const getEmail = prop('email');
const toUpperCase = method('toUpperCase');

// Point-free pipeline
const getUpperName = pipe(
    prop('name'),
    method('toUpperCase')
);

getUpperName({ name: 'alice', email: 'a@b.com' });  // 'ALICE'
```

5.6.2 When to Use Point-Free

Good: Simple, linear transformations

Good

```
// Clear and readable
const processName = pipe(trim, toLowerCase, capitalize);
```

Bad: Complex logic or when it hurts readability

× Bad

```
// This is too clever
const process = pipe(
    fork(join, head, tail),
    converge(multiply, [add(1), subtract(1)])
);
```

Good

```
// Just write it clearly
const process = x => {
    const a = x + 1;
    const b = x - 1;
    return a * b;
};
```

Guideline: If you have to think hard to understand it, don't use point-free.

5.7 Composing with Multiple Arguments

Basic `pipe` and `compose` work with single-argument functions. For multiple arguments, we need additional techniques (covered in Chapter 6: Currying).

For now, here's a multi-argument compose:

```
// First function can take multiple args, rest take one
const pipeWith = (...fns) => (...args) => {
    const [first, ...rest] = fns;
    return rest.reduce((acc, fn) => fn(acc), first(...args));
};

// Usage
const add = (a, b) => a + b;
const double = x => x * 2;
const square = x => x * x;

const compute = pipeWith(add, double, square);
compute(2, 3);  // square(double(add(2, 3))) = square(double(5)) = square(10) = 100
```

5.8 Real-World Composition Patterns

5.8.1 Pattern 1: Validation Pipeline

```
const pipe = (...fns) => x => fns.reduce((acc, fn) => fn(acc), x);

// Each validator returns { valid, value, error }
const createValidator = (test, errorMsg) => result => {
```

```
    if (!result.valid) return result;  // Short-circuit on first error

    if (!test(result.value)) {
        return { valid: false, value: result.value, error: errorMsg };
    }
    return result;
};

const notEmpty = createValidator(
    v => v.trim().length > 0,
    'Cannot be empty'
);

const minLength = n => createValidator(
    v => v.length >= n,
    `Must be at least ${n} characters`
);

const hasUppercase = createValidator(
    v => /[A-Z]/.test(v),
    'Must contain uppercase letter'
);

const hasNumber = createValidator(
    v => /[0-9]/.test(v),
    'Must contain a number'
);

// Compose validators
const validatePassword = pipe(
    notEmpty,
    minLength(8),
    hasUppercase,
    hasNumber
);

// Usage
const result = validatePassword({ valid: true, value: 'weak' });
// { valid: false, value: 'weak', error: 'Must be at least 8 characters' }

const result2 = validatePassword({ valid: true, value: 'StrongPass1' });
// { valid: true, value: 'StrongPass1', error: undefined }
```

5.8.2 Pattern 2: Async Pipeline

```
// Async pipe: awaits each step
const pipeAsync = (...fns) => async x => {
    let result = x;
    for (const fn of fns) {
        result = await fn(result);
    }
    return result;
};

// Usage
```

```
const processUser = pipeAsync(
    fetchUser,
    validateUser,
    enrichWithProfile,
    saveToCache
);

const user = await processUser(userId);
```

5.9 Chapter Summary

Chapter Summary

Function	Direction	Use Case
`compose`	Right-to-left (→)	Mathematical convention, store enhancers
`pipe`	Left-to-right (←)	Web dev standard, data pipelines
`tap`	Pass-through	Debugging, logging
`flow`	Left-to-right	Same as pipe (Lodash name)

Key Insight

Break complex transformations into small, focused functions. Combine them with `pipe` for readable, maintainable code.

5.10 Practice Exercises

Exercise 5.1: Build pipe and compose

Implement both from scratch:

Exercise

```
function pipe(...fns) {
    // Your code here
}

function compose(...fns) {
    // Your code here
}

// Test
const add1 = x => x + 1;
const mult2 = x => x * 2;
const sub3 = x => x - 3;

pipe(add1, mult2, sub3)(5);     // ((5 + 1) * 2) - 3 = 9
compose(sub3, mult2, add1)(5);  // Same result: 9
```

Exercise 5.2: String Processing Pipeline

Create a text processing pipeline that:

1. Trims whitespace
2. Converts to lowercase
3. Removes punctuation
4. Splits into words
5. Removes words shorter than 3 characters
6. Joins with hyphens

Exercise

```
const processText = pipe(
    // Your functions here
);

processText('  Hello, World! This is a TEST...  ');
// 'hello-world-this-test'
```

Chapter 6

Partial Application & Currying

Partial application and currying let you create specialized functions from general ones by pre-filling arguments.

6.1 The Problem: Repeated Arguments

```
// You keep passing the same first argument
fetchFromApi('https://api.github.com', '/users');
fetchFromApi('https://api.github.com', '/repos');
fetchFromApi('https://api.github.com', '/gists');

// Or the same configuration
formatDate(date1, 'YYYY-MM-DD', 'en-US');
formatDate(date2, 'YYYY-MM-DD', 'en-US');
formatDate(date3, 'YYYY-MM-DD', 'en-US');
```

What if you could create a specialized version with some arguments "locked in"?

6.2 Partial Application

Partial application creates a new function with some arguments pre-filled:

```
// General function
function greet(greeting, name) {
    return `${greeting}, ${name}!`;
}

// Partially apply the first argument
function sayHello(name) {
    return greet('Hello', name);
}

function sayHi(name) {
    return greet('Hi', name);
}

sayHello('Alice');  // 'Hello, Alice!'
sayHi('Bob');       // 'Hi, Bob!'
```

6.2.1 Using bind for Partial Application

JavaScript's `bind` can partially apply arguments:

```
function greet(greeting, name) {
    return `${greeting}, ${name}!`;
}

// bind(thisArg, ...args) - we use null for thisArg
const sayHello = greet.bind(null, 'Hello');
const sayGoodbye = greet.bind(null, 'Goodbye');

sayHello('Alice');    // 'Hello, Alice!'
sayGoodbye('Bob');    // 'Goodbye, Bob!'
```

6.2.2 A Generic partial Function

```
const partial = (fn, ...presetArgs) => {
    return (...laterArgs) => fn(...presetArgs, ...laterArgs);
};

// Usage
function createUser(role, department, name, email) {
    return { role, department, name, email };
}

// Create specialized functions
const createAdmin = partial(createUser, 'admin', 'IT');
const createEmployee = partial(createUser, 'employee', 'Sales');

createAdmin('Alice', 'alice@test.com');
// { role: 'admin', department: 'IT', name: 'Alice', email: 'alice@test.com' }

createEmployee('Bob', 'bob@test.com');
// { role: 'employee', department: 'Sales', name: 'Bob', email: 'bob@test.com' }
```

6.2.3 Web Dev Example: API Client

```
const partial = (fn, ...presetArgs) => (...laterArgs) => fn(...presetArgs,
  ↪  ...laterArgs);

async function apiRequest(baseUrl, method, endpoint, data = null) {
    const options = {
        method,
        headers: { 'Content-Type': 'application/json' }
    };

    if (data) options.body = JSON.stringify(data);

    const response = await fetch(`${baseUrl}${endpoint}`, options);
    return response.json();
}

// Create specialized functions
const githubApi = partial(apiRequest, 'https://api.github.com');
const githubGet = partial(githubApi, 'GET');
const githubPost = partial(githubApi, 'POST');
```

```
// Use them
const user = await githubGet('/users/octocat');
const gist = await githubPost('/gists', { files: {...} });
```

6.3 Currying

Currying transforms a function that takes multiple arguments into a sequence of functions that each take a single argument:

```
// Regular function
function add(a, b, c) {
    return a + b + c;
}
add(1, 2, 3);  // 6

// Curried version
function addCurried(a) {
    return function(b) {
        return function(c) {
            return a + b + c;
        };
    };
}
addCurried(1)(2)(3);  // 6
```

6.3.1 Arrow Function Syntax

```
// Same curried function with arrows
const addCurried = a => b => c => a + b + c;

addCurried(1)(2)(3);  // 6

// Can call incrementally
const add1 = addCurried(1);      // b => c => 1 + b + c
const add1and2 = add1(2);        // c => 1 + 2 + c
const result = add1and2(3);      // 6
```

6.3.2 The Difference: Partial vs Curry

Partial Application	Currying
Fix some arguments at once	Transform to single-arg chain
`partial(fn, a, b)(c, d)`	`curry(fn)(a)(b)(c)(d)`
Flexible grouping	One arg at a time

```
// Partial: fix multiple args at once
const partial = (fn, ...args) => (...more) => fn(...args, ...more);
```

```
const add5and6 = partial(add, 5, 6);  // Fix two args
add5and6(7);  // 18

// Curry: always one at a time
const addCurried = a => b => c => a + b + c;
addCurried(5)(6)(7);  // 18
```

6.4 Auto-Curry: Flexible Currying

Auto-curry lets you call with any number of arguments:

```
function curry(fn) {
    return function curried(...args) {
        if (args.length >= fn.length) {
            // Enough arguments: call the function
            return fn(...args);
        } else {
            // Not enough: return function that collects more
            return (...more) => curried(...args, ...more);
        }
    };
}

// Usage
const add = (a, b, c) => a + b + c;
const curriedAdd = curry(add);

// All of these work:
curriedAdd(1, 2, 3);    // 6 (all at once)
curriedAdd(1)(2)(3);    // 6 (one at a time)
curriedAdd(1, 2)(3);    // 6 (mixed)
curriedAdd(1)(2, 3);    // 6 (mixed)
```

6.4.1 How Auto-Curry Works

```
function curry(fn) {
    return function curried(...args) {
        // fn.length is the number of declared parameters
        if (args.length >= fn.length) {
            return fn(...args);  // Call with all args
        }
        return (...more) => curried(...args, ...more);  // Collect more
    };
}

// Trace curry(add)(1)(2)(3):
// Call 1: args=[1], 1 < 3, return (...more) => curried(1, ...more)
// Call 2: args=[1,2], 2 < 3, return (...more) => curried(1, 2, ...more)
// Call 3: args=[1,2,3], 3 >= 3, return add(1, 2, 3) = 6
```

6.5 Currying in Practice

6.5.1 Data-Last for Piping

Curried functions work beautifully with `pipe` when data comes last:

```
const curry = fn => function curried(...args) {
    return args.length >= fn.length
        ? fn(...args)
        : (...more) => curried(...args, ...more);
};

// Curried utilities (data-last)
const map = curry((fn, array) => array.map(fn));
const filter = curry((predicate, array) => array.filter(predicate));
const reduce = curry((fn, initial, array) => array.reduce(fn, initial));

// Now they compose beautifully
const pipe = (...fns) => x => fns.reduce((acc, fn) => fn(acc), x);

const processNumbers = pipe(
    filter(n => n > 0),           // Keep positives
    map(n => n * 2),              // Double them
    reduce((sum, n) => sum + n, 0) // Sum them
);

processNumbers([-1, 2, -3, 4, 5]);  // (2 + 4 + 5) * 2 = 22
```

6.5.2 Web Dev Example: Validation

```
const curry = fn => (...args) =>
    args.length >= fn.length ? fn(...args) : curry(fn.bind(null, ...args));

// Validators return { valid, error } or just boolean
const createRule = curry((test, errorMsg, value) => ({
    valid: test(value),
    error: test(value) ? null : errorMsg,
    value
}));

const minLength = n => createRule(
    v => v.length >= n,
    `Must be at least ${n} characters`
);

const maxLength = n => createRule(
    v => v.length <= n,
    `Must be at most ${n} characters`
);

const matchesPattern = (regex, msg) => createRule(
    v => regex.test(v),
    msg
);

// Compose validators
const validateAll = (...validators) => value => {
```

```
    for (const validate of validators) {
        const result = validate(value);
        if (!result.valid) return result;
    }
    return { valid: true, error: null, value };
};

const validateUsername = validateAll(
    minLength(3),
    maxLength(20),
    matchesPattern(/^[a-zA-Z0-9_]+$/, 'Only letters, numbers, underscore')
);

validateUsername('ab');  // { valid: false, error: 'Must be at least 3 characters' }
validateUsername('alice_123');  // { valid: true, error: null, value: 'alice_123' }
```

6.6 Partial Application vs Currying: When to Use Each

6.6.1 Use Partial Application When:

- You want to fix several arguments at once
- Working with existing non-curried functions
- The argument order doesn't match your needs

```
// Fix multiple args at once
const logError = partial(console.log, '[ERROR]', new Date().toISOString());
logError('Something went wrong');  // [ERROR] 2024-01-15T10:30:00.000Z Something went
  ↪ wrong

// Working with existing functions
const parseBase10 = partial(parseInt, undefined, 10);
['1', '2', '3'].map(parseBase10);  // [1, 2, 3] not [1, NaN, NaN]
```

6.6.2 Use Currying When:

- Building composable utilities
- Creating point-free pipelines
- Designing an API meant for composition

```
// Composable utilities
const add = curry((a, b) => a + b);
const multiply = curry((a, b) => a * b);

const add5 = add(5);
const times2 = multiply(2);

const transform = pipe(add5, times2);
transform(10);  // (10 + 5) * 2 = 30
```

6.7 Right-to-Left Partial Application

Sometimes you need to fill arguments from the right:

```
const partialRight = (fn, ...presetArgs) => {
    return (...laterArgs) => fn(...laterArgs, ...presetArgs);
};

// Example: parseInt has (string, radix) signature
const parseInt10 = partialRight(parseInt, 10);

['1', '2', '3'].map(parseInt10);  // [1, 2, 3]

// Compare to the problem:
['1', '2', '3'].map(parseInt);    // [1, NaN, NaN] - radix gets index!
```

6.8 Chapter Summary

Chapter Summary

Technique	Definition	Use Case
Partial Application	Pre-fill some arguments	Specialize existing functions
Currying	Transform to single-arg chain	Composable utilities
Auto-Curry	Flexible argument collection	Best of both worlds
Data-Last	Data argument comes last	Enables piping

Key Insight

Currying and partial application let you build specialized tools from general ones. Combined with `pipe`, they enable powerful, readable data transformations.

6.9 Practice Exercises

Exercise 6.1: Implement curry

Exercise

```
function curry(fn) {
    // Your code here
}

// Test
const add = (a, b, c) => a + b + c;
const curried = curry(add);

console.log(curried(1)(2)(3));    // 6
console.log(curried(1, 2)(3));    // 6
console.log(curried(1)(2, 3));    // 6
console.log(curried(1, 2, 3));    // 6
```

Exercise 6.2: Implement partial and partialRight

Exercise

```
function partial(fn, ...presetArgs) {
    // Your code here
}

function partialRight(fn, ...presetArgs) {
    // Your code here
}

// Test
const greet = (greeting, name, punct) => `${greeting}, ${name}${punct}`;

const sayHello = partial(greet, 'Hello');
console.log(sayHello('Alice', '!'));  // 'Hello, Alice!'

const greetBob = partialRight(greet, 'Bob', '!');
console.log(greetBob('Hi'));  // 'Hi, Bob!'
```

Chapter 7

The Pipeline Injection Protocol

This chapter presents a systematic algorithm for converting imperative code to functional pipelines. Follow these steps to refactor any loop-based code.

7.1 The 4-Phase Protocol

Phase 1: Isolate State

Identify mutable state that accumulates results.

Phase 2: Extract Predicates

Turn `if` conditions into named pure functions.

Phase 3: Decouple Transformations

Turn mutations into pure transformation functions.

Phase 4: Compose Pipeline

Combine using `filter`, `map`, `reduce`.

7.2 Phase 1: Isolate State

Goal: Find the mutable variables that accumulate results.

7.2.1 Pattern Recognition

Look for:

```
let result = [];            // Accumulator array
let total = 0;              // Accumulator number
let found = null;           // Search result
let isValid = true;         // Flag
```

7.2.2 Example

```
// BEFORE: Find the state
function getActiveEmails(users) {
    const emails = [];  // ← STATE: accumulator
    for (let i = 0; i < users.length; i++) {
        if (users[i].isActive) {
            if (users[i].email) {
                emails.push(users[i].email.toLowerCase());
            }
        }
    }
    return emails;
}
```

Identified State: `emails` array is the accumulator.

7.2.3 State Types and Their HOF Equivalents

State Pattern	HOF Replacement
`results.push(item)`	`map` or `filter`
`total += value`	`reduce`
`found = item; break;`	`find`
`isValid = false; break;`	`some` or `every`
`count++`	`filter().length` or `reduce`

7.3 Phase 2: Extract Predicates

Goal: Turn `if` conditions into named boolean functions.

7.3.1 Pattern Recognition

Every `if (condition)` becomes a predicate:

```
// BEFORE
if (users[i].isActive) { ... }
if (users[i].email) { ... }

// AFTER
const isActive = user => user.isActive;
const hasEmail = user => user.email != null;
```

7.3.2 Example Continued

```
// Extract predicates from the conditions
const isActive = user => user.isActive;
const hasEmail = user => user.email != null;

// The loop conditions are now named functions
```

7.3.3 Combining Predicates

```
// Multiple conditions can be combined
const isActiveWithEmail = user => isActive(user) && hasEmail(user);

// Or kept separate for reusability
// filter(isActive).filter(hasEmail)
```

7.3.4 Common Predicate Patterns

```
// Null/undefined checks
const exists = x => x != null;
const hasProperty = prop => obj => obj[prop] != null;

// Comparisons
const isGreaterThan = n => x => x > n;
const isLessThan = n => x => x < n;
const equals = target => x => x === target;

// String checks
const startsWith = prefix => str => str.startsWith(prefix);
const contains = substr => str => str.includes(substr);
const matchesPattern = regex => str => regex.test(str);

// Object checks
const hasRole = role => user => user.role === role;
const isActive = user => user.active === true;
const belongsTo = dept => emp => emp.department === dept;
```

7.4 Phase 3: Decouple Transformations

Goal: Turn mutations into pure functions that return new values.

7.4.1 Pattern Recognition

Look for:

```
item.prop = value;            // Property mutation
items.push(transform(item)); // Transform + push
result = process(item);       // Assignment after processing
```

7.4.2 Example Continued

```
// BEFORE (inside loop)
emails.push(users[i].email.toLowerCase());

// AFTER
const extractEmail = user => user.email;
const normalizeEmail = email => email.toLowerCase();

// Or combined:
const getEmail = user => user.email.toLowerCase();
```

7.4.3 Common Transformation Patterns

```
// Property extraction
const prop = key => obj => obj[key];
const props = (...keys) => obj => keys.map(k => obj[k]);

// Object reshaping
const pick = (...keys) => obj =>
    keys.reduce((acc, k) => ({ ...acc, [k]: obj[k] }), {});

const omit = (...keys) => obj =>
    Object.keys(obj)
        .filter(k => !keys.includes(k))
        .reduce((acc, k) => ({ ...acc, [k]: obj[k] }), {});

// Value transformations
const toUpperCase = str => str.toUpperCase();
const toLowerCase = str => str.toLowerCase();
const trim = str => str.trim();

// Number transformations
const add = n => x => x + n;
const multiply = n => x => x * n;
const clamp = (min, max) => x => Math.max(min, Math.min(max, x));

// Object transformations
const withDefaults = defaults => obj => ({ ...defaults, ...obj });
const rename = (oldKey, newKey) => obj => {
    const { [oldKey]: value, ...rest } = obj;
    return { ...rest, [newKey]: value };
};
```

7.5 Phase 4: Compose Pipeline

Goal: Combine predicates and transformations into a pipeline.

7.5.1 The Mapping

Imperative	Functional
`if (condition)` inside loop	`.filter(predicate)`
`transform(item)`	`.map(transformer)`
`accumulator += value`	`.reduce(fn, initial)`
`if (...) { break; }`	`.find(predicate)`

7.5.2 Example Completed

× Bad

```
// BEFORE: Imperative
function getActiveEmails(users) {
    const emails = [];
    for (let i = 0; i < users.length; i++) {
        if (users[i].isActive) {
            if (users[i].email) {
                emails.push(users[i].email.toLowerCase());
            }
        }
    }
    return emails;
}
```

Good

```
// AFTER: Functional Pipeline
const isActive = user => user.isActive;
const hasEmail = user => user.email != null;
const getEmail = user => user.email.toLowerCase();

function getActiveEmails(users) {
    return users
        .filter(isActive)
        .filter(hasEmail)
        .map(getEmail);
}

// Or ultra-concise (but less readable):
const getActiveEmails = users => users
    .filter(u => u.isActive && u.email)
    .map(u => u.email.toLowerCase());
```

7.6 Complete Worked Examples

7.6.1 Example 1: Sum of Squares of Evens

× Bad

```
// BEFORE: Imperative
function sumOfSquaresOfEvens(numbers) {
    let sum = 0;  // STATE
    for (let i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 === 0) {  // PREDICATE
            sum += numbers[i] * numbers[i];  // TRANSFORM + ACCUMULATE
        }
    }
    return sum;
}
```

Good

```
// PHASE 1: State = sum (accumulator) → reduce
// PHASE 2: Predicate = isEven
// PHASE 3: Transform = square
// PHASE 4: Compose

const isEven = n => n % 2 === 0;
const square = n => n * n;

function sumOfSquaresOfEvens(numbers) {
    return numbers
        .filter(isEven)
        .map(square)
        .reduce((sum, n) => sum + n, 0);
}

// Test
sumOfSquaresOfEvens([1, 2, 3, 4, 5, 6]);  // 4 + 16 + 36 = 56
```

7.6.2 Example 2: Find First Admin

× Bad

```
// BEFORE: Imperative
function findFirstAdmin(users) {
    let admin = null;  // STATE: search result
    for (let i = 0; i < users.length; i++) {
        if (users[i].role === 'admin') {  // PREDICATE
            admin = users[i];  // Found!
            break;  // Early exit
        }
    }
    return admin;
}
```

Good

```
// Early exit + single result → find

const isAdmin = user => user.role === 'admin';

function findFirstAdmin(users) {
    return users.find(isAdmin);
}
```

7.6.3 Example 3: Validate All Fields

× Bad

```
// BEFORE: Imperative
function validateForm(fields) {
    let isValid = true;  // STATE: flag
    const errors = [];
    for (let i = 0; i < fields.length; i++) {
        if (fields[i].required && !fields[i].value) {  // PREDICATE
            isValid = false;
            errors.push(`${fields[i].name} is required`);
        }
    }
    return { isValid, errors };
}
```

Good

```
// Multiple concerns: validation check + error collection

const isRequiredAndEmpty = field => field.required && !field.value;
const toErrorMessage = field => `${field.name} is required`;

function validateForm(fields) {
    const invalidFields = fields.filter(isRequiredAndEmpty);
    return {
        isValid: invalidFields.length === 0,
        errors: invalidFields.map(toErrorMessage)
    };
}
```

7.6.4 Example 4: Group By Category

× Bad

```
// BEFORE: Imperative
function groupByCategory(products) {
    const groups = {};  // STATE: object accumulator
    for (let i = 0; i < products.length; i++) {
        const category = products[i].category;  // TRANSFORM (extract)
        if (!groups[category]) {
            groups[category] = [];
```

```
        }
        groups[category].push(products[i]);  // ACCUMULATE
    }
    return groups;
}
```

Good

```
// Object accumulator → reduce

function groupByCategory(products) {
    return products.reduce((groups, product) => {
        const category = product.category;
        return {
            ...groups,
            [category]: [...(groups[category] || []), product]
        };
    }, {});
}

// More efficient version (mutation in reduce is OK):
function groupByCategory(products) {
    return products.reduce((groups, product) => {
        const category = product.category;
        if (!groups[category]) groups[category] = [];
        groups[category].push(product);
        return groups;
    }, {});
}
```

7.6.5 Example 5: Nested Loop Flattening

× Bad

```
// BEFORE: Imperative (nested loops)
function getAllActiveMembers(teams) {
    const activeMembers = [];
    for (let i = 0; i < teams.length; i++) {
        for (let j = 0; j < teams[i].members.length; j++) {
            if (teams[i].members[j].active) {
                activeMembers.push(teams[i].members[j]);
            }
        }
    }
    return activeMembers;
}
```

Good

```
// Nested loop accessing child array → flatMap

const isActive = member => member.active;
const getMembers = team => team.members;
```

```
function getAllActiveMembers(teams) {
    return teams
        .flatMap(getMembers)
        .filter(isActive);
}
```

7.7 Decision Flowchart

```
START: Analyze the loop │ ├── Does it build an array? │ ├── By filtering items?
→ filter() │ ├── By transforming items? → map() │ └── By both? → filter().map()
or flatMap() │ ├── Does it build an object? │ ├── Key-value lookup? → reduce() to
object │ ├── Grouping? → reduce() with array values │ └── Counting? → reduce()
with number values │ ├── Does it calculate a single value? │ └── reduce() │ ├──
Does it search for one item? │ ├── Need the item? → find() │ └── Need the index?
→ findIndex() │ ├── Does it check a condition? │ ├── "Does any match?" → some() │
└── "Do all match?" → every() │ └── Does it have nested loops? └── flatMap() for the
inner arrays
```

7.8 Anti-Patterns to Avoid

7.8.1 Anti-Pattern 1: Reduce-Spread ($O(n^2)$)

× Bad

```
// BAD: Creates new object on every iteration
const byId = items.reduce((acc, item) => ({
    ...acc,
    [item.id]: item
}), {});
```

Good

```
// GOOD: Mutate accumulator (OK in reduce)
const byId = items.reduce((acc, item) => {
    acc[item.id] = item;
    return acc;
}, {});

// ALSO GOOD: Use Object.fromEntries
const byId = Object.fromEntries(
    items.map(item => [item.id, item])
);
```

7.8.2 Anti-Pattern 2: Filter Then Length

× Bad

```
// BAD: Creates intermediate array just to count
const count = items.filter(x => x.active).length;
```

Good

```
// GOOD: Use reduce to count directly
const count = items.reduce((n, x) => x.active ? n + 1 : n, 0);

// ALSO GOOD: If you need the items too, filter is fine
const activeItems = items.filter(x => x.active);
const count = activeItems.length;
```

7.8.3 Anti-Pattern 3: Map for Side Effects

× Bad

```
// BAD: Using map for side effects (returns unused array)
users.map(user => {
    sendEmail(user);  // Side effect!
});
```

Good

```
// GOOD: Use forEach for side effects
users.forEach(user => {
    sendEmail(user);
});

// ALSO GOOD: for...of loop
for (const user of users) {
    sendEmail(user);
}
```

7.9 When NOT to Convert

Sometimes imperative is better:

7.9.1 1. Performance-Critical Code

```
// Array methods have overhead. For millions of items:
// Imperative
let sum = 0;
for (let i = 0; i < hugeArray.length; i++) {
    sum += hugeArray[i];
}
```

```
// May be faster than:
const sum = hugeArray.reduce((a, b) => a + b, 0);
```

7.9.2 2. Complex Control Flow

```
// Hard to express functionally
for (let i = 0; i < items.length; i++) {
    if (condition1) {
        doA();
        continue;
    }
    if (condition2) {
        doB();
        if (condition3) break;
    }
    doC();
}
```

7.9.3 3. Early Exit on Find

```
// find() is good, but for complex exit logic:
for (const item of items) {
    const result = complexOperation(item);
    if (result.success) {
        return result;  // Complex early return
    }
    if (result.fatal) {
        throw new Error(result.message);
    }
}
```

7.9.4 4. Mutation is Intentional

```
// When you actually need to modify in place
for (const item of items) {
    item.processed = true;
    item.timestamp = Date.now();
}
```

7.10 Chapter Summary

Chapter Summary

7.10.1 The 4-Phase Protocol

Phase	Action	Look For
1	Isolate State	`let result = []`, `let sum = 0`
2	Extract Predicates	`if (condition)`
3	Decouple Transforms	`item.x = y`, `push(transform(x))`
4	Compose Pipeline	`filter`, `map`, `reduce`

7.10.2 Quick Reference

Pattern	Description
Loop + if + push	`filter`
Loop + transform + push	`map`
Loop + accumulate value	`reduce`
Nested loops	`flatMap`
Loop + break on find	`find` / `findIndex`
Loop + flag check	`some` / `every`

Key Insight

Most loops follow predictable patterns. Recognize the pattern, apply the corresponding HOF, and your code becomes more declarative and maintainable.

7.11 Practice Exercises

Exercise 7.1: Refactor to Pipeline

Convert this imperative code:

Exercise

```
function processOrders(orders) {
    const result = [];
    for (let i = 0; i < orders.length; i++) {
        if (orders[i].status === 'completed') {
            if (orders[i].total > 100) {
                result.push({
                    id: orders[i].id,
                    total: orders[i].total,
                    discountedTotal: orders[i].total * 0.9
                });
            }
        }
    }
    return result;
}
```

Exercise 7.2: Nested Loop Refactoring

Convert this:

Exercise

```
function getSkillsFromTeams(teams) {
    const skills = [];
    for (let i = 0; i < teams.length; i++) {
        for (let j = 0; j < teams[i].members.length; j++) {
            for (let k = 0; k < teams[i].members[j].skills.length; k++) {
                if (!skills.includes(teams[i].members[j].skills[k])) {
                    skills.push(teams[i].members[j].skills[k]);
                }
            }
        }
    }
    return skills;
}
```

Part II Summary

You now have intermediate techniques:

Concept	What You Learned
Function Factories	Create specialized functions from configuration
Closures	Functions that remember their scope
Private State	True encapsulation via closures
compose/pipe	Combine functions into pipelines
tap	Debug without breaking the pipeline
Partial Application	Pre-fill some arguments
Currying	Transform to single-argument chain
Pipeline Protocol	Systematic imperative-to-functional conversion

What's Next

In **Part III: Real-World Web Development**, you'll apply these patterns to:

- Async JavaScript and Promises (Chapter 8)
- React hooks and components (Chapter 9)
- State management with Redux (Chapter 10)
- Node.js middleware and APIs (Chapter 11)

Part III

Real-World Web Development

Chapter 8

HOFs in Async JavaScript

Asynchronous programming is where higher-order functions truly shine. Promises, async/await, and event-driven patterns are all built on HOF foundations.

8.1 Promises Are Built on HOFs

A Promise's `.then()`, `.catch()`, and `.finally()` are all higher-order functions—they take callbacks and return new Promises.

```
fetch('/api/user')
    .then(response => response.json())  // HOF: takes callback
    .then(user => user.name)            // HOF: takes callback
    .catch(error => console.error(error))  // HOF: takes callback
    .finally(() => hideLoader());       // HOF: takes callback
```

8.1.1 The Promise Chain as a Pipeline

Think of Promise chains as async pipelines:

```
// Sync pipeline
const result = pipe(
    getData,
    transform,
    validate,
    save
)(input);

// Async pipeline (Promise chain)
const result = getData(input)
    .then(transform)
    .then(validate)
    .then(save);
```

8.1.2 Passing Functions Directly

When your callback just passes its argument to another function, you can pass the function directly:

× Bad

```
// Verbose
fetch('/api/users')
    .then(response => response.json())
    .then(users => console.log(users));
```

Good

```
// When function signatures match, pass directly
fetch('/api/users')
    .then(response => response.json())
    .then(console.log);  // console.log receives users directly
```

8.2 Promise.all with map

`Promise.all` + `map` is the pattern for parallel async operations:

```
const userIds = [1, 2, 3, 4, 5];

// Fetch all users in parallel
const fetchUser = id => fetch(`/api/users/${id}`).then(r => r.json());

const users = await Promise.all(
    userIds.map(fetchUser)
);
// All 5 requests run simultaneously
```

8.2.1 Step by Step

1. map creates array of Promises

```
const promises = userIds.map(id => fetchUser(id));
// [Promise<User1>, Promise<User2>, Promise<User3>]
```

2. Promise.all waits for all to resolve

```
const users = await Promise.all(promises);
// [User1, User2, User3]
```

8.2.2 Real-World Example: Enriching Data

Parallel Data Enrichment

```
async function enrichOrders(orders) {
    return Promise.all(
        orders.map(async order => {
            // Fetch related data in parallel for each order
            const [customer, products] = await Promise.all([
                fetchCustomer(order.customerId),
                Promise.all(order.productIds.map(fetchProduct))
            ]);

            return {
                ...order,
                customer,
                products
            };
        })
    );
}
```

8.2.3 Handling Partial Failures

```
// Promise.all fails fast - one rejection fails everything
// Use Promise.allSettled for graceful handling

const results = await Promise.allSettled(
    urls.map(url => fetch(url).then(r => r.json()))
);

const successful = results
    .filter(r => r.status === 'fulfilled')
    .map(r => r.value);

const failed = results
    .filter(r => r.status === 'rejected')
    .map(r => r.reason);

console.log(`${successful.length} succeeded, ${failed.length} failed`);
```

8.3 Sequential vs Parallel Execution

8.3.1 Parallel: Promise.all + map

```
// All requests start immediately, complete in any order
const results = await Promise.all(items.map(processAsync));
```

8.3.2 Sequential: for...of with await

```
// Each request waits for the previous one
const results = [];
for (const item of items) {
    const result = await processAsync(item);
    results.push(result);
}
```

8.3.3 Sequential: reduce with async

```
// Sequential using reduce
const results = await items.reduce(async (accPromise, item) => {
    const acc = await accPromise;
    const result = await processAsync(item);
    return [...acc, result];
}, Promise.resolve([]));
```

8.3.4 When to Use Each

Pattern	Use When
Parallel (`Promise.all`)	Independent operations, maximize speed
Sequential (`for...of`)	Order matters, rate limiting, dependencies
Controlled (`p-limit`)	Parallel but with concurrency limit

8.4 Controlled Concurrency

Sometimes you need parallel execution but with limits (e.g., API rate limits):

Concurrency Limiter HOF

```
function createLimiter(concurrency) {
    let active = 0;
    const queue = [];

    const runNext = () => {
        if (queue.length === 0 || active >= concurrency) return;

        active++;
        const { fn, resolve, reject } = queue.shift();

        fn()
            .then(resolve)
            .catch(reject)
            .finally(() => {
                active--;
                runNext();
            });
```

```
    };

    return (fn) => new Promise((resolve, reject) => {
        queue.push({ fn, resolve, reject });
        runNext();
    });
}

// Usage
const limit = createLimiter(3);  // Max 3 concurrent

const results = await Promise.all(
    urls.map(url => limit(() => fetch(url)))
);
// Only 3 requests active at any time
```

For production applications, use battle-tested libraries like `p-limit` which handle edge cases and provide additional features.

8.5 Async Higher-Order Functions

8.5.1 Async Map

```
// Sequential async map
async function asyncMap(array, asyncFn) {
    const results = [];
    for (const item of array) {
        results.push(await asyncFn(item));
    }
    return results;
}

// Parallel async map
async function asyncMapParallel(array, asyncFn) {
    return Promise.all(array.map(asyncFn));
}

// Usage
const users = await asyncMap(userIds, fetchUser);  // Sequential
const users = await asyncMapParallel(userIds, fetchUser);  // Parallel
```

8.5.2 Async Filter

```
// Filter with async predicate
async function asyncFilter(array, asyncPredicate) {
    const results = await Promise.all(
        array.map(async item => ({
            item,
            keep: await asyncPredicate(item)
```

```
        }))
    );
    return results
        .filter(({ keep }) => keep)
        .map(({ item }) => item);
}

// Usage: Filter users who have valid subscriptions
const activeUsers = await asyncFilter(users, async user => {
    const subscription = await fetchSubscription(user.id);
    return subscription.active;
});
```

8.5.3 Async Find

```
// Find with async predicate (sequential - stops early)
async function asyncFind(array, asyncPredicate) {
    for (const item of array) {
        if (await asyncPredicate(item)) {
            return item;
        }
    }
    return undefined;
}

// Usage
const validServer = await asyncFind(servers, async server => {
    try {
        const health = await checkHealth(server);
        return health.status === 'ok';
    } catch {
        return false;
    }
});
```

8.5.4 Async Reduce

```
// Sequential async reduce
async function asyncReduce(array, asyncReducer, initialValue) {
    let accumulator = initialValue;
    for (const item of array) {
        accumulator = await asyncReducer(accumulator, item);
    }
    return accumulator;
}

// Usage: Process files sequentially, accumulating results
const report = await asyncReduce(
    files,
    async (report, file) => {
        const content = await readFile(file);
        const analysis = await analyzeContent(content);
        return {
```

```
            ...report,
            [file]: analysis
        };
    },
    {}
);
```

8.6 Retry Logic as HOF

Wrap any async function with retry capability:

Basic Retry Wrapper

```
function withRetry(asyncFn, maxRetries = 3, delay = 1000) {
    return async function(...args) {
        let lastError;

        for (let attempt = 1; attempt ≤ maxRetries; attempt++) {
            try {
                return await asyncFn(...args);
            } catch (error) {
                lastError = error;
                console.log(`Attempt ${attempt} failed:`, error.message);

                if (attempt < maxRetries) {
                    await sleep(delay);
                }
            }
        }

        throw lastError;
    };
}

const sleep = ms => new Promise(resolve => setTimeout(resolve, ms));

// Usage
const fetchWithRetry = withRetry(fetch, 3, 1000);
const response = await fetchWithRetry('/api/data');
```

8.6.1 Exponential Backoff

```
function withExponentialBackoff(asyncFn, maxRetries = 5, baseDelay = 100) {
    return async function(...args) {
        let lastError;

        for (let attempt = 0; attempt < maxRetries; attempt++) {
            try {
                return await asyncFn(...args);
            } catch (error) {
                lastError = error;

                if (attempt < maxRetries - 1) {
```

```
                    const delay = baseDelay * Math.pow(2, attempt);
                    const jitter = Math.random() * delay * 0.1;  // 10% jitter
                    await sleep(delay + jitter);
                }
            }
        }

        throw lastError;
    };
}

// Usage
const resilientFetch = withExponentialBackoff(fetch);
// Retries: 100ms, 200ms, 400ms, 800ms, 1600ms (with jitter)
```

8.6.2 Conditional Retry

```
function withConditionalRetry(asyncFn, shouldRetry, maxRetries = 3) {
    return async function(...args) {
        let lastError;

        for (let attempt = 1; attempt <= maxRetries; attempt++) {
            try {
                return await asyncFn(...args);
            } catch (error) {
                lastError = error;

                // Check if we should retry this error
                if (!shouldRetry(error) || attempt === maxRetries) {
                    throw error;
                }

                await sleep(1000 * attempt);
            }
        }

        throw lastError;
    };
}

// Only retry on network errors or 5xx status
const isRetryable = error =>
    error.name === 'NetworkError' ||
    (error.status && error.status >= 500);

const smartFetch = withConditionalRetry(fetch, isRetryable);
```

8.7 Debounce and Throttle

8.7.1 Debounce: Wait for Pause

Debounce delays execution until after a pause in calls:

```
function debounce(fn, delay) {
    let timeoutId;

    return function(...args) {
        clearTimeout(timeoutId);

        timeoutId = setTimeout(() => {
            fn.apply(this, args);
        }, delay);
    };
}

// Usage: Search input
const searchInput = document.getElementById('search');
const handleSearch = debounce(async (query) => {
    const results = await searchApi(query);
    displayResults(results);
}, 300);

searchInput.addEventListener('input', (e) => {
    handleSearch(e.target.value);
});
```

Debounce is perfect for search inputs, auto-save features, and window resize handlers where you want to wait for the user to finish their action.

8.7.2 Debounce with Promise

```
function debounceAsync(fn, delay) {
    let timeoutId;
    let pendingPromise = null;
    let resolve, reject;

    return function(...args) {
        clearTimeout(timeoutId);

        if (!pendingPromise) {
            pendingPromise = new Promise((res, rej) => {
                resolve = res;
                reject = rej;
            });
        }

        timeoutId = setTimeout(async () => {
            try {
                const result = await fn.apply(this, args);
                resolve(result);
            } catch (error) {
                reject(error);
            } finally {
                pendingPromise = null;
            }
```

```
        }, delay);

        return pendingPromise;
    };
}

// Usage
const debouncedSearch = debounceAsync(searchApi, 300);
const results = await debouncedSearch('query');  // Returns promise
```

8.7.3 Throttle: Rate Limit

Throttle ensures function runs at most once per interval:

```
function throttle(fn, interval) {
    let lastTime = 0;
    let timeoutId;

    return function(...args) {
        const now = Date.now();

        if (now - lastTime ≥ interval) {
            lastTime = now;
            fn.apply(this, args);
        } else {
            // Schedule for end of interval
            clearTimeout(timeoutId);
            timeoutId = setTimeout(() => {
                lastTime = Date.now();
                fn.apply(this, args);
            }, interval - (now - lastTime));
        }
    };
}

// Usage: Scroll handler
const handleScroll = throttle(() => {
    console.log('Scroll position:', window.scrollY);
}, 100);

window.addEventListener('scroll', handleScroll);
```

Key Insight

Debounce vs Throttle: Debounce waits for a pause (good for search), while throttle runs at regular intervals (good for scroll events). Choose based on whether you want the last call or regular sampling.

8.8 Timeout Wrapper

Add timeout to any async function:

```
function withTimeout(asyncFn, ms) {
    return async function(...args) {
        const timeoutPromise = new Promise((_, reject) => {
            setTimeout(() => reject(new Error(`Timeout after ${ms}ms`)), ms);
        });

        return Promise.race([
            asyncFn(...args),
            timeoutPromise
        ]);
    };
}

// Usage
const quickFetch = withTimeout(fetch, 5000);

try {
    const response = await quickFetch('/api/slow-endpoint');
} catch (error) {
    if (error.message.includes('Timeout')) {
        console.log('Request took too long');
    }
}
```

8.8.1 With AbortController

```
function withAbortableTimeout(asyncFn, ms) {
    return async function(...args) {
        const controller = new AbortController();
        const timeoutId = setTimeout(() => controller.abort(), ms);

        try {
            // Pass signal to the function if it supports it
            const result = await asyncFn(...args, { signal: controller.signal });
            clearTimeout(timeoutId);
            return result;
        } catch (error) {
            clearTimeout(timeoutId);
            if (error.name === 'AbortError') {
                throw new Error(`Timeout after ${ms}ms`);
            }
            throw error;
        }
    };
}

// Usage with fetch
const fetchWithTimeout = withAbortableTimeout(
    (url, options = {}) => fetch(url, options),
    5000
);
```

8.9 Async Pipeline

Build async pipelines that process data through stages:

```
const pipeAsync = (...fns) => async (input) => {
    let result = input;
    for (const fn of fns) {
        result = await fn(result);
    }
    return result;
};

// Usage
const processUser = pipeAsync(
    validateInput,
    fetchUserFromDb,
    enrichWithPermissions,
    formatForResponse
);

const user = await processUser({ id: 123 });
```

8.9.1 With Error Handling

```
const pipeAsyncSafe = (...fns) => async (input) => {
    let result = input;

    for (const fn of fns) {
        try {
            result = await fn(result);
        } catch (error) {
            return { success: false, error, failedAt: fn.name };
        }
    }

    return { success: true, data: result };
};

// Usage
const result = await pipeAsyncSafe(
    validateInput,
    fetchUserFromDb,
    enrichWithPermissions
)({ id: 123 });

if (!result.success) {
    console.error(`Pipeline failed at ${result.failedAt}:`, result.error);
}
```

8.10 Event Handling Patterns

8.10.1 Once: Execute Only First Time

```
function once(fn) {
    let called = false;
    let result;
```

```
    return function(...args) {
        if (!called) {
            called = true;
            result = fn.apply(this, args);
        }
        return result;
    };
}

// Usage
const initializeOnce = once(() => {
    console.log('Initializing...');
    return setupApplication();
});

initializeOnce();  // 'Initializing...'
initializeOnce();  // (nothing, returns cached result)
```

8.10.2 After: Execute After N Calls

```
function after(n, fn) {
    let count = 0;

    return function(...args) {
        count++;
        if (count >= n) {
            return fn.apply(this, args);
        }
    };
}

// Usage: Run after all async operations complete
const allDone = after(3, () => console.log('All 3 operations complete!'));

fetchData1().then(allDone);
fetchData2().then(allDone);
fetchData3().then(allDone);
// Logs after the 3rd call
```

8.10.3 Before: Execute Only First N Times

```
function before(n, fn) {
    let count = 0;
    let result;

    return function(...args) {
        if (count < n) {
            count++;
            result = fn.apply(this, args);
        }
        return result;
    };
}
```

```
// Usage: Limit to first 3 attempts
const limitedRetry = before(3, () => {
    console.log('Retrying...');
    return attemptOperation();
});
```

8.11 Practice Exercises

Exercise 8.1: Parallel with Limit

Implement a function that processes items in parallel but with a concurrency limit:

Exercise

```
async function mapWithLimit(items, limit, asyncFn) {
    // Your code here
}

// Usage
const results = await mapWithLimit(
    [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    3,  // Max 3 concurrent
    async (n) => {
        await sleep(1000);
        return n * 2;
    }
);
```

Exercise 8.2: Retry with Backoff

Create a retry wrapper with exponential backoff and max retries:

Exercise

```
function withRetry(asyncFn, options = {}) {
    // Options: maxRetries, baseDelay, maxDelay, shouldRetry
    // Your code here
}

// Usage
const resilientFetch = withRetry(fetch, {
    maxRetries: 5,
    baseDelay: 100,
    maxDelay: 5000,
    shouldRetry: (error) => error.status >= 500
});
```

Exercise 8.3: Async Queue

Build a queue that processes async tasks one at a time:

Exercise

```
function createAsyncQueue() {
    // Return: { add(asyncFn), pause(), resume(), clear() }
    // Your code here
}

// Usage
const queue = createAsyncQueue();
queue.add(() => fetchData(1));
queue.add(() => fetchData(2));
queue.add(() => fetchData(3));
// Processes one at a time in order
```

8.12 Chapter Summary

Chapter Summary

Pattern	Description
Parallel execution	`Promise.all` + `map` for independent operations
Sequential execution	`for...of` + `await` for ordered operations
Controlled concurrency	Limiter HOF for API rate limits
Retry logic	`withRetry` wrapper for resilient network calls
Debounce	Wait for pause (user input, auto-save)
Throttle	Rate limit (scroll, mouse events)
Timeout	`withTimeout` wrapper to prevent hanging
Cancellation	AbortController for user navigation

Key Insight

Async operations compose just like sync ones. Wrap them with HOFs to add retry, timeout, debounce, and other behaviors without changing the original function.

Chapter 9

HOFs in React

React is built on higher-order function concepts. Understanding HOFs deeply will make you a better React developer.

9.1 React Hooks Are HOF-Based

Every React hook follows HOF patterns:

```
// useState: returns state and a function to update it
const [count, setCount] = useState(0);

// useEffect: takes a function, optionally returns a cleanup function
useEffect(() => {
    const handler = () => console.log('clicked');
    window.addEventListener('click', handler);

    return () => window.removeEventListener('click', handler);  // Cleanup
}, []);

// useCallback: takes a function, returns a memoized function
const handleClick = useCallback(() => {
    console.log('clicked');
}, []);

// useMemo: takes a function, returns the memoized result
const expensiveValue = useMemo(() => {
    return computeExpensiveValue(data);
}, [data]);
```

9.2 useCallback: Memoizing Functions

`useCallback` prevents creating new function references on every render:

9.2.1 The Problem

```
function Parent() {
    const [count, setCount] = useState(0);

    // This creates a NEW function every render
    const handleClick = () => {
        console.log('clicked');
    };
```

```
    return (
        <>
            <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
            <ExpensiveChild onClick={handleClick} />
        </>
    );
}

// ExpensiveChild re-renders every time Parent renders
// because handleClick is a new reference each time!
```

9.2.2 The Solution

```
function Parent() {
    const [count, setCount] = useState(0);

    // Same function reference across renders
    const handleClick = useCallback(() => {
        console.log('clicked');
    }, []);  // Empty deps = never recreate

    return (
        <>
            <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
            <ExpensiveChild onClick={handleClick} />
        </>
    );
}

const ExpensiveChild = React.memo(({ onClick }) => {
    console.log('ExpensiveChild rendered');
    return <button onClick={onClick}>Click me</button>;
});
```

Use `useCallback` when passing callbacks to memoized child components with `React.memo`. This prevents unnecessary re-renders and improves performance in large component trees.

9.2.3 When Dependencies Change

```
function SearchBox({ onSearch }) {
    const [query, setQuery] = useState('');

    // Recreate only when query changes
    const handleSubmit = useCallback(() => {
        onSearch(query);
    }, [query, onSearch]);

    return (
        <form onSubmit={handleSubmit}>
            <input value={query} onChange={e => setQuery(e.target.value)} />
```

```
        </form>
    );
}
```

9.3 useMemo: Memoizing Values

`useMemo` memoizes the result of a computation:

```
function ProductList({ products, filterText }) {
    // Only recompute when products or filterText change
    const filteredProducts = useMemo(() => {
        console.log('Filtering products...');
        return products.filter(p =>
            p.name.toLowerCase().includes(filterText.toLowerCase())
        );
    }, [products, filterText]);

    // Only recompute when filteredProducts change
    const totalValue = useMemo(() => {
        return filteredProducts.reduce((sum, p) => sum + p.price, 0);
    }, [filteredProducts]);

    return (
        <div>
            <p>Total: ${totalValue}</p>
            <ul>
                {filteredProducts.map(p => (
                    <li key={p.id}>{p.name}: ${p.price}</li>
                ))}
            </ul>
        </div>
    );
}
```

9.3.1 useMemo vs useCallback

Definition: useMemo vs useCallback

`useMemo` memoizes the **result** of calling the function. `useCallback` memoizes the **function itself**. These are equivalent: `useCallback(fn, deps)` and `useMemo(() => fn, deps)`.

```
// useMemo: memoizes the RESULT of calling the function
const memoizedValue = useMemo(() => computeValue(a, b), [a, b]);

// useCallback: memoizes the FUNCTION ITSELF
const memoizedFn = useCallback(() => computeValue(a, b), [a, b]);
```

9.3.2 When to Use useMemo

1. **Expensive computations**

```
const sortedItems = useMemo(() => {
    return [...items].sort((a, b) => a.name.localeCompare(b.name));
}, [items]);
```

2. **Referential equality for objects/arrays**

```
// Without useMemo, new object every render
const style = { color: 'red', fontSize: size };  // New ref!

// With useMemo, stable reference
const style = useMemo(() => ({
    color: 'red',
    fontSize: size
}), [size]);
```

3. **Dependencies for other hooks**

```
const config = useMemo(() => ({
    endpoint: '/api/users',
    params: { role: userRole }
}), [userRole]);

useEffect(() => {
    fetchData(config);
}, [config]);  // Won't re-run unless userRole changes
```

9.4 The Stale Closure Problem

The most common HOF-related bug in React:

9.4.1 The Problem

× **Bad**

```
function Counter() {
    const [count, setCount] = useState(0);

    useEffect(() => {
        const interval = setInterval(() => {
            console.log('Count is:', count);  // Always logs 0!
            setCount(count + 1);              // Always sets to 1!
        }, 1000);

        return () => clearInterval(interval);
    }, []);  // Empty deps: closure captures initial count (0)

    return <div>{count}</div>;
}
```

The closure captures `count` at the time the effect runs (0), and never sees updates. This is the **stale closure** problem—one of the most common React bugs!

9.4.2 Fix 1: Functional Updates

Good

```
function Counter() {
    const [count, setCount] = useState(0);

    useEffect(() => {
        const interval = setInterval(() => {
            setCount(c => c + 1);  // c is always current!
        }, 1000);

        return () => clearInterval(interval);
    }, []);

    return <div>{count}</div>;
}
```

9.4.3 Fix 2: Include in Dependencies

```
function Counter() {
    const [count, setCount] = useState(0);

    useEffect(() => {
        const interval = setInterval(() => {
            console.log('Count is:', count);  // Now correct
            setCount(count + 1);
        }, 1000);

        return () => clearInterval(interval);
    }, [count]);  // Re-run effect when count changes

    return <div>{count}</div>;
}
```

This approach creates a new interval on every count change! While it works, it's inefficient. Use functional updates (Fix 1) or refs (Fix 3) instead.

9.4.4 Fix 3: useRef for Mutable Reference

Good

```
function Counter() {
    const [count, setCount] = useState(0);
    const countRef = useRef(count);

    // Keep ref in sync
    useEffect(() => {
        countRef.current = count;
    }, [count]);

    useEffect(() => {
        const interval = setInterval(() => {
            console.log('Count is:', countRef.current);  // Always current
            setCount(c => c + 1);
        }, 1000);

        return () => clearInterval(interval);
    }, []);

    return <div>{count}</div>;
}
```

9.5 Custom Hooks as HOF Patterns

Custom hooks are essentially higher-order patterns for stateful logic:

9.5.1 useToggle

```
function useToggle(initialValue = false) {
    const [value, setValue] = useState(initialValue);

    const toggle = useCallback(() => {
        setValue(v => !v);
    }, []);

    const setTrue = useCallback(() => setValue(true), []);
    const setFalse = useCallback(() => setValue(false), []);

    return [value, toggle, setTrue, setFalse];
}

// Usage
function Modal() {
    const [isOpen, toggle, open, close] = useToggle(false);

    return (
        <>
            <button onClick={open}>Open Modal</button>
            {isOpen && (
                <div className="modal">
                    <button onClick={close}>Close</button>
                </div>
```

```
            )}
        </>
    );
}
```

9.5.2 useDebounce

```
function useDebounce(value, delay) {
    const [debouncedValue, setDebouncedValue] = useState(value);

    useEffect(() => {
        const timer = setTimeout(() => {
            setDebouncedValue(value);
        }, delay);

        return () => clearTimeout(timer);
    }, [value, delay]);

    return debouncedValue;
}

// Usage
function Search() {
    const [query, setQuery] = useState('');
    const debouncedQuery = useDebounce(query, 300);

    useEffect(() => {
        if (debouncedQuery) {
            searchApi(debouncedQuery).then(setResults);
        }
    }, [debouncedQuery]);

    return <input value={query} onChange={e => setQuery(e.target.value)} />;
}
```

9.5.3 useAsync

```
function useAsync(asyncFn, deps = []) {
    const [state, setState] = useState({
        loading: true,
        error: null,
        data: null
    });

    useEffect(() => {
        let cancelled = false;

        setState(s => ({ ...s, loading: true }));

        asyncFn()
            .then(data => {
                if (!cancelled) {
                    setState({ loading: false, error: null, data });
```

```
                }
            })
            .catch(error => {
                if (!cancelled) {
                    setState({ loading: false, error, data: null });
                }
            });

        return () => { cancelled = true; };
    }, deps);

    return state;
}

// Usage
function UserProfile({ userId }) {
    const { loading, error, data: user } = useAsync(
        () => fetchUser(userId),
        [userId]
    );

    if (loading) return <Spinner />;
    if (error) return <Error message={error.message} />;
    return <Profile user={user} />;
}
```

9.6 Higher-Order Components (HOCs)

HOCs are functions that take a component and return a new enhanced component:

```
// HOC that adds loading state
function withLoading(WrappedComponent) {
    return function WithLoadingComponent({ isLoading, ...props }) {
        if (isLoading) {
            return <Spinner />;
        }
        return <WrappedComponent {...props} />;
    };
}

// Usage
const UserListWithLoading = withLoading(UserList);

<UserListWithLoading isLoading={loading} users={users} />
```

9.6.1 HOC with Configuration

```
function withAuth(WrappedComponent, { redirectTo = '/login' } = {}) {
    return function WithAuthComponent(props) {
        const { isAuthenticated, isLoading } = useAuth();

        if (isLoading) {
            return <Spinner />;
```

```
        }

        if (!isAuthenticated) {
            return <Navigate to={redirectTo} />;
        }

        return <WrappedComponent {...props} />;
    };
}

// Usage
const ProtectedDashboard = withAuth(Dashboard);
const ProtectedAdmin = withAuth(AdminPanel, { redirectTo: '/unauthorized' });
```

9.6.2 Composing Multiple HOCs

```
// Using compose for multiple HOCs
const compose = (...fns) => x => fns.reduceRight((acc, fn) => fn(acc), x);

const EnhancedComponent = compose(
    withAuth,
    withLoading,
    withErrorBoundary
)(BaseComponent);

// Equivalent to:
const EnhancedComponent = withAuth(withLoading(withErrorBoundary(BaseComponent)));
```

9.7 Practice Exercises

Exercise 9.1: useThrottle Hook

Create a hook that throttles a value:

Exercise

```
function useThrottle(value, interval) {
    // Your code here
}

// Usage
const [position, setPosition] = useState({ x: 0, y: 0 });
const throttledPosition = useThrottle(position, 100);
```

Exercise 9.2: useLocalStorage Hook

Create a hook that syncs state with localStorage:

Exercise

```
function useLocalStorage(key, initialValue) {
    // Your code here
}

// Usage
const [theme, setTheme] = useLocalStorage('theme', 'light');
// State persists across page reloads
```

Exercise 9.3: useAsyncCallback Hook

Create a hook that tracks loading/error state for an async callback:

Exercise

```
function useAsyncCallback(asyncFn) {
    // Return [execute, { loading, error, data }]
    // Your code here
}

// Usage
const [submitForm, { loading, error }] = useAsyncCallback(
    async (data) => {
        const response = await api.submit(data);
        return response;
    }
);
```

9.8 Chapter Summary

Chapter Summary

Pattern	Description
`useCallback`	Memoize function references
`useMemo`	Memoize computed values
Functional updates	Avoid stale closures in state setters
`useRef`	Mutable container for latest values
Custom hooks	Reusable stateful logic
Render props	Share logic via function children
HOCs	Enhance components with additional behavior
Compound components	Flexible, composable component APIs

Key Insight

React's hooks are HOFs in disguise. Understanding closures and function references is essential for avoiding bugs and writing performant React code.

Chapter 10

HOFs in State Management

Redux, Context, and other state management patterns are built on higher-order function principles.

10.1 Redux: HOFs Everywhere

Redux is essentially a collection of higher-order patterns:

```
// createStore: returns an object with methods that close over state
const store = createStore(reducer);

// Reducer: a pure function that returns new state
const reducer = (state, action) => newState;

// Middleware: HOF that takes store, returns HOF that takes next
const middleware = store => next => action => { ... };

// connect: HOC that subscribes component to store
const ConnectedComponent = connect(mapState, mapDispatch)(Component);
```

10.2 The Reducer Pattern

Reducers are pure functions that encapsulate state transitions:

```
// Basic reducer
function counterReducer(state = 0, action) {
    switch (action.type) {
        case 'INCREMENT':
            return state + 1;
        case 'DECREMENT':
            return state - 1;
        case 'SET':
            return action.payload;
        default:
            return state;
    }
}

// More complex reducer
function todosReducer(state = [], action) {
    switch (action.type) {
        case 'ADD_TODO':
            return [...state, {
                id: Date.now(),
```

```
                text: action.payload,
                completed: false
            }];

        case 'TOGGLE_TODO':
            return state.map(todo ⇒
                todo.id ═══ action.payload
                    ? { ...todo, completed: !todo.completed }
                    : todo
            );

        case 'DELETE_TODO':
            return state.filter(todo ⇒ todo.id ═══ action.payload);

        default:
            return state;
    }
}
```

10.2.1 Higher-Order Reducers

Undoable Higher-Order Reducer

```
// HOF that adds undo/redo capability to any reducer
function undoable(reducer) {
    const initialState = {
        past: [],
        present: reducer(undefined, {}),
        future: []
    };

    return function(state = initialState, action) {
        const { past, present, future } = state;

        switch (action.type) {
            case 'UNDO':
                if (past.length ═══ 0) return state;
                return {
                    past: past.slice(0, -1),
                    present: past[past.length - 1],
                    future: [present, ...future]
                };

            case 'REDO':
                if (future.length ═══ 0) return state;
                return {
                    past: [...past, present],
                    present: future[0],
                    future: future.slice(1)
                };

            default:
                const newPresent = reducer(present, action);
                if (newPresent ═══ present) return state;
                return {
                    past: [...past, present],
```

```
                    present: newPresent,
                    future: []
                };
        }
    };
}

// Usage
const undoableTodos = undoable(todosReducer);
```

10.2.2 Resettable Reducer HOF

```
function resettable(reducer, resetActionType = 'RESET') {
    const initialState = reducer(undefined, {});

    return function(state, action) {
        if (action.type === resetActionType) {
            return initialState;
        }
        return reducer(state, action);
    };
}

// Usage
const resettableCounter = resettable(counterReducer);
dispatch({ type: 'RESET' });  // Returns to initial state
```

10.3 Redux Middleware

Middleware signature is a triple-nested HOF:

Definition: Middleware Pattern

The signature `store ⇒ next ⇒ action ⇒ result` creates a triple-nested HOF that can intercept, modify, or delay actions.

```
const middleware = store ⇒ next ⇒ action ⇒ {
    // Do something before
    const result = next(action);
    // Do something after
    return result;
};
```

10.3.1 Logger Middleware

```
const logger = store ⇒ next ⇒ action ⇒ {
    console.group(action.type);
    console.log('Prev State:', store.getState());
    console.log('Action:', action);
```

```
    const result = next(action);

    console.log('Next State:', store.getState());
    console.groupEnd();

    return result;
};
```

10.3.2 Thunk Middleware

```
const thunk = store => next => action => {
    // If action is a function, call it with dispatch and getState
    if (typeof action === 'function') {
        return action(store.dispatch, store.getState);
    }

    // Otherwise, pass action along
    return next(action);
};

// Usage: action creator that returns a function
const fetchUser = (id) => async (dispatch, getState) => {
    dispatch({ type: 'FETCH_USER_START' });

    try {
        const user = await api.getUser(id);
        dispatch({ type: 'FETCH_USER_SUCCESS', payload: user });
    } catch (error) {
        dispatch({ type: 'FETCH_USER_ERROR', payload: error.message });
    }
};

// Dispatch the thunk
dispatch(fetchUser(123));
```

10.3.3 Analytics Middleware

```
const analytics = (trackingFn) => store => next => action => {
    // Track certain actions
    if (action.meta?.track) {
        trackingFn({
            event: action.type,
            properties: action.meta.trackingData || action.payload
        });
    }

    return next(action);
};

// Usage
dispatch({
    type: 'PURCHASE_COMPLETE',
```

```
    payload: { orderId: 123 },
    meta: {
        track: true,
        trackingData: { value: 99.99, currency: 'USD' }
    }
});
```

10.4 Selectors as HOFs

Selectors are functions that extract and derive data from state:

```
// Basic selectors
const selectTodos = state => state.todos;
const selectUser = state => state.user;

// Derived selectors (compose from basic ones)
const selectCompletedTodos = state =>
    selectTodos(state).filter(todo => todo.completed);

const selectIncompleteTodos = state =>
    selectTodos(state).filter(todo => !todo.completed);

const selectTodoCount = state => selectTodos(state).length;

// Parameterized selector (HOF)
const selectTodoById = (id) => (state) =>
    selectTodos(state).find(todo => todo.id === id);
```

10.4.1 Memoized Selectors with Reselect

```
import { createSelector } from 'reselect';

// Input selectors
const selectTodos = state => state.todos;
const selectFilter = state => state.filter;

// Memoized selector: only recomputes when inputs change
const selectVisibleTodos = createSelector(
    [selectTodos, selectFilter],
    (todos, filter) => {
        console.log('Computing visible todos...');

        switch (filter) {
            case 'completed':
                return todos.filter(t => t.completed);
            case 'active':
                return todos.filter(t => !t.completed);
            default:
                return todos;
        }
    }
);
```

```
// Usage
const visibleTodos = selectVisibleTodos(store.getState());
```

Use memoized selectors with Reselect when computing derived data from the store. This prevents expensive recalculations on every state change, improving performance significantly.

10.5 Practice Exercises

Exercise 10.1: Logging Middleware

Create a middleware that logs all actions with timing:

Exercise

```
const timingLogger = store => next => action => {
    // Log action, time it, log result
    // Your code here
};
```

Exercise 10.2: Undo/Redo HOF

Create a higher-order reducer that adds undo/redo to any reducer:

Exercise

```
function withUndoRedo(reducer, options = {}) {
    // Support: UNDO, REDO, CLEAR_HISTORY actions
    // Options: maxHistory (limit past states)
    // Your code here
}
```

10.6 Chapter Summary

Chapter Summary

Pattern	Description
Reducer	Pure function: `(state, action) ⇒ newState`
Higher-Order Reducer	HOF that enhances a reducer (undo, reset)
Middleware	Triple HOF: `store ⇒ next ⇒ action ⇒ result`
Selector	Function that extracts/derives data from state
Memoized Selector	Selector that caches results
Action Creator	Factory function that creates actions
Thunk	Action that's a function: `(dispatch, getState) ⇒ ...`

Key Insight

Redux architecture is built on function composition. Understanding HOFs deeply makes Redux patterns intuitive and allows you to create powerful abstractions.

Chapter 11

HOFs in Node.js & APIs

Server-side JavaScript is built on HOF patterns—from Express middleware to stream transforms to validation pipelines.

11.1 Express Middleware Pattern

Express middleware follows the HOF pattern:

```
// Middleware signature
const middleware = (req, res, next) => {
    // Do something
    next();  // Call next middleware
};

// Usage
app.use(middleware);
app.get('/route', middleware, handler);
```

11.1.1 Creating Middleware Factories

Logger Middleware Factory

```
function createLogger(options = {}) {
    const { format = 'combined', skip = () => false } = options;

    return (req, res, next) => {
        if (skip(req)) {
            return next();
        }

        const start = Date.now();

        res.on('finish', () => {
            const duration = Date.now() - start;
            console.log(`${req.method} ${req.url} ${res.statusCode} -
              ↪  ${duration}ms`);
        });

        next();
    };
}

// Usage
app.use(createLogger({ skip: req => req.url === '/health' }));
```

11.1.2 Authentication Middleware Factory

```
function requireAuth(options = {}) {
    const { tokenHeader = 'authorization', onError } = options;

    return async (req, res, next) => {
        const token = req.headers[tokenHeader];

        if (!token) {
            const error = new Error('No token provided');
            error.status = 401;
            return onError ? onError(error, req, res, next) : next(error);
        }

        try {
            const user = await verifyToken(token);
            req.user = user;
            next();
        } catch (error) {
            error.status = 401;
            return onError ? onError(error, req, res, next) : next(error);
        }
    };
}

// Usage
app.get('/protected', requireAuth(), (req, res) => {
    res.json({ user: req.user });
});
```

11.1.3 Role-Based Authorization

```
function requireRole(...allowedRoles) {
    return (req, res, next) => {
        if (!req.user) {
            return res.status(401).json({ error: 'Not authenticated' });
        }

        const hasRole = allowedRoles.some(role =>
            req.user.roles.includes(role)
        );

        if (!hasRole) {
            return res.status(403).json({ error: 'Insufficient permissions' });
        }

        next();
    };
}

// Usage
app.get('/admin', requireAuth(), requireRole('admin'), adminHandler);
app.get('/dashboard', requireAuth(), requireRole('admin', 'manager'),
  ↪ dashboardHandler);
```

11.2 Middleware Composition

11.2.1 Compose Multiple Middleware

```
function compose(...middlewares) {
    return (req, res, next) => {
        let index = 0;

        function dispatch(i) {
            if (i >= middlewares.length) {
                return next();
            }

            const middleware = middlewares[i];

            try {
                middleware(req, res, () => dispatch(i + 1));
            } catch (error) {
                next(error);
            }
        }

        dispatch(0);
    };
}

// Usage
const apiMiddleware = compose(
    cors(),
    rateLimit(),
    requireAuth(),
    validateRequest()
);

app.use('/api', apiMiddleware);
```

11.2.2 Conditional Middleware

```
function when(condition, middleware) {
    return (req, res, next) => {
        if (condition(req)) {
            return middleware(req, res, next);
        }
        next();
    };
}

// Usage
app.use(when(
    req => req.path.startsWith('/api'),
    requireAuth()
));

app.use(when(
    req => req.method === 'POST',
```

```
    validateBody()
));
```

11.2.3 Async Middleware Wrapper

```
// Express doesn't catch async errors by default
function asyncHandler(fn) {
    return (req, res, next) => {
        Promise.resolve(fn(req, res, next)).catch(next);
    };
}

// Usage
app.get('/users/:id', asyncHandler(async (req, res) => {
    const user = await User.findById(req.params.id);
    if (!user) {
        throw new NotFoundError('User not found');
    }
    res.json(user);
}));
```

11.3 Request Validation

11.3.1 Validation Middleware Factory

```
function validate(schema) {
    return (req, res, next) => {
        const { error, value } = schema.validate(req.body, {
            abortEarly: false
        });

        if (error) {
            const errors = error.details.map(d => ({
                field: d.path.join('.'),
                message: d.message
            }));
            return res.status(400).json({ errors });
        }

        req.validatedBody = value;
        next();
    };
}

// Usage with Joi
const createUserSchema = Joi.object({
    email: Joi.string().email().required(),
    password: Joi.string().min(8).required(),
    name: Joi.string().required()
});

app.post('/users', validate(createUserSchema), asyncHandler(async (req, res) => {
    const user = await User.create(req.validatedBody);
```

```
    res.status(201).json(user);
}));
```

11.4 Error Handling Patterns

11.4.1 Error Handler Factory

```
function createErrorHandler(options = {}) {
    const { logErrors = true, includeStack = false } = options;

    return (error, req, res, next) => {
        if (logErrors) {
            console.error('Error:', {
                message: error.message,
                stack: error.stack,
                url: req.url,
                method: req.method
            });
        }

        const status = error.status || error.statusCode || 500;
        const response = {
            error: error.message || 'Internal Server Error'
        };

        if (includeStack && process.env.NODE_ENV !== 'production') {
            response.stack = error.stack;
        }

        res.status(status).json(response);
    };
}

// Usage
app.use(createErrorHandler({
    logErrors: true,
    includeStack: process.env.NODE_ENV === 'development'
}));
```

11.5 Practice Exercises

Exercise 11.1: Request Timing Middleware

Create middleware that adds timing headers to responses:

Exercise

```
function timing() {
    // Add X-Response-Time header
    // Your code here
}

// Usage
app.use(timing());
// Response includes: X-Response-Time: 42ms
```

Exercise 11.2: Circuit Breaker Middleware

Create a circuit breaker for external API calls:

Exercise

```
function circuitBreaker(options = {}) {
    // Options: threshold, resetTimeout
    // States: CLOSED, OPEN, HALF_OPEN
    // Your code here
}

// Usage
const breaker = circuitBreaker({ threshold: 5, resetTimeout: 30000 });
app.get('/external', breaker, asyncHandler(async (req, res) => {
    const data = await externalApi.fetch();
    res.json(data);
}));
```

11.6 Chapter Summary

Chapter Summary

Pattern	Description
Middleware Factory	HOF that creates configured middleware
Middleware Composition	Combine multiple middleware into one
Async Handler Wrapper	Catch async errors in Express
Validation Pipeline	Compose validation rules
Response Transformer	Wrap/modify response format
Stream Transform	Process data streams with HOFs
Service Decorator	Add cross-cutting concerns to services
Query Builder	Fluent API for database queries

Key Insight

Express and Node.js APIs are naturally HOF-based. Embrace the pattern with factories, composition, and decorators for clean, reusable code.

Part III Summary: Real-World Patterns Mastered

You now have practical patterns for:

Domain	Patterns Learned
Async JS	Parallel/sequential execution, retry, debounce, throttle, cancellation
React	Hooks, memoization, custom hooks, HOCs, render props
State	Reducers, middleware, selectors, action creators
Node.js	Middleware, validation, error handling, streams

What's Next

In **Part IV: Advanced Patterns**, you'll explore:

- Performance optimization and lazy evaluation (Chapter 12)
- Functional error handling with Either (Chapter 13)
- Working with complex data using lenses (Chapter 14)
- Metaprogramming with Proxies (Chapter 15)

Part III Practice Project: Full-Stack API

Practice Project: Full-Stack API with HOF Patterns

Build a complete API using the patterns from Part III:

1. **Middleware stack**
   - Request logging with timing
   - Authentication (JWT)
   - Rate limiting
   - Request validation
2. **Resource routes (users, posts)**
   - CRUD operations
   - Pagination
   - Filtering
3. **State management (for React frontend)**
   - Redux store with middleware

- Async action creators
- Memoized selectors

4. **Error handling**
   - Custom error classes
   - Global error handler
   - Validation errors
5. **Async utilities**
   - Retry wrapper for external APIs
   - Debounced search endpoint
   - Cached responses

Focus on:

- Clean separation of concerns
- Reusable middleware factories
- Proper error propagation
- Memoization where appropriate

Part IV

Advanced Patterns

Chapter 12

Performance & Optimization

Higher-order functions have performance implications. This chapter teaches you when HOFs help, when they hurt, and how to optimize.

12.1 Understanding the Cost of HOFs

Every HOF call involves:

1. **Function creation** (if inline)
2. **Function invocation overhead**
3. **Closure context allocation** (if variables captured)
4. **Intermediate data structures** (arrays from map/filter)

12.1.1 Measuring Performance

Simple Benchmark Utility

```
// Simple benchmark utility
function benchmark(name, fn, iterations = 1000000) {
    const start = performance.now();
    for (let i = 0; i < iterations; i++) {
        fn();
    }
    const end = performance.now();
    console.log(`${name}: ${(end - start).toFixed(2)}ms`);
}

// Compare approaches
const numbers = Array.from({ length: 1000 }, (_, i) => i);

benchmark('for loop', () => {
    let sum = 0;
    for (let i = 0; i < numbers.length; i++) {
        sum += numbers[i] * 2;
    }
    return sum;
}, 10000);

benchmark('forEach', () => {
    let sum = 0;
    numbers.forEach(n => { sum += n * 2; });
    return sum;
}, 10000);
```

```
benchmark('reduce', () => {
    return numbers.reduce((sum, n) => sum + n * 2, 0);
}, 10000);
```

Terminal

```
Typical results (varies by engine):
for loop: ~15ms
forEach: ~25ms
reduce: ~30ms
```

12.1.2 The Reality: It Usually Doesn't Matter

For most web applications:

- Arrays under 10,000 items: Use HOFs freely
- Hot paths called millions of times: Consider optimization
- User-facing latency: Network/DOM are bigger bottlenecks

12.2 The Chain Problem

Each array method creates a new array:

```
const data = Array.from({ length: 100000 }, (_, i) => i);

// Creates 3 intermediate arrays!
const result = data
    .filter(n => n % 2 === 0)      // Array 1: 50,000 items
    .map(n => n * 2)               // Array 2: 50,000 items
    .filter(n => n > 1000)         // Array 3: ~49,500 items
    .map(n => ({ value: n }));     // Array 4: ~49,500 items

// Memory: 4 arrays allocated
// Time: 4 full iterations
```

12.2.1 Measuring Chain Overhead

```
const data = Array.from({ length: 100000 }, (_, i) => i);

benchmark('Chained HOFs', () => {
    return data
        .filter(n => n % 2 === 0)
        .map(n => n * 2)
        .filter(n => n > 1000);
}, 100);

benchmark('Single reduce', () => {
    return data.reduce((acc, n) => {
        if (n % 2 === 0) {
            const doubled = n * 2;
```

```
            if (doubled > 1000) {
                acc.push(doubled);
            }
        }
        return acc;
    }, []);
}, 100);

benchmark('For loop', () => {
    const result = [];
    for (let i = 0; i < data.length; i++) {
        const n = data[i];
        if (n % 2 === 0) {
            const doubled = n * 2;
            if (doubled > 1000) {
                result.push(doubled);
            }
        }
    }
    return result;
}, 100);
```

Terminal

```
Typical results:
Chained HOFs: ~80ms
Single reduce: ~30ms
For loop: ~20ms
```

12.3 Transducers: Single-Pass Processing

Transducers compose transformations without creating intermediate arrays.

Definition: Transducer

A **transducer** is a function that transforms a reducer. It enables composition of transformations that execute in a single pass, without creating intermediate collections.

12.3.1 The Concept

```
// Normal: Each step creates new array
data.filter(isEven).map(double);  // 2 arrays, 2 passes

// Transducer: Compose the operations, single pass
transduce(compose(filter(isEven), map(double)), data);  // 1 array, 1 pass
```

12.3.2 Building Transducers

Transducer Implementation

```
// A transducer is a function that transforms a reducer
// transducer: (reducer) => reducer

// Map transducer
const map = (fn) => (reducer) => (acc, item) => {
    return reducer(acc, fn(item));
};

// Filter transducer
const filter = (predicate) => (reducer) => (acc, item) => {
    return predicate(item) ? reducer(acc, item) : acc;
};

// Take transducer (with early termination)
const take = (n) => (reducer) => {
    let count = 0;
    return (acc, item) => {
        if (count >= n) return acc;
        count++;
        return reducer(acc, item);
    };
};

// Compose transducers (left-to-right for data flow)
const compose = (...fns) => x => fns.reduceRight((acc, fn) => fn(acc), x);

// The final reducer (how to combine results)
const pushReducer = (acc, item) => {
    acc.push(item);
    return acc;
};

// Transduce function
const transduce = (xform, reducer, initial, data) => {
    const transformedReducer = xform(reducer);
    return data.reduce(transformedReducer, initial);
};
```

12.3.3 Using Transducers

```
const data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

// Compose: filter even, double, take 3
const xform = compose(
    filter(n => n % 2 === 0),
    map(n => n * 2),
    take(3)
);

const result = transduce(xform, pushReducer, [], data);
// [4, 8, 12]
```

```
// Only processes: 1, 2, 3, 4, 5, 6 (stops after 3 results)
// Single pass through data
// No intermediate arrays
```

Key Insight

Transducers provide the best of both worlds: the composability of HOFs with the efficiency of imperative loops. Use them when processing large datasets with multiple transformations.

12.3.4 Performance Comparison

```
const largeData = Array.from({ length: 1000000 }, (_, i) => i);

benchmark('Array chain', () => {
    return largeData
        .filter(n => n % 2 === 0)
        .map(n => n * 2)
        .slice(0, 100);
}, 10);

benchmark('Transducer', () => {
    const xform = compose(
        filter(n => n % 2 === 0),
        map(n => n * 2),
        take(100)
    );
    return transduce(xform, pushReducer, [], largeData);
}, 10);
```

Terminal

```
Array chain: ~150ms (processes ALL 1M items)
Transducer: ~0.5ms (stops after 200 items)
```

12.4 Lazy Evaluation with Generators

Generators enable lazy, pull-based iteration:

Lazy Operations with Generators

```
// Lazy map
function* lazyMap(iterable, fn) {
    for (const item of iterable) {
        yield fn(item);
    }
}

// Lazy filter
function* lazyFilter(iterable, predicate) {
    for (const item of iterable) {
```

```
        if (predicate(item)) {
            yield item;
        }
    }
}

// Lazy take
function* lazyTake(iterable, n) {
    let count = 0;
    for (const item of iterable) {
        if (count >= n) return;
        yield item;
        count++;
    }
}

// Compose lazily
function lazyPipe(iterable, ...operations) {
    return operations.reduce((iter, op) => op(iter), iterable);
}
```

12.4.1 Using Lazy Pipelines

```
const numbers = function* () {
    let i = 0;
    while (true) {
        yield i++;
    }
};

// Process infinite sequence lazily
const result = lazyPipe(
    numbers(),
    iter => lazyFilter(iter, n => n % 2 === 0),
    iter => lazyMap(iter, n => n * 2),
    iter => lazyTake(iter, 5)
);

// Only compute what we need
console.log([...result]);  // [0, 4, 8, 12, 16]
// Only generated: 0, 1, 2, 3, 4, 5, 6, 7, 8
```

12.4.2 Lazy Class Implementation

Complete Lazy Evaluation Class

```
class Lazy {
    constructor(iterable) {
        this.iterable = iterable;
    }

    static from(iterable) {
        return new Lazy(iterable);
    }
```

```
    static range(start, end) {
        return new Lazy(function* () {
            for (let i = start; i < end; i++) {
                yield i;
            }
        }());
    }

    static infinite(startFrom = 0) {
        return new Lazy(function* () {
            let i = startFrom;
            while (true) yield i++;
        }());
    }

    *[Symbol.iterator]() {
        yield* this.iterable;
    }

    map(fn) {
        const source = this.iterable;
        return new Lazy(function* () {
            for (const item of source) {
                yield fn(item);
            }
        }());
    }

    filter(predicate) {
        const source = this.iterable;
        return new Lazy(function* () {
            for (const item of source) {
                if (predicate(item)) yield item;
            }
        }());
    }

    take(n) {
        const source = this.iterable;
        return new Lazy(function* () {
            let count = 0;
            for (const item of source) {
                if (count >= n) return;
                yield item;
                count++;
            }
        }());
    }

    // Terminal operations
    toArray() {
        return [...this.iterable];
    }

    reduce(fn, initial) {
        let acc = initial;
```

```
        for (const item of this.iterable) {
            acc = fn(acc, item);
        }
        return acc;
    }
}
```

Example: Lazy Evaluation in Action

Using the `Lazy` class, we can chain operations on an infinite sequence and only compute the values we need.

```
const result = Lazy.infinite()
    .filter(n => n % 2 === 0)
    .map(n => n ** 2)
    .take(10)
    .toArray();

console.log(result);
// [0, 4, 16, 36, 64, 100, 144, 196, 256, 324]
```

12.5 Iterator Helpers (ES2025)

Modern JavaScript includes native lazy iteration:

```
// ES2025 Iterator Helpers
const result = Iterator.from([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
    .filter(n => n % 2 === 0)
    .map(n => n * 2)
    .take(3)
    .toArray();

// [4, 8, 12] - lazy evaluation, single pass
```

Iterator Helpers are coming to JavaScript natively. They provide lazy evaluation built into the language, combining the readability of HOFs with the efficiency of generators.

12.5.1 Available Iterator Helpers

Method	Description
`map(fn)`	Transform each element
`filter(predicate)`	Keep elements passing predicate
`take(n)`	Limit to first n elements
`drop(n)`	Skip first n elements
`flatMap(fn)`	Map and flatten
`reduce(fn, initial)`	Fold to single value
`toArray()`	Collect to array
`forEach(fn)`	Side effects
`some(predicate)`	Any element passes?
`every(predicate)`	All elements pass?
`find(predicate)`	First element passing predicate

12.6 Memoization Strategies

12.6.1 Basic Memoization

```
function memoize(fn) {
    const cache = new Map();

    return function (...args) {
        const key = JSON.stringify(args);

        if (cache.has(key)) {
            return cache.get(key);
        }

        const result = fn.apply(this, args);
        cache.set(key, result);
        return result;
    };
}

// Usage
const expensiveCalculation = memoize((n) => {
    console.log('Computing...');
    let result = 0;
    for (let i = 0; i < n * 1000000; i++) {
        result += Math.sqrt(i);
    }
    return result;
});

expensiveCalculation(10);  // Computing... (slow)
expensiveCalculation(10);  // (instant, cached)
```

12.6.2 Memoization with LRU Cache

LRU Cache Memoization

```
function memoizeWithLRU(fn, maxSize = 100) {
    const cache = new Map();

    return function (...args) {
        const key = JSON.stringify(args);

        if (cache.has(key)) {
            // Move to end (most recently used)
            const value = cache.get(key);
            cache.delete(key);
            cache.set(key, value);
            return value;
        }

        const result = fn.apply(this, args);

        // Evict oldest if at capacity
        if (cache.size ≥ maxSize) {
            const firstKey = cache.keys().next().value;
            cache.delete(firstKey);
        }

        cache.set(key, result);
        return result;
    };
}
```

12.6.3 Memoization with TTL

```
function memoizeWithTTL(fn, ttlMs = 60000) {
    const cache = new Map();

    return function (...args) {
        const key = JSON.stringify(args);
        const now = Date.now();

        if (cache.has(key)) {
            const { value, timestamp } = cache.get(key);
            if (now - timestamp < ttlMs) {
                return value;
            }
            cache.delete(key);
        }

        const result = fn.apply(this, args);
        cache.set(key, { value: result, timestamp: now });
        return result;
    };
}

// Usage
const fetchUser = memoizeWithTTL(async (id) => {
```

```
    const response = await fetch(`/api/users/${id}`);
    return response.json();
}, 30000);  // Cache for 30 seconds
```

12.7 V8 Optimization Insights

12.7.1 Monomorphism: Keep Types Consistent

× Bad

```
// BAD: Polymorphic (mixed types) - slow
function processItems(items) {
    return items.map(item => item.value * 2);
}

processItems([{ value: 1 }, { value: 2 }]);  // Objects
processItems([1, 2, 3]);                      // Numbers - different type!
// V8 can't optimize - must handle multiple types
```

Good

```
// GOOD: Monomorphic (consistent types) - fast
function processObjects(items) {
    return items.map(item => item.value * 2);
}

function processNumbers(items) {
    return items.map(n => n * 2);
}
// V8 optimizes each function for its specific type
```

12.7.2 Hidden Classes: Consistent Object Shapes

× Bad

```
// BAD: Different property orders = different hidden classes
const user1 = { name: 'Alice', age: 30 };
const user2 = { age: 25, name: 'Bob' };  // Different order!

// BAD: Adding properties later
const user3 = { name: 'Charlie' };
user3.age = 35;  // Added later - new hidden class
```

Good

```
// GOOD: Same shape, same order
function createUser(name, age) {
    return { name, age };  // Always same shape
}
```

```
const users = [
    createUser('Alice', 30),
    createUser('Bob', 25),
    createUser('Charlie', 35)
];
// All share same hidden class - fast property access
```

12.8 Performance Decision Matrix

Scenario	Recommendation
< 1,000 items	Use array methods freely
1,000 - 10,000 items	Array methods OK, profile if slow
10,000 - 100,000 items	Consider single reduce or transducers
> 100,000 items	Use lazy evaluation or for loops
Hot path (called 1000s/sec)	Benchmark, consider for loop
Infinite/streaming data	Use generators or async iterators
Heavy computation per item	Memoize the transformation
Need early exit	Use find/some/every or lazy

12.8.1 Quick Optimization Checklist

```
// 1. Combine filters
// Instead of:
data.filter(a).filter(b).filter(c)
// Use:
data.filter(x => a(x) && b(x) && c(x))

// 2. Combine map operations
// Instead of:
data.map(a).map(b).map(c)
// Use:
data.map(x => c(b(a(x))))

// 3. Filter before map (reduce items early)
// Instead of:
data.map(transform).filter(predicate)
// Use:
data.filter(predicate).map(transform)

// 4. Use appropriate method
// Instead of:
data.filter(predicate).length > 0
// Use:
data.some(predicate)

// Instead of:
data.filter(predicate)[0]
```

```
// Use:
data.find(predicate)
```

12.9 Practice Exercises

Exercise 12.1: Transducer Implementation

Complete the transducer implementation with `flatMap` and `takeWhile`:

Exercise

```
const flatMap = (fn) => (reducer) => (acc, item) => {
    // Your code here
};

const takeWhile = (predicate) => (reducer) => {
    // Your code here - stop when predicate returns false
};
```

Exercise 12.2: Lazy Pipeline

Create a lazy pipeline that processes a large CSV file line by line:

Exercise

```
function processCSV(lines) {
    // Return lazy pipeline that:
    // 1. Skips header
    // 2. Parses each line to object
    // 3. Filters by condition
    // 4. Transforms data
    // 5. Takes first N results
}
```

12.10 Chapter Summary

Chapter Summary

Pattern	Description
Array methods	Small data, readability priority
Single reduce	Medium data, avoid intermediates
Transducers	Large data, composable transforms
Generators/Lazy	Infinite/streaming, early exit
Iterator Helpers	Modern JS, native lazy eval
Memoization	Expensive pure functions
For loops	Maximum performance critical paths

Key Insight

Optimize based on data, not dogma. Measure first, optimize where it matters, and prefer readability for non-critical paths.

Chapter 13

Functional Error Handling

Traditional try/catch breaks function composition. Functional error handling keeps your pipelines clean and errors explicit.

13.1 The Problem with Exceptions

Exceptions break composition:

```
// This pipeline can break at any point
const result = data
    .map(parse)      // Might throw!
    .filter(validate) // Might throw!
    .map(transform);  // Might throw!

// You're forced to wrap everything
try {
    const result = data
        .map(parse)
        .filter(validate)
        .map(transform);
} catch (error) {
    // Which step failed? What data caused it?
    // Hard to know without complex error handling
}
```

> Exceptions are hidden side effects. A function that throws doesn't declare it in its type signature, making errors invisible until runtime.

13.1.1 Exceptions Are Side Effects

```
// This function lies about its return type
function divide(a, b) {
    if (b === 0) throw new Error('Division by zero');
    return a / b;
}

// Type says: (number, number) => number
// Reality: (number, number) => number | throws
```

13.2 The Either Pattern

Definition: Either

Either represents a value that can be one of two types:

- **Right**: Success (the "right" path)
- **Left**: Failure (the "wrong" path)

Either Implementation

```
// Either implementation
const Right = (value) => ({
    map: (fn) => Right(fn(value)),
    flatMap: (fn) => fn(value),
    fold: (onLeft, onRight) => onRight(value),
    isRight: true,
    isLeft: false,
    value
});

const Left = (value) => ({
    map: (fn) => Left(value),  // Don't apply fn, pass through error
    flatMap: (fn) => Left(value),
    fold: (onLeft, onRight) => onLeft(value),
    isRight: false,
    isLeft: true,
    value
});

// Helper to wrap potentially throwing functions
const tryCatch = (fn) => {
    try {
        return Right(fn());
    } catch (error) {
        return Left(error);
    }
};
```

13.2.1 Using Either

```
// Safe division
const divide = (a, b) =>
    b === 0
        ? Left(new Error('Division by zero'))
        : Right(a / b);

// Chain operations safely
const result = divide(10, 2)        // Right(5)
    .map(n => n * 2)                 // Right(10)
    .map(n => n + 1)                 // Right(11)
    .fold(
        error => `Error: ${error.message}`,
        value => `Result: ${value}`
```

```
    );

// "Result: 11"

// With error
const errorResult = divide(10, 0)   // Left(Error)
    .map(n => n * 2)                 // Left(Error) - skipped
    .map(n => n + 1)                 // Left(Error) - skipped
    .fold(
        error => `Error: ${error.message}`,
        value => `Result: ${value}`
    );

// "Error: Division by zero"
```

13.3 Railway Oriented Programming

Think of your pipeline as a railway with two tracks:

- **Success track**: Data flows through transformations
- **Failure track**: Errors bypass transformations

```
Success: ──────[fn1]──────[fn2]──────[fn3]──────> Result │ │ │ Failure:
─────────┴──────────┴──────────┴──────> Error
```

Railway Validation Pipeline

```
// Each function is a "switch" that can derail to error track
const validateEmail = (data) =>
    data.email?.includes('@')
        ? Right(data)
        : Left({ field: 'email', message: 'Invalid email' });

const validatePassword = (data) =>
    data.password?.length >= 8
        ? Right(data)
        : Left({ field: 'password', message: 'Password too short' });

const validateAge = (data) =>
    data.age >= 18
        ? Right(data)
        : Left({ field: 'age', message: 'Must be 18 or older' });

// Railway pipeline
const validateUser = (data) =>
    Right(data)
        .flatMap(validateEmail)
        .flatMap(validatePassword)
        .flatMap(validateAge);
```

Example: Railway in Action

Valid data flows through all checks and returns `Right`. Invalid data short-circuits at the first error and returns `Left`.

```
validateUser({
    email: 'test@example.com',
    password: 'securepass123',
    age: 25
});
// Right({ email: 'test@example.com', ... })

validateUser({ email: 'invalid', password: 'short', age: 16 });
// Left({ field: 'email', message: 'Invalid email' })
// Stops at first error
```

13.4 Collecting All Errors

Sometimes you want all errors, not just the first:

Validation That Collects Errors

```
// Validation that collects all errors
const Validation = {
    Success: (value) => ({
        map: (fn) => Validation.Success(fn(value)),
        flatMap: (fn) => fn(value),
        fold: (onFailure, onSuccess) => onSuccess(value),
        concat: (other) => other,
        isSuccess: true,
        value
    }),

    Failure: (errors) => ({
        map: (fn) => Validation.Failure(errors),
        flatMap: (fn) => Validation.Failure(errors),
        fold: (onFailure, onSuccess) => onFailure(errors),
        concat: (other) => other.isSuccess
            ? Validation.Failure(errors)
            : Validation.Failure([...errors, ...other.value]),
        isSuccess: false,
        value: errors
    })
};

// Validators return Validation
const validateEmail = (email) =>
    email?.includes('@')
        ? Validation.Success(email)
        : Validation.Failure([{ field: 'email', message: 'Invalid email' }]);

const validatePassword = (password) =>
    password?.length >= 8
```

```
        ? Validation.Success(password)
        : Validation.Failure([{ field: 'password', message: 'Password too short' }]);
```

13.5 Maybe/Option Pattern

For values that might not exist:

Maybe Implementation

```
const Maybe = {
    Just: (value) => ({
        map: (fn) => Maybe.Just(fn(value)),
        flatMap: (fn) => fn(value),
        getOrElse: (defaultValue) => value,
        filter: (predicate) => predicate(value)
            ? Maybe.Just(value)
            : Maybe.Nothing(),
        fold: (onNothing, onJust) => onJust(value),
        isJust: true,
        isNothing: false,
        value
    }),

    Nothing: () => ({
        map: (fn) => Maybe.Nothing(),
        flatMap: (fn) => Maybe.Nothing(),
        getOrElse: (defaultValue) => defaultValue,
        filter: (predicate) => Maybe.Nothing(),
        fold: (onNothing, onJust) => onNothing(),
        isJust: false,
        isNothing: true,
        value: null
    }),

    fromNullable: (value) =>
        value == null ? Maybe.Nothing() : Maybe.Just(value)
};
```

13.5.1 Safe Property Access

```
// Safe property access
const prop = (key) => (obj) => Maybe.fromNullable(obj[key]);

const user = {
    name: 'Alice',
    address: {
        city: 'New York'
    }
};

// Safe nested access
const getCity = (user) =>
    Maybe.fromNullable(user)
```

```
        .flatMap(prop('address'))
        .flatMap(prop('city'))
        .getOrElse('Unknown');

getCity(user);                    // 'New York'
getCity({ name: 'Bob' });         // 'Unknown'
getCity(null);                    // 'Unknown'
```

13.6 Result Type (Rust-Inspired)

A more explicit Either with Ok/Err:

Result Type Implementation

```
const Result = {
    Ok: (value) => ({
        map: (fn) => Result.Ok(fn(value)),
        mapErr: (fn) => Result.Ok(value),
        flatMap: (fn) => fn(value),
        unwrap: () => value,
        unwrapOr: (defaultValue) => value,
        match: ({ ok, err }) => ok(value),
        isOk: true,
        isErr: false,
        value
    }),

    Err: (error) => ({
        map: (fn) => Result.Err(error),
        mapErr: (fn) => Result.Err(fn(error)),
        flatMap: (fn) => Result.Err(error),
        unwrap: () => { throw error; },
        unwrapOr: (defaultValue) => defaultValue,
        match: ({ ok, err }) => err(error),
        isOk: false,
        isErr: true,
        error
    }),

    fromThrowable: (fn) => (...args) => {
        try {
            return Result.Ok(fn(...args));
        } catch (error) {
            return Result.Err(error);
        }
    }
};
```

Example: Result Type in Action

The Result type lets us chain operations on fallible computations and handle errors elegantly with pattern matching.

```
const parseJSON = Result.fromThrowable(JSON.parse);

parseJSON('{"name": "Alice"}')
    .map(data => data.name)
    .map(name => name.toUpperCase())
    .match({
        ok: name => console.log(`Hello, ${name}!`),
        err: error => console.error(`Parse error: ${error.message}`)
    });
```

13.7 Async Either (TaskEither)

Combining Either with Promises:

TaskEither for Async Operations

```
const TaskEither = {
    tryCatch: (asyncFn) => {
        const promise = asyncFn()
            .then(value => ({ isRight: true, value }))
            .catch(error => ({ isRight: false, error }));

        return {
            map: (fn) => TaskEither.tryCatch(async () => {
                const result = await promise;
                if (result.isRight) return fn(result.value);
                throw result.error;
            }),

            flatMap: (fn) => TaskEither.tryCatch(async () => {
                const result = await promise;
                if (result.isRight) return fn(result.value).run();
                throw result.error;
            }),

            fold: async (onLeft, onRight) => {
                const result = await promise;
                return result.isRight
                    ? onRight(result.value)
                    : onLeft(result.error);
            },

            run: async () => {
                const result = await promise;
                if (result.isRight) return result.value;
                throw result.error;
            }
        };
    }
};
```

Example: Async Error Handling

TaskEither combines the power of Either with async/await, letting us chain async operations with clean error handling.

```
const fetchUser = (id) => TaskEither.tryCatch(
    () => fetch(`/api/users/${id}`).then(r => r.json())
);

const result = await fetchUser(123)
    .map(user => user.name)
    .map(name => name.toUpperCase())
    .fold(
        error => `Error: ${error.message}`,
        name => `Hello, ${name}!`
    );
```

13.8 Practice Exercises

Exercise 13.1: Either Implementation

Complete the Either implementation with these additional methods:

Exercise

```
// Add to Right/Left:
// - orElse(fn): if Left, run fn to get new Either
// - tap(fn): run side effect without changing value
// - toMaybe(): convert to Maybe
// - swap(): swap Right/Left
```

Exercise 13.2: Validation Pipeline

Create a user registration validation that collects all errors:

Exercise

```
const validateRegistration = (data) => {
    // Validate: email, password, confirmPassword, age, terms
    // Return all errors, not just the first
};
```

13.9 Chapter Summary

Chapter Summary

Type	Purpose	Use When
Either	Success or Failure	Operations that can fail
Maybe	Value or Nothing	Optional values, null safety
Validation	Collect all errors	Form validation
Result	Ok or Err	Explicit error handling
TaskEither	Async Either	Async operations that can fail

Key Insight

Make errors part of your type system, not hidden exceptions. This makes error handling explicit, composable, and impossible to forget.

Chapter 14

Working with Complex Data

Lenses and optics provide composable tools for working with nested, immutable data structures.

14.1 The Problem with Deep Updates

Updating nested immutable data is verbose:

× Bad

```
const state = {
    user: {
        profile: {
            address: {
                city: 'New York'
            }
        }
    }
};

// Update city - ugly!
const newState = {
    ...state,
    user: {
        ...state.user,
        profile: {
            ...state.user.profile,
            address: {
                ...state.user.profile.address,
                city: 'Los Angeles'
            }
        }
    }
};
```

14.2 What is a Lens?

Definition: Lens

A lens is a pair of functions:

- **get**: Extract a value from a structure
- **set**: Return a new structure with a value changed

Lenses are composable, allowing you to focus on deeply nested values.

Simple Lens Implementation

```
// Simple lens implementation
const lens = (getter, setter) => ({
    get: getter,
    set: setter,

    // Modify using a function
    over: (fn) => (obj) => setter(fn(getter(obj)), obj),

    // Compose with another lens
    compose: (other) => lens(
        (obj) => other.get(getter(obj)),
        (value, obj) => setter(other.set(value, getter(obj)), obj)
    )
});

// Create a lens for a property
const prop = (key) => lens(
    (obj) => obj[key],
    (value, obj) => ({ ...obj, [key]: value })
);
```

14.2.1 Using Lenses

```
const nameLens = prop('name');
const addressLens = prop('address');
const cityLens = prop('city');

const user = {
    name: 'Alice',
    address: { city: 'NYC', zip: '10001' }
};

// Get
nameLens.get(user);  // 'Alice'

// Set
nameLens.set('Bob', user);
// { name: 'Bob', address: { ... } }

// Over (modify with function)
nameLens.over(s => s.toUpperCase())(user);
// { name: 'ALICE', ... }
```

14.2.2 Composing Lenses

```
// Compose to access nested properties
const userCityLens = addressLens.compose(cityLens);

userCityLens.get(user);  // 'NYC'
userCityLens.set('LA', user);
// { name: 'Alice', address: { city: 'LA', zip: '10001' } }
```

Key Insight

Lenses turn nested updates into simple, declarative operations. Build small lenses, compose them for deep access, and your state updates become clear and reusable.

14.3 Lens Utilities

```
// Lens for array index
const index = (i) => lens(
    (arr) => arr[i],
    (value, arr) => {
        const copy = [...arr];
        copy[i] = value;
        return copy;
    }
);

// Lens for nested path
const path = (...keys) => keys.map(prop).reduce((a, b) => a.compose(b));

// Usage
const state = {
    users: [
        { name: 'Alice', scores: [85, 90, 78] },
        { name: 'Bob', scores: [92, 88, 95] }
    ]
};

const firstUserLens = prop('users').compose(index(0));
const firstUserNameLens = firstUserLens.compose(prop('name'));
const firstUserFirstScoreLens = firstUserLens
    .compose(prop('scores'))
    .compose(index(0));

firstUserNameLens.get(state);  // 'Alice'
firstUserFirstScoreLens.set(100, state);
// Updates Alice's first score to 100
```

14.4 Prisms: For Optional/Union Types

Prisms handle values that might not exist:

Prism Implementation

```
const prism = (getOption, reverseGet) => ({
    getOption,  // Returns Maybe
    reverseGet, // Constructs the whole from the part

    modify: (fn) => (whole) => {
        const part = getOption(whole);
        return part.isJust
            ? reverseGet(fn(part.value))
            : whole;
```

```
    }
});

// Prism for array head
const headPrism = prism(
    (arr) => arr.length > 0 ? Maybe.Just(arr[0]) : Maybe.Nothing(),
    (value) => [value]
);

// Prism for specific type in union
const stringPrism = prism(
    (value) => typeof value === 'string'
        ? Maybe.Just(value)
        : Maybe.Nothing(),
    (value) => value
);
```

14.5 Traversals: For Multiple Focuses

Traversals work with multiple values at once:

```
// Traversal over array elements
const each = {
    over: (fn) => (arr) => arr.map(fn),
    get: (arr) => arr,  // Gets all elements
    set: (value) => (arr) => arr.map(() => value)
};

// Compose lens with traversal
const state = {
    users: [
        { name: 'Alice', active: true },
        { name: 'Bob', active: false },
        { name: 'Charlie', active: true }
    ]
};

// Update all user names to uppercase
const usersLens = prop('users');
const updateAllNames = (state) =>
    usersLens.over(users =>
        each.over(user =>
            prop('name').over(s => s.toUpperCase())(user)
        )(users)
    )(state);
```

14.6 Practical Lens Patterns

14.6.1 Pattern 1: Redux-Style State Updates

Lenses for Redux

```
// Lenses for different parts of state
const todosLens = prop('todos');
const filterLens = prop('filter');
const todoByIdLens = (id) => todosLens.compose(
    lens(
        (todos) => todos.find(t => t.id === id),
        (todo, todos) => todos.map(t => t.id === id ? todo : t)
    )
);

// Actions using lenses
const addTodo = (todo) => todosLens.over(todos => [...todos, todo]);
const toggleTodo = (id) => todoByIdLens(id).over(todo => ({
    ...todo,
    completed: !todo.completed
}));
const setFilter = (filter) => filterLens.set(filter);
```

14.6.2 Pattern 2: Form State Management

```
const formLens = prop('form');
const fieldLens = (name) => formLens.compose(prop(name));
const valueLens = (name) => fieldLens(name).compose(prop('value'));
const errorLens = (name) => fieldLens(name).compose(prop('error'));
const touchedLens = (name) => fieldLens(name).compose(prop('touched'));

// Form actions
const setValue = (name, value) => valueLens(name).set(value);
const setError = (name, error) => errorLens(name).set(error);
const setTouched = (name) => touchedLens(name).set(true);
```

14.7 Immer as a Lens Alternative

Immer provides a simpler API for the same goal:

With Lenses

```
const cityLens = path(
    'user',
    'profile',
    'address',
    'city'
);

const newState =
    cityLens.set(
        'LA',
        state
    );
```

With Immer

```
const newState =
    produce(state, draft => {
        draft.user
            .profile
            .address
            .city = 'LA';
    });
```

14.7.1 When to Use Lenses vs Immer

Lenses	Immer
Composable, reusable accessors	One-off updates
Type-safe (with TS)	Simple syntax
No runtime dependency	Requires library
Functional style	Mutable-style syntax

14.8 Practice Exercises

Exercise 14.1: Lens Implementation

Complete the lens implementation with these features:

Exercise

```
const lens = (getter, setter) => ({
    get: getter,
    set: setter,
    over: (fn) => (obj) => // Your code,
    compose: (other) => // Your code,

    // Add these:
    // view: alias for get
    // modify: alias for over
    // toMaybe: get as Maybe (for optional values)
});
```

Exercise 14.2: Array Lenses

Create lenses for array operations:

Exercise

```
const arrayLenses = {
    // Focus on element at index
    at: (index) => // lens,

    // Focus on first element
    head: // lens,

    // Focus on last element
    last: // lens,

    // Focus on elements matching predicate
    where: (predicate) => // traversal-like
};
```

14.9 Chapter Summary

Chapter Summary

Optic	Focus	Use Case
Lens	Exactly one value	Property access/update
Prism	Zero or one value	Optional values, union types
Traversal	Zero or more values	Collections, multiple targets
Iso	Bidirectional transform	Type conversions

Key Insight

Lenses make immutable updates composable. Build small lenses, compose them for deep access, and your state updates become declarative and reusable.

Chapter 15

Metaprogramming with Proxies + HOFs

Combining Proxies with higher-order functions enables powerful metaprogramming patterns.

15.1 Proxy Fundamentals

Proxy intercepts operations on objects:

```
const target = { name: 'Alice', age: 30 };

const handler = {
    get(target, prop, receiver) {
        console.log(`Getting ${prop}`);
        return Reflect.get(target, prop, receiver);
    },

    set(target, prop, value, receiver) {
        console.log(`Setting ${prop} to ${value}`);
        return Reflect.set(target, prop, value, receiver);
    }
};

const proxy = new Proxy(target, handler);

proxy.name;         // Logs: "Getting name", returns "Alice"
proxy.age = 31;     // Logs: "Setting age to 31"
```

15.2 HOF-Based Proxy Factories

15.2.1 Observable Objects

Observable Proxy Factory

```
function observable(target, onChange) {
    return new Proxy(target, {
        set(target, prop, value, receiver) {
            const oldValue = target[prop];
            const result = Reflect.set(target, prop, value, receiver);

            if (oldValue !== value) {
                onChange(prop, value, oldValue);
            }

            return result;
        },
```

```
        deleteProperty(target, prop) {
            const oldValue = target[prop];
            const result = Reflect.deleteProperty(target, prop);

            if (result) {
                onChange(prop, undefined, oldValue);
            }

            return result;
        }
    });
}

// Usage
const state = observable({ count: 0 }, (prop, newVal, oldVal) => {
    console.log(`${prop} changed from ${oldVal} to ${newVal}`);
});

state.count++;  // "count changed from 0 to 1"
```

15.2.2 Deep Observable

Deep Observable with Nested Tracking

```
function deepObservable(target, onChange, path = []) {
    if (typeof target !== 'object' || target === null) {
        return target;
    }

    // Recursively wrap nested objects
    for (const key of Object.keys(target)) {
        if (typeof target[key] === 'object' && target[key] !== null) {
            target[key] = deepObservable(
                target[key],
                onChange,
                [...path, key]
            );
        }
    }

    return new Proxy(target, {
        set(target, prop, value, receiver) {
            const fullPath = [...path, prop].join('.');
            const oldValue = target[prop];

            // Wrap new objects
            if (typeof value === 'object' && value !== null) {
                value = deepObservable(value, onChange, [...path, prop]);
            }

            const result = Reflect.set(target, prop, value, receiver);

            if (oldValue !== value) {
                onChange(fullPath, value, oldValue);
            }
```

```
            return result;
        }
    });
}
```

Example: Deep Observable in Action

Deep observables track changes at any depth, providing the full path to the changed property.

```
const state = deepObservable({
    user: { profile: { name: 'Alice' } }
}, (path, newVal, oldVal) => {
    console.log(`${path}: ${oldVal} -> ${newVal}`);
});

state.user.profile.name = 'Bob';
// "user.profile.name: Alice -> Bob"
```

15.3 Validation Proxy

Type Validation Proxy

```
function validated(target, schema) {
    return new Proxy(target, {
        set(target, prop, value, receiver) {
            const validator = schema[prop];

            if (validator) {
                const result = validator(value);
                if (!result.valid) {
                    throw new TypeError(
                        `Invalid value for ${prop}: ${result.error}`
                    );
                }
            }

            return Reflect.set(target, prop, value, receiver);
        }
    });
}

// Validation helpers (HOFs)
const isType = (type) => (value) => ({
    valid: typeof value === type,
    error: `Expected ${type}, got ${typeof value}`
});

const inRange = (min, max) => (value) => ({
    valid: value >= min && value <= max,
    error: `Must be between ${min} and ${max}`
});
```

```
const matches = (regex) => (value) => ({
    valid: regex.test(value),
    error: `Must match pattern ${regex}`
});

// Compose validators
const and = (...validators) => (value) => {
    for (const v of validators) {
        const result = v(value);
        if (!result.valid) return result;
    }
    return { valid: true };
};
```

Example: Validated Object Usage

The validation proxy automatically validates every property assignment against its schema.

```
const user = validated({}, {
    name: and(isType('string'), (v) => ({
        valid: v.length >= 2,
        error: 'Name must be at least 2 characters'
    })),
    age: and(isType('number'), inRange(0, 150)),
    email: and(isType('string'), matches(/^[^@]+@[^@]+\.[^@]+$/))
});

user.name = 'Alice';  // OK
user.age = 30;        // OK
user.age = -5;        // Throws: "Invalid value for age: Must be between 0 and 150"
```

15.4 Lazy Initialization

```
function lazy(initializer) {
    let instance = null;
    let initialized = false;

    return new Proxy({}, {
        get(target, prop, receiver) {
            if (!initialized) {
                instance = initializer();
                initialized = true;
            }
            return Reflect.get(instance, prop, instance);
        },

        set(target, prop, value, receiver) {
            if (!initialized) {
                instance = initializer();
                initialized = true;
            }
```

```
            return Reflect.set(instance, prop, value, instance);
        }
    });
}

// Usage
const expensiveService = lazy(() => {
    console.log('Initializing expensive service...');
    return {
        process: (data) => data.toUpperCase(),
        config: { timeout: 5000 }
    };
});

// Nothing happens yet
console.log('Service created');

// Now it initializes
expensiveService.process('hello');
// Logs: "Initializing expensive service..."
```

15.5 Method Chaining Proxy

Auto-Chainable Methods

```
function chainable(target) {
    return new Proxy(target, {
        get(target, prop, receiver) {
            const value = Reflect.get(target, prop, receiver);

            if (typeof value === 'function') {
                return function (...args) {
                    const result = value.apply(target, args);
                    // If method returns undefined, return proxy for chaining
                    return result === undefined ? receiver : result;
                };
            }

            return value;
        }
    });
}

// Usage
class Builder {
    constructor() {
        this.config = {};
    }

    setName(name) {
        this.config.name = name;
        // No return - would normally break chaining
    }

    setAge(age) {
```

```
        this.config.age = age;
    }

    build() {
        return { ...this.config };
    }
}

const builder = chainable(new Builder());

const result = builder
    .setName('Alice')  // Returns proxy, not undefined
    .setAge(30)        // Returns proxy, not undefined
    .build();          // Returns actual result

console.log(result);  // { name: 'Alice', age: 30 }
```

15.6 Negative Array Indices

```
function negativeArray(arr) {
    return new Proxy(arr, {
        get(target, prop, receiver) {
            const index = Number(prop);

            if (Number.isInteger(index) && index < 0) {
                return target[target.length + index];
            }

            return Reflect.get(target, prop, receiver);
        },

        set(target, prop, value, receiver) {
            const index = Number(prop);

            if (Number.isInteger(index) && index < 0) {
                target[target.length + index] = value;
                return true;
            }

            return Reflect.set(target, prop, value, receiver);
        }
    });
}

// Usage
const arr = negativeArray([1, 2, 3, 4, 5]);

arr[-1];  // 5 (last element)
arr[-2];  // 4 (second to last)
arr[-1] = 10;
console.log(arr);  // [1, 2, 3, 4, 10]
```

15.7 Immutable Proxy

Deep Immutable Proxy

```
function immutable(target, deep = true) {
    if (typeof target !== 'object' || target === null) {
        return target;
    }

    // Deep freeze nested objects
    if (deep) {
        for (const key of Object.keys(target)) {
            if (typeof target[key] === 'object') {
                target[key] = immutable(target[key], true);
            }
        }
    }

    return new Proxy(target, {
        set(target, prop, value) {
            throw new TypeError(
                `Cannot modify immutable property: ${prop}`
            );
        },

        deleteProperty(target, prop) {
            throw new TypeError(
                `Cannot delete immutable property: ${prop}`
            );
        },

        defineProperty(target, prop, descriptor) {
            throw new TypeError(
                `Cannot define property on immutable object: ${prop}`
            );
        }
    });
}
```

15.8 Method Interception (AOP)

Aspect-Oriented Programming with Proxies

```
function intercept(target, interceptors) {
    return new Proxy(target, {
        get(target, prop, receiver) {
            const value = Reflect.get(target, prop, receiver);

            if (typeof value !== 'function') {
                return value;
            }

            const { before, after, around } = interceptors[prop] || {};
```

```
            return function (...args) {
                // Before advice
                if (before) {
                    before.call(this, args);
                }

                let result;

                // Around advice (or original)
                if (around) {
                    result = around.call(
                        this,
                        () => value.apply(this, args),
                        args
                    );
                } else {
                    result = value.apply(this, args);
                }

                // After advice
                if (after) {
                    after.call(this, result, args);
                }

                return result;
            };
        }
    });
}
```

Example: Method Interception Example

AOP-style interception allows adding cross-cutting concerns (logging, timing, caching) without modifying original methods.

```
const service = {
    fetchData(id) {
        console.log(`Fetching ${id}`);
        return { id, data: 'result' };
    }
};

const interceptedService = intercept(service, {
    fetchData: {
        before: (args) => console.log(`Before: args = ${args}`),
        after: (result) => console.log(`After: result = ${JSON.stringify(result)}`),
        around: (proceed, args) => {
            console.log('Around: start');
            const result = proceed();
            console.log('Around: end');
            return result;
        }
    }
});
```

15.9 Virtual Properties

Computed Properties via Proxy

```
function withVirtuals(target, virtuals) {
    return new Proxy(target, {
        get(target, prop, receiver) {
            if (prop in virtuals) {
                const virtual = virtuals[prop];
                return typeof virtual === 'function'
                    ? virtual.call(receiver, target)
                    : virtual;
            }
            return Reflect.get(target, prop, receiver);
        },

        set(target, prop, value, receiver) {
            if (prop in virtuals) {
                throw new TypeError(
                    `Cannot set virtual property: ${prop}`
                );
            }
            return Reflect.set(target, prop, value, receiver);
        },

        ownKeys(target) {
            return [...Reflect.ownKeys(target), ...Object.keys(virtuals)];
        },

        getOwnPropertyDescriptor(target, prop) {
            if (prop in virtuals) {
                return {
                    enumerable: true,
                    configurable: true,
                    get: () => virtuals[prop]
                };
            }
            return Reflect.getOwnPropertyDescriptor(target, prop);
        }
    });
}

// Usage
const user = withVirtuals(
    { firstName: 'Alice', lastName: 'Smith', birthYear: 1990 },
    {
        fullName: (target) => `${target.firstName} ${target.lastName}`,
        age: (target) => new Date().getFullYear() - target.birthYear
    }
);

user.fullName;  // 'Alice Smith'
user.age;       // 34 (or current age)
```

15.10 Fluent API Builder

Dynamic Fluent API with Proxy

```
function fluentApi(basePath, execute) {
    const buildProxy = (path) => new Proxy(() => {}, {
        get(target, prop) {
            if (prop === 'then') {
                // Allow await
                return (resolve) => resolve(execute(path));
            }

            return buildProxy([...path, prop]);
        },

        apply(target, thisArg, args) {
            return buildProxy([...path, { args }]);
        }
    });

    return buildProxy([basePath]);
}

// Usage: API client
const api = fluentApi('', async (path) => {
    const url = path
        .map(p => typeof p === 'object' ? p.args.join('/') : p)
        .join('/');

    console.log(`Fetching: ${url}`);
    return { url };
});

// These all work:
await api.users;                    // Fetching: users
await api.users(123);               // Fetching: users/123
await api.users(123).posts;         // Fetching: users/123/posts
await api.users(123).posts(456);    // Fetching: users/123/posts/456
```

15.11 Practice Exercises

Exercise 15.1: Memoizing Proxy

Create a proxy that memoizes method calls:

Exercise

```
function memoizeProxy(target) {
    // Cache method results based on arguments
    // Your code here
}

const service = memoizeProxy({
    expensiveCalculation(n) {
        console.log('Computing...');
        return n * 2;
    }
});

service.expensiveCalculation(5);  // Logs "Computing...", returns 10
service.expensiveCalculation(5);  // Returns 10 (cached, no log)
```

Exercise 15.2: Type Checking Proxy

Create a proxy that enforces types at runtime:

Exercise

```
function typed(schema) {
    // schema: { propName: 'string' | 'number' | Constructor }
    // Your code here
}

const user = typed({
    name: 'string',
    age: 'number',
    createdAt: Date
})({});

user.name = 'Alice';  // OK
user.name = 123;      // Throws TypeError
```

15.12 Chapter Summary

Chapter Summary

Pattern	Description
Observable	React to property changes
Validation	Runtime type checking
Lazy	Defer expensive initialization
Immutable	Prevent modifications
Intercept	Add cross-cutting concerns (logging, timing)
Virtual	Computed properties
Fluent API	Chainable, readable interfaces
Negative index	Python-style array access

Key Insight

Proxies + HOFs enable powerful metaprogramming. Use them at system boundaries (APIs, state management) but avoid in hot paths due to performance overhead.

Part IV Summary: Advanced Patterns Mastered

You now have advanced techniques:

Topic	Key Patterns
Performance	Transducers, lazy evaluation, memoization, V8 optimization
Error Handling	Either, Maybe, Result, Railway-oriented programming
Complex Data	Lenses, prisms, traversals, safe path access
Metaprogramming	Observable, validation, interception, fluent APIs

What's Next

In **Part V: Mastery & Reference**, you'll find:

- Debugging and testing strategies (Chapter 16)
- Quick reference and decision guides (Chapter 17)

Part IV Practice Project: Reactive State System

Build a reactive state management system combining all Part IV concepts:

Practice Project: Reactive State System

Create a comprehensive state management system with:

1. **Immutable state with lens-based updates** – Use lenses for declarative state modifications
2. **Observable changes (Proxy-based)** – Track all state mutations automatically
3. **Computed values (lazy, memoized)** – Derive values efficiently with caching
4. **Validation on updates (Either-based)** – Ensure state integrity with functional validation
5. **Efficient selectors** – Use transducers for large collection processing

Example API:

```
const store = createReactiveStore({
    initialState: {
        users: [],
        filters: { active: true, role: null },
        settings: { pageSize: 20 }
    },

    lenses: {
        users: prop('users'),
        filters: prop('filters'),
        activeFilter: path('filters', 'active')
    },

    computed: {
        filteredUsers: (state) => /* lazy, memoized */,
        userCount: (state) => /* derived */
    },

    validators: {
        'filters.active': isBoolean,
        'settings.pageSize': and(isNumber, inRange(1, 100))
    }
});

// Subscribe to changes
store.subscribe((path, newValue, oldValue) => {
    console.log(`${path} changed`);
});

// Update with lens
store.lens('activeFilter').set(false);

// Get computed value (lazy)
store.computed.filteredUsers;

// Validate on update
store.lens('settings.pageSize').set(200);  // Throws validation error
```

Part V

Mastery & Reference

Chapter 16

Debugging & Testing Higher-Order Functions

Debugging HOFs requires different strategies than debugging imperative code. This chapter gives you the tools to trace, test, and troubleshoot functional pipelines.

16.1 The Challenge of Debugging HOFs

HOFs are harder to debug because:

1. **Stack traces are abstract**: You see `Array.map` instead of your logic
2. **Intermediate values are hidden**: Chained operations don't expose their outputs
3. **Closures capture state**: Variables might not be what you expect
4. **Point-free style hides data**: No variable names to inspect

```
// What failed here?
const result = data
    .filter(isValid)      // Which items were removed?
    .map(transform)       // What did transform produce?
    .reduce(combine, []); // Why is result wrong?

// Hard to know without visibility into each step
```

16.2 The tap Pattern: Non-Invasive Debugging

`tap` lets you inspect values without breaking the pipeline:

The tap Utility

```
// The tap utility
const tap = (fn) => (value) => {
    fn(value);
    return value;  // Pass through unchanged
};

// Usage: inspect intermediate values
const result = data
    .filter(isValid)
    .map(tap(x => console.log('After filter:', x)))  // Logs each item
    .map(transform)
    .map(tap(x => console.log('After transform:', x)))
```

```
    .reduce(combine, []);
```

16.2.1 Tap Variations

```
// Log with label
const tapLog = (label) => tap(x => console.log(label, x));

// Log array length
const tapCount = (label) => tap(arr =>
    console.log(label, 'count:', arr.length)
);

// Conditional logging
const tapIf = (predicate, fn) => tap(x => {
    if (predicate(x)) fn(x);
});

// Log and return for single values
const tapValue = (label) => (value) => {
    console.log(label, value);
    return value;
};

// Usage
const result = data
    .filter(isValid)
    .map(tapLog('filtered item:'))
    .map(transform)
    .map(tapIf(x => x.error, x => console.warn('Error:', x)))
    .reduce(combine, []);
```

16.2.2 Pipeline Tap for Arrays

```
// Tap the entire array between operations
const tapArray = (label) => (array) => {
    console.log(label, {
        length: array.length,
        first: array[0],
        last: array[array.length - 1],
        sample: array.slice(0, 3)
    });
    return array;
};

// Insert between chain methods
const result = tapArray('after filter')(
    data.filter(isValid)
);

// Or wrap the chain
function debugPipeline(data) {
    let step1 = data.filter(isValid);
    console.log('Step 1 (filter):', step1.length, 'items');
```

```
    let step2 = step1.map(transform);
    console.log('Step 2 (transform):', step2[0]);

    let step3 = step2.reduce(combine, []);
    console.log('Step 3 (reduce):', step3);

    return step3;
}
```

16.3 Tracing Function Calls

16.3.1 Wrapper for Tracing

```
function traced(name, fn) {
    return function(...args) {
        console.group(`${name}()`);
        console.log('Arguments:', args);

        try {
            const result = fn.apply(this, args);
            console.log('Result:', result);
            console.groupEnd();
            return result;
        } catch (error) {
            console.error('Error:', error.message);
            console.groupEnd();
            throw error;
        }
    };
}

// Usage
const processUser = traced('processUser', (user) => {
    return {
        ...user,
        fullName: `${user.firstName} ${user.lastName}`
    };
});

users.map(processUser);
// Logs each call with arguments and results
```

16.3.2 Tracing with Timing

```
function timedTrace(name, fn) {
    return function(...args) {
        const start = performance.now();
        const result = fn.apply(this, args);
        const duration = performance.now() - start;

        console.log(`${name}: ${duration.toFixed(2)}ms`, {
            args: args.length,
```

```
            result: typeof result
        });

        return result;
    };
}

// Find slow operations
const slowTransform = timedTrace('transform', transform);
data.map(slowTransform);
```

16.3.3 Accumulating Trace

```
function createTracer() {
    const traces = [];

    const trace = (name, fn) => (...args) => {
        const start = performance.now();
        const result = fn(...args);
        const duration = performance.now() - start;

        traces.push({
            name,
            args: args.length > 0 ? args : undefined,
            result,
            duration,
            timestamp: Date.now()
        });

        return result;
    };

    const getTraces = () => [...traces];
    const clear = () => traces.length = 0;

    const summary = () => {
        const byName = traces.reduce((acc, t) => {
            if (!acc[t.name]) acc[t.name] = { calls: 0, totalTime: 0 };
            acc[t.name].calls++;
            acc[t.name].totalTime += t.duration;
            return acc;
        }, {});

        return Object.entries(byName).map(([name, stats]) => ({
            name,
            calls: stats.calls,
            totalTime: stats.totalTime.toFixed(2) + 'ms',
            avgTime: (stats.totalTime / stats.calls).toFixed(2) + 'ms'
        }));
    };

    return { trace, getTraces, clear, summary };
}

// Usage
```

```
const tracer = createTracer();

const tracedFilter = tracer.trace('filter', isValid);
const tracedMap = tracer.trace('transform', transform);

data.filter(tracedFilter).map(tracedMap);

console.table(tracer.summary());
```

16.4 Debugging Closures

16.4.1 Inspecting Closed-Over Variables

```
// Problem: closure captures unexpected value
function createHandlers(items) {
    return items.map((item, index) => {
        // What does this closure actually capture?
        return () => console.log(item, index);
    });
}

// Debug version: expose captured values
function createHandlersDebug(items) {
    return items.map((item, index) => {
        const handler = () => console.log(item, index);

        // Attach debug info
        handler._debug = { item, index, capturedAt: new Date() };

        return handler;
    });
}

const handlers = createHandlersDebug(['a', 'b', 'c']);
console.log(handlers[0]._debug);  // { item: 'a', index: 0, ... }
```

16.4.2 The Closure Snapshot Pattern

```
function createCounter() {
    let count = 0;

    const counter = {
        increment() { return ++count; },
        decrement() { return --count; },
        getCount() { return count; },

        // Debug: snapshot internal state
        _snapshot() {
            return { count, timestamp: Date.now() };
        }
    };

    return counter;
```

```
}

const c = createCounter();
c.increment();
c.increment();
console.log(c._snapshot());  // { count: 2, timestamp: ... }
```

16.4.3 Detecting Stale Closures

```
// React hook for detecting stale closures
function useStaleClosureDetector(value, name = 'value') {
    const renderCount = useRef(0);
    const valueAtCapture = useRef(value);

    renderCount.current++;

    useEffect(() => {
        valueAtCapture.current = value;
    }, [value]);

    return (capturedValue) => {
        if (capturedValue !== valueAtCapture.current) {
            console.warn(
                `Stale closure detected for "${name}"!`,
                `Captured: ${capturedValue}, Current: ${valueAtCapture.current}`,
                `Render: ${renderCount.current}`
            );
            return true;
        }
        return false;
    };
}

// Usage in component
function MyComponent({ data }) {
    const checkStale = useStaleClosureDetector(data, 'data');

    useEffect(() => {
        const timer = setInterval(() => {
            checkStale(data);  // Will warn if data is stale
            processData(data);
        }, 1000);

        return () => clearInterval(timer);
    }, []);  // Missing dependency - will be stale!
}
```

16.5 Debugging Async HOFs

16.5.1 Tracing Async Operations

```
function tracedAsync(name, asyncFn) {
    return async function(...args) {
        const id = Math.random().toString(36).substr(2, 9);
        console.log(`[${id}] ${name} START`, args);

        try {
            const result = await asyncFn.apply(this, args);
            console.log(`[${id}] ${name} SUCCESS`, result);
            return result;
        } catch (error) {
            console.log(`[${id}] ${name} ERROR`, error.message);
            throw error;
        }
    };
}

// Usage with Promise.all
const fetchUser = tracedAsync('fetchUser', async (id) => {
    const res = await fetch(`/api/users/${id}`);
    return res.json();
});

await Promise.all([1, 2, 3].map(fetchUser));
// Logs start/success for each with unique IDs
```

16.5.2 Debugging Promise Chains

```
// Tap for promises
const tapAsync = (fn) => async (value) => {
    await fn(value);
    return value;
};

// Usage
const result = await fetchData()
    .then(tapAsync(data => console.log('Fetched:', data.length)))
    .then(transform)
    .then(tapAsync(data => console.log('Transformed:', data)))
    .then(save);
```

16.5.3 Async Pipeline Debugger

```
function createAsyncPipeline(...fns) {
    return async (input) => {
        let value = input;
        const steps = [];

        for (let i = 0; i < fns.length; i++) {
            const fn = fns[i];
            const stepName = fn.name || `step${i}`;
            const start = performance.now();
```

```
            try {
                value = await fn(value);
                steps.push({
                    step: stepName,
                    success: true,
                    duration: performance.now() - start,
                    output: value
                });
            } catch (error) {
                steps.push({
                    step: stepName,
                    success: false,
                    duration: performance.now() - start,
                    error: error.message
                });

                console.error('Pipeline failed at step:', stepName);
                console.table(steps);
                throw error;
            }
        }

        console.log('Pipeline completed');
        console.table(steps.map(s => ({
            ...s,
            output: typeof s.output === 'object' ? '[Object]' : s.output
        })));

        return value;
    };
}

// Usage
const pipeline = createAsyncPipeline(
    fetchData,
    validateData,
    transformData,
    saveData
);

await pipeline(inputData);
```

16.6 Testing Higher-Order Functions

16.6.1 Testing Function Factories

```
describe('createMultiplier', () => {
    it('creates a function that multiplies by the given factor', () => {
        const double = createMultiplier(2);
        const triple = createMultiplier(3);

        expect(double(5)).toBe(10);
        expect(triple(5)).toBe(15);
    });
```

```
    it('returns different functions for different factors', () => {
        const double = createMultiplier(2);
        const anotherDouble = createMultiplier(2);

        expect(double).not.toBe(anotherDouble);  // Different instances
        expect(double(5)).toBe(anotherDouble(5)); // Same behavior
    });

    it('handles edge cases', () => {
        const zero = createMultiplier(0);
        const negative = createMultiplier(-1);

        expect(zero(100)).toBe(0);
        expect(negative(5)).toBe(-5);
    });
});
```

16.6.2 Testing Closures

```
describe('createCounter', () => {
    it('maintains independent state per instance', () => {
        const counter1 = createCounter();
        const counter2 = createCounter();

        counter1.increment();
        counter1.increment();

        expect(counter1.getCount()).toBe(2);
        expect(counter2.getCount()).toBe(0);  // Independent!
    });

    it('respects initial value', () => {
        const counter = createCounter(10);

        expect(counter.getCount()).toBe(10);
        counter.increment();
        expect(counter.getCount()).toBe(11);
    });

    it('reset returns to initial value', () => {
        const counter = createCounter(5);
        counter.increment();
        counter.increment();
        counter.reset();

        expect(counter.getCount()).toBe(5);
    });
});
```

16.6.3 Testing Wrapped Functions

```
describe('memoize', () => {
    it('caches results for same arguments', () => {
        const expensive = jest.fn(x => x * 2);
        const memoized = memoize(expensive);

        memoized(5);
        memoized(5);
        memoized(5);

        expect(expensive).toHaveBeenCalledTimes(1);  // Only called once!
    });

    it('calls function for different arguments', () => {
        const fn = jest.fn(x => x * 2);
        const memoized = memoize(fn);

        memoized(1);
        memoized(2);
        memoized(3);

        expect(fn).toHaveBeenCalledTimes(3);
    });

    it('returns correct cached value', () => {
        const fn = jest.fn(x => x * 2);
        const memoized = memoize(fn);

        const first = memoized(5);
        const second = memoized(5);

        expect(first).toBe(10);
        expect(second).toBe(10);
        expect(first).toBe(second);
    });
});
```

16.6.4 Testing Debounce/Throttle

```
describe('debounce', () => {
    beforeEach(() => {
        jest.useFakeTimers();
    });

    afterEach(() => {
        jest.useRealTimers();
    });

    it('delays execution', () => {
        const fn = jest.fn();
        const debounced = debounce(fn, 100);

        debounced();
        expect(fn).not.toHaveBeenCalled();
```

```
        jest.advanceTimersByTime(100);
        expect(fn).toHaveBeenCalledTimes(1);
    });

    it('only executes once for rapid calls', () => {
        const fn = jest.fn();
        const debounced = debounce(fn, 100);

        debounced();
        debounced();
        debounced();

        jest.advanceTimersByTime(100);
        expect(fn).toHaveBeenCalledTimes(1);
    });

    it('uses latest arguments', () => {
        const fn = jest.fn();
        const debounced = debounce(fn, 100);

        debounced('first');
        debounced('second');
        debounced('third');

        jest.advanceTimersByTime(100);
        expect(fn).toHaveBeenCalledWith('third');
    });
});
```

16.6.5 Testing Compose/Pipe

```
describe('pipe', () => {
    it('applies functions left to right', () => {
        const addOne = x => x + 1;
        const double = x => x * 2;
        const square = x => x * x;

        const pipeline = pipe(addOne, double, square);

        // 5 -> 6 -> 12 -> 144
        expect(pipeline(5)).toBe(144);
    });

    it('handles single function', () => {
        const double = x => x * 2;
        const pipeline = pipe(double);

        expect(pipeline(5)).toBe(10);
    });

    it('returns identity for no functions', () => {
        const pipeline = pipe();

        expect(pipeline(5)).toBe(5);
```

```
    });

    it('preserves function errors', () => {
        const throwing = () => { throw new Error('oops'); };
        const pipeline = pipe(x => x + 1, throwing, x => x * 2);

        expect(() => pipeline(5)).toThrow('oops');
    });
});
```

16.7 Testing Strategies for HOFs

16.7.1 Strategy 1: Test the Factory and Product Separately

```
// Factory
function createValidator(rules) {
    return (value) => {
        const errors = rules
            .filter(rule => !rule.test(value))
            .map(rule => rule.message);
        return { valid: errors.length === 0, errors };
    };
}

// Tests
describe('createValidator', () => {
    describe('factory behavior', () => {
        it('returns a function', () => {
            const validator = createValidator([]);
            expect(typeof validator).toBe('function');
        });

        it('creates independent validators', () => {
            const v1 = createValidator([
                { test: x => x > 0, message: 'Must be positive' }
            ]);
            const v2 = createValidator([
                { test: x => x < 0, message: 'Must be negative' }
            ]);

            expect(v1(5).valid).toBe(true);
            expect(v2(5).valid).toBe(false);
        });
    });

    describe('validator behavior', () => {
        const emailValidator = createValidator([
            { test: v => v.includes('@'), message: 'Must contain @' },
            { test: v => v.includes('.'), message: 'Must contain .' }
        ]);

        it('returns valid for passing input', () => {
            expect(emailValidator('test@example.com')).toEqual({
                valid: true,
                errors: []
```

```
            });
        });

        it('returns all failing rules', () => {
            expect(emailValidator('invalid')).toEqual({
                valid: false,
                errors: ['Must contain @', 'Must contain .']
            });
        });
    });
});
```

16.7.2 Strategy 2: Test Composition Behavior

```
describe('middleware composition', () => {
    it('applies middlewares in order', () => {
        const order = [];

        const m1 = (next) => (req) => {
            order.push('m1');
            return next(req);
        };
        const m2 = (next) => (req) => {
            order.push('m2');
            return next(req);
        };
        const m3 = (next) => (req) => {
            order.push('m3');
            return next(req);
        };

        const handler = (req) => {
            order.push('handler');
            return 'done';
        };
        const composed = compose(m1, m2, m3)(handler);

        composed({});

        expect(order).toEqual(['m1', 'm2', 'm3', 'handler']);
    });

    it('allows middleware to modify request', () => {
        const addHeader = (next) => (req) => {
            return next({
                ...req,
                headers: { ...req.headers, 'X-Custom': 'value' }
            });
        };

        const handler = (req) => req.headers;
        const composed = compose(addHeader)(handler);

        expect(composed({ headers: {} })).toEqual({ 'X-Custom': 'value' });
    });
```

```
    it('allows middleware to short-circuit', () => {
        const auth = (next) => (req) => {
            if (!req.token) return { error: 'Unauthorized' };
            return next(req);
        };

        const handler = jest.fn(() => 'success');
        const composed = compose(auth)(handler);

        expect(composed({})).toEqual({ error: 'Unauthorized' });
        expect(handler).not.toHaveBeenCalled();
    });
});
```

16.7.3 Strategy 3: Property-Based Testing

```
// Using fast-check for property-based testing
import fc from 'fast-check';

describe('map/filter properties', () => {
    it('map preserves length', () => {
        fc.assert(
            fc.property(
                fc.array(fc.integer()),
                (arr) => {
                    const result = arr.map(x => x * 2);
                    return result.length === arr.length;
                }
            )
        );
    });

    it('filter returns subset', () => {
        fc.assert(
            fc.property(
                fc.array(fc.integer()),
                (arr) => {
                    const result = arr.filter(x => x > 0);
                    return result.length <= arr.length;
                }
            )
        );
    });

    it('compose is associative', () => {
        const f = x => x + 1;
        const g = x => x * 2;
        const h = x => x - 3;

        fc.assert(
            fc.property(
                fc.integer(),
                (n) => {
                    const left = compose(f, compose(g, h))(n);
```

```
                    const right = compose(compose(f, g), h)(n);
                    return left === right;
                }
            )
        );
    });
});
```

16.8 Common HOF Bugs and Fixes

16.8.1 Bug 1: Mutating in map/filter

× Bad

```
// BUG: Mutating objects in map
const users = [{ name: 'Alice', active: false }];
const activated = users.map(user => {
    user.active = true;  // Mutates original!
    return user;
});
```

Good

```
// FIX: Create new objects
const activated = users.map(user => ({
    ...user,
    active: true
}));
```

16.8.2 Bug 2: Forgetting Return in reduce

× Bad

```
// BUG: No return in reduce
const sum = numbers.reduce((acc, n) => {
    acc + n;  // Missing return!
}, 0);
// sum is undefined
```

Good

```
// FIX: Return the accumulator
const sum = numbers.reduce((acc, n) => {
    return acc + n;
}, 0);

// Or use arrow shorthand
const sum = numbers.reduce((acc, n) => acc + n, 0);
```

16.8.3 Bug 3: Wrong reduce Initial Value

× Bad

```
// BUG: Wrong type for initial value
const result = items.reduce((acc, item) => {
    acc.push(item.name);  // TypeError: acc.push is not a function
    return acc;
}, {});  // Should be []
```

Good

```
// FIX: Match initial value to intended accumulator type
const result = items.reduce((acc, item) => {
    acc.push(item.name);
    return acc;
}, []);
```

16.8.4 Bug 4: Async in forEach

× Bad

```
// BUG: forEach doesn't wait for async
items.forEach(async (item) => {
    await saveItem(item);  // These run in parallel, not sequentially!
});
console.log('Done');  // Logs immediately, before saves complete
```

Good

```
// FIX 1: Use for...of for sequential
for (const item of items) {
    await saveItem(item);
}

// FIX 2: Use Promise.all for parallel
await Promise.all(items.map(item => saveItem(item)));
```

16.8.5 Bug 5: Filter with Async Predicate

× Bad

```
// BUG: filter doesn't work with async
const valid = await items.filter(async (item) => {
    return await isValid(item);  // Returns Promise, which is truthy!
});
// valid contains ALL items
```

Good

```
// FIX: Use Promise.all + filter
const validityChecks = await Promise.all(
    items.map(async (item) => ({
        item,
        valid: await isValid(item)
    }))
);
const valid = validityChecks
    .filter(({ valid }) => valid)
    .map(({ item }) => item);
```

16.9 Debug Utilities Library

debug-utils.js - A collection of HOF debugging tools

```
export const debug = {
    // Basic tap
    tap: (fn) => (value) => {
        fn(value);
        return value;
    },

    // Labeled logging
    log: (label) => (value) => {
        console.log(label, value);
        return value;
    },

    // Table logging for arrays
    table: (label) => (array) => {
        console.log(label);
        console.table(array);
        return array;
    },

    // Timing wrapper
    time: (label, fn) => (...args) => {
        console.time(label);
        const result = fn(...args);
        console.timeEnd(label);
        return result;
    },

    // Async timing
    timeAsync: (label, fn) => async (...args) => {
        console.time(label);
        const result = await fn(...args);
        console.timeEnd(label);
        return result;
    },

    // Trace calls
    trace: (name, fn) => (...args) => {
```

```
        console.log(`${name} called with:`, args);
        const result = fn(...args);
        console.log(`${name} returned:`, result);
        return result;
    },

    // Count calls
    count: (name, fn) => {
        let count = 0;
        const wrapped = (...args) => {
            count++;
            return fn(...args);
        };
        wrapped.getCount = () => count;
        wrapped.reset = () => count = 0;
        return wrapped;
    },

    // Assert in pipeline
    assert: (predicate, message) => (value) => {
        if (!predicate(value)) {
            console.error('Assertion failed:', message, value);
            throw new Error(`Assertion failed: ${message}`);
        }
        return value;
    },

    // Breakpoint (debugger statement)
    breakpoint: (label) => (value) => {
        console.log('Breakpoint:', label, value);
        debugger;  // Will pause if DevTools open
        return value;
    },

    // Snapshot for comparison
    snapshot: (storage, key) => (value) => {
        storage[key] = JSON.parse(JSON.stringify(value));
        return value;
    }
};
```

These debug utilities can be inserted into any pipeline without changing the data flow—they're transparent wrappers that log and return the original value.

Usage Examples:

```
import { debug } from './debug-utils';

const result = data
    .filter(isValid)
    .map(debug.log('after filter:'))
    .map(transform)
    .map(debug.assert(x => x.id != null, 'id should exist'))
```

```
    .reduce(combine, []);

// With timing
const timedProcess = debug.time('process', process);
const result = timedProcess(data);

// Count calls
const countedTransform = debug.count('transform', transform);
data.map(countedTransform);
console.log('Transform called:', countedTransform.getCount(), 'times');
```

16.10 Chapter 16 Practice Exercises

Exercise 16.1: Debug Pipeline

Create a debugging wrapper that shows input/output of each step:

Exercise

```
function createDebugPipeline(...fns) {
    // Your code here
    // Should log before and after each function
}

// Usage
const pipeline = createDebugPipeline(
    x => x + 1,
    x => x * 2,
    x => x.toString()
);

pipeline(5);
// Logs:
// Step 1: input=5, output=6
// Step 2: input=6, output=12
// Step 3: input=12, output="12"
```

Exercise 16.2: Test Suite

Write tests for this `retry` HOF:

Exercise

```
function retry(fn, maxAttempts = 3, delayMs = 1000) {
    return async (...args) => {
        let lastError;

        for (let attempt = 1; attempt <= maxAttempts; attempt++) {
            try {
                return await fn(...args);
            } catch (error) {
                lastError = error;
                if (attempt < maxAttempts) {
                    await new Promise(r => setTimeout(r, delayMs));
                }
            }
        }

        throw lastError;
    };
}

// Write tests covering:
// - Successful first attempt
// - Success after failures
// - All attempts fail
// - Delay between attempts
```

Exercise 16.3: Closure Debugger

Create a utility to inspect closure state:

Exercise

```
function withClosureInspector(factory) {
    // Your code here
    // Should return a modified factory that exposes closure variables
}

// Usage
const createCounterWithInspector = withClosureInspector(createCounter);
const counter = createCounterWithInspector();

counter.increment();
counter.increment();

console.log(counter._inspectClosure());
// { count: 2 }
```

Exercise 16.4: Performance Test

Create a benchmark utility for comparing HOF implementations:

Exercise

```
function benchmark(implementations, testData, iterations = 1000) {
    // Your code here
    // Should run each implementation and compare times
}

// Usage
benchmark({
    'for loop': (data) => {
        const result = [];
        for (let i = 0; i < data.length; i++) {
            if (data[i] > 0) result.push(data[i] * 2);
        }
        return result;
    },
    'filter+map': (data) => data.filter(x => x > 0).map(x => x * 2),
    'reduce': (data) => data.reduce((acc, x) =>
        x > 0 ? [...acc, x * 2] : acc, [])
}, testArray, 1000);

// Output:
// for loop: 15.2ms (fastest)
// filter+map: 18.7ms (+23%)
// reduce: 145.3ms (+856%) <- reduce with spread is slow!
```

16.11 Chapter 16 Summary

Chapter Summary

Technique	Use Case
tap	Inspect values without modifying pipeline
traced	Log function calls with arguments/results
timed	Measure function performance
snapshot	Capture intermediate state for comparison
assert	Validate assumptions in pipeline
Mock functions	Test that callbacks are called correctly
Fake timers	Test debounce/throttle behavior
Property testing	Verify invariants hold for all inputs

Key Insight

HOFs require visibility strategies—tap, trace, and test wrappers let you see inside the black boxes.

Chapter 17

Quick Reference & Decision Guides

A comprehensive reference for all HOF patterns covered in this guide.

17.1 Method Quick Reference

17.1.1 Array Methods

Method	Signature	Returns	Mutates?	Use When
`map`	`(fn) ⇒ array`	Array (same length)	No	Transform each element
`filter`	`(predicate) ⇒`	Array (0 to N)	No	Keep elements passing test
`reduce`	`(fn, init) ⇒`	Single value	No	Combine into one result
`find`	`(predicate) ⇒`	First match or undefined	No	Get first matching element
`findIndex`	`(predicate) ⇒`	Index or -1	No	Get position for update/delete
`some`	`(predicate) ⇒`	true/false	No	Check if any element matches
`every`	`(predicate) ⇒`	true/false	No	Check if all elements match
`flatMap`	`(fn) ⇒ array`	Flattened array	No	Transform and flatten
`forEach`	`(fn) ⇒ undefi`	undefined	No	Side effects only
`sort`	`(compareFn) ⇒`	Sorted array	**Yes**	Sort in place

17.1.2 Early Exit Methods

Method	Stops When	Returns
`find`	First match	The element
`findIndex`	First match	The index
`some`	First true	true
`every`	First false	false

17.2 Decision Trees

17.2.1 Which Array Method?

```
Need to... │ ├── Transform each element? │ └── Same number of outputs? → map │ └──
Different number? → flatMap │ ├── Remove some elements? │ └── filter │ ├── Combine
into single value? │ └── reduce │ ├── Find something? │ ├── First match? → find │
├── Position? → findIndex │ ├── Does any match? → some │ └── Do all match? → every │
├── Check existence? │ ├── Primitive value? → includes │ └── By condition? → some │
└── Just iterate (side effects)? └── forEach (but consider for...of)
```

17.2.2 Which Composition Pattern?

```
Building a pipeline? │ ├── Left-to-right (data flow)? → pipe │ └── Data first,
read naturally │ ├── Right-to-left (mathematical)? → compose │ └── f(g(x)) style
│ ├── Object with methods? → Method chaining │ └── builder.setA().setB().build() │
├── Middleware pattern? → Compose with next │ └── Each wraps the next │ └── Async
operations? → Async pipe └── await between steps
```

17.2.3 Which Wrapper Pattern?

```
Need to add behavior? │ ├── Cache results? → Memoize │ └── Pure function, expensive
computation │ ├── Limit call frequency? │ ├── Wait for pause? → Debounce │ │ └──
Search input, resize handler │ └── Max calls per time? → Throttle │ └── Scroll
handler, API limits │ ├── Ensure single execution? → Once │ └── Initialization,
connection setup │ ├── Add logging/timing? → Traced │ └── Debug, monitoring │ ├──
Add error handling? → WithErrorHandler │ └── Fallbacks, logging │ └── Add retries? →
Retry └── Network requests, transient failures
```

17.3 Pattern Quick Reference

17.3.1 Factory Pattern

```
function createX(config) {
    // Private state via closure
    let state = initialState;

    // Return public interface
    return {
        method() { /* uses config and state */ }
    };
}

const instance = createX({ option: value });
```

17.3.2 Memoization

```
function memoize(fn) {
    const cache = new Map();
    return (...args) => {
        const key = JSON.stringify(args);
        if (!cache.has(key)) {
            cache.set(key, fn(...args));
        }
        return cache.get(key);
    };
}
```

17.3.3 Debounce

```
function debounce(fn, ms) {
    let timeoutId;
    return (...args) => {
        clearTimeout(timeoutId);
        timeoutId = setTimeout(() => fn(...args), ms);
    };
}
```

17.3.4 Throttle

```
function throttle(fn, ms) {
    let lastCall = 0;
    return (...args) => {
        const now = Date.now();
        if (now - lastCall >= ms) {
            lastCall = now;
            return fn(...args);
        }
    };
}
```

17.3.5 Pipe

```
const pipe = (...fns) => (x) => fns.reduce((v, f) => f(v), x);
```

17.3.6 Compose

```
const compose = (...fns) => (x) => fns.reduceRight((v, f) => f(v), x);
```

17.3.7 Curry

```
const curry = (fn) => {
    const arity = fn.length;
    return function curried(...args) {
        if (args.length >= arity) return fn(...args);
        return (...more) => curried(...args, ...more);
    };
};
```

17.3.8 Partial Application

```
const partial = (fn, ...preset) => (...later) => fn(...preset, ...later);
```

17.4 Common Transformations

17.4.1 Array to Object (by key)

```
const byId = users.reduce((acc, user) => {
    acc[user.id] = user;
    return acc;
}, {});

// ES2019+
const byId = Object.fromEntries(users.map(u => [u.id, u]));
```

17.4.2 Group By

```
const grouped = items.reduce((acc, item) => {
    const key = item.category;
    (acc[key] = acc[key] || []).push(item);
    return acc;
}, {});

// ES2024+
const grouped = Object.groupBy(items, item => item.category);
```

17.4.3 Count Occurrences

```
const counts = items.reduce((acc, item) => {
    acc[item] = (acc[item] || 0) + 1;
    return acc;
}, {});
```

17.4.4 Unique Values

```
const unique = [...new Set(array)];

// By property
const uniqueByName = [...new Map(users.map(u => [u.name, u])).values()];
```

17.4.5 Flatten

```
// One level
const flat = arrays.flat();

// Deep
const deepFlat = arrays.flat(Infinity);

// With transform
const flatMapped = arrays.flatMap(arr => arr.map(transform));
```

17.4.6 Partition

```
const partition = (arr, predicate) =>
    arr.reduce(([pass, fail], item) =>
        predicate(item) ? [[...pass, item], fail] : [pass, [...fail, item]],
    [[], []]);

const [evens, odds] = partition(numbers, n => n % 2 === 0);
```

17.4.7 Zip

```
const zip = (a, b) => a.map((item, i) => [item, b[i]]);

const zipped = zip([1, 2, 3], ['a', 'b', 'c']);
// [[1, 'a'], [2, 'b'], [3, 'c']]
```

17.4.8 Chunk

```
const chunk = (arr, size) =>
    arr.reduce((acc, _, i) =>
        i % size === 0 ? [...acc, arr.slice(i, i + size)] : acc,
    []);

chunk([1, 2, 3, 4, 5], 2);  // [[1, 2], [3, 4], [5]]
```

17.5 Performance Quick Guide

Scenario	Recommendation
< 1,000 items	Use HOFs freely
1,000 - 10,000 items	HOFs OK, profile if slow
10,000 - 100,000 items	Consider single reduce or lazy
> 100,000 items	Use lazy evaluation or loops
Hot path (1000s calls/sec)	Benchmark, consider loops
Infinite/streaming data	Use generators/iterators

17.5.1 Optimization Checklist

SLOW

```
// 1. Combine filters
data.filter(a).filter(b).filter(c)

// 2. Filter before map
data.map(transform).filter(predicate)

// 3. Use early-exit methods
data.filter(predicate).length > 0
data.filter(predicate)[0]

// 4. Avoid reduce with spread O(n²)
items.reduce((acc, item) ⇒
    ({...acc, [item.id]: item}), {})
```

FAST

```
// 1. Single filter
data.filter(x ⇒ a(x) && b(x) &&
  ↪  c(x))

// 2. Filter first
data.filter(predicate).map(transform)

// 3. Use early-exit methods
data.some(predicate)
data.find(predicate)

// 4. Mutate accumulator O(n)
items.reduce((acc, item) ⇒ {
    acc[item.id] = item;
    return acc;
}, {})
```

17.6 Functional Error Handling

17.6.1 Either Pattern

```
const Right = (value) ⇒ ({
    map: (fn) ⇒ Right(fn(value)),
    flatMap: (fn) ⇒ fn(value),
    fold: (onLeft, onRight) ⇒ onRight(value),
    isRight: true
});

const Left = (error) ⇒ ({
    map: (fn) ⇒ Left(error),
    flatMap: (fn) ⇒ Left(error),
    fold: (onLeft, onRight) ⇒ onLeft(error),
    isLeft: true
});
```

```
// Usage
const divide = (a, b) => b === 0
    ? Left('Division by zero')
    : Right(a / b);

divide(10, 2)
    .map(x => x * 2)
    .fold(
        err => console.error(err),
        result => console.log(result)  // 10
    );
```

17.6.2 Maybe Pattern

```
const Maybe = {
    Just: (value) => ({
        map: (fn) => Maybe.Just(fn(value)),
        getOrElse: () => value,
        isJust: true
    }),
    Nothing: () => ({
        map: () => Maybe.Nothing(),
        getOrElse: (defaultValue) => defaultValue,
        isNothing: true
    }),
    fromNullable: (value) =>
        value == null ? Maybe.Nothing() : Maybe.Just(value)
};

// Safe property access
const getCity = (user) =>
    Maybe.fromNullable(user)
        .map(u => u.address)
        .map(a => a.city)
        .getOrElse('Unknown');
```

17.7 Lens Quick Reference

```
// Create lens
const lens = (getter, setter) => ({
    get: getter,
    set: setter,
    over: (fn) => (obj) => setter(fn(getter(obj)), obj)
});

// Property lens
const prop = (key) => lens(
    (obj) => obj[key],
    (value, obj) => ({ ...obj, [key]: value })
);

// Usage
const nameLens = prop('name');
```

```
nameLens.get({ name: 'Alice' });  // 'Alice'
nameLens.set('Bob', { name: 'Alice' });  // { name: 'Bob' }
nameLens.over(s => s.toUpperCase())({ name: 'Alice' });  // { name: 'ALICE' }

// Compose lenses
const addressLens = prop('address');
const cityLens = prop('city');
const userCityLens = {
    get: (user) => cityLens.get(addressLens.get(user)),
    set: (city, user) => addressLens.set(
        cityLens.set(city, addressLens.get(user)),
        user
    )
};
```

17.8 Async Patterns

17.8.1 Sequential Processing

```
// Process one at a time
async function processSequentially(items, asyncFn) {
    const results = [];
    for (const item of items) {
        results.push(await asyncFn(item));
    }
    return results;
}

// Or with reduce
const processSequentially = (items, asyncFn) =>
    items.reduce(async (accPromise, item) => {
        const acc = await accPromise;
        const result = await asyncFn(item);
        return [...acc, result];
    }, Promise.resolve([]));
```

17.8.2 Parallel Processing

```
// All at once
const results = await Promise.all(items.map(asyncFn));

// With concurrency limit
async function parallelLimit(items, asyncFn, limit) {
    const results = [];
    const executing = [];

    for (const item of items) {
        const p = asyncFn(item).then(result => {
            executing.splice(executing.indexOf(p), 1);
            return result;
        });

        results.push(p);
```

```
        executing.push(p);

        if (executing.length ≥ limit) {
            await Promise.race(executing);
        }
    }

    return Promise.all(results);
}
```

17.8.3 Async Pipe

```
const pipeAsync = (...fns) => (input) =>
    fns.reduce((chain, fn) => chain.then(fn), Promise.resolve(input));

// Usage
const result = await pipeAsync(
    fetchUser,
    validateUser,
    saveUser
)(userId);
```

17.9 React HOF Patterns

17.9.1 Higher-Order Components (HOC)

```
function withAuth(Component) {
    return function AuthenticatedComponent(props) {
        const { user, loading } = useAuth();

        if (loading) return <Spinner />;
        if (!user) return <Redirect to="/login" />;

        return <Component {...props} user={user} />;
    };
}

const ProtectedPage = withAuth(DashboardPage);
```

17.9.2 Custom Hooks with HOFs

```
function useMemoizedCallback(fn, deps) {
    const memoized = useMemo(() => memoize(fn), deps);
    return memoized;
}

function useDebounce(value, delay) {
    const [debouncedValue, setDebouncedValue] = useState(value);

    useEffect(() => {
        const timer = setTimeout(() => setDebouncedValue(value), delay);
```

```
        return () => clearTimeout(timer);
    }, [value, delay]);

    return debouncedValue;
}

function useThrottledCallback(callback, delay, deps) {
    const throttled = useMemo(() => throttle(callback, delay), deps);

    useEffect(() => {
        return () => throttled.cancel?.();
    }, [throttled]);

    return throttled;
}
```

17.9.3 Event Handler Factories

```
// Factory for field handlers
const useFormHandlers = (setForm) => ({
    handleChange: (field) => (e) => {
        setForm(prev => ({ ...prev, [field]: e.target.value }));
    },
    handleToggle: (field) => () => {
        setForm(prev => ({ ...prev, [field]: !prev[field] }));
    },
    handleReset: (field, defaultValue) => () => {
        setForm(prev => ({ ...prev, [field]: defaultValue }));
    }
});

// Usage
const handlers = useFormHandlers(setFormData);
<input onChange={handlers.handleChange('email')} />
<input type="checkbox" onChange={handlers.handleToggle('subscribe')} />
```

17.10 Common Mistakes Quick Reference

17.11 Library Quick Reference

17.12 ES2025+ Features

17.12.1 Iterator Helpers (Lazy)

```
// Lazy array operations
Iterator.from([1, 2, 3, 4, 5])
    .filter(n => n % 2 === 0)
    .map(n => n * 2)
    .take(2)
    .toArray();  // [4, 8]
```

Mistake	Problem	Fix
Mutating in map/filter	Modifies original data	Spread/clone objects
No return in reduce	Returns undefined	Always return accumulator
Wrong reduce initial	Type mismatch	Match initial to intended type
async in forEach	Doesn't wait	Use for...of or Promise.all
async filter predicate	Always truthy	Promise.all then filter
Spread in reduce loop	$O(n^2)$ performance	Mutate accumulator in reduce
Missing dependency in useCallback	Stale closure	Include all dependencies
var in loop closures	All share same value	Use let or IIFE

Library	Primary Use	Key Features
Ramda	FP utilities	Data-last, curried, immutable
Lodash/fp	FP utilities	Data-last, curried version of Lodash
RxJS	Reactive streams	Observables, operators, async
Effect-TS	Effect system	Typed effects, error handling, DI
Immer	Immutable updates	Mutable-style API, structural sharing
Sanctuary	Type-safe FP	Stricter types, Maybe/Either

17.12.2 Array.fromAsync

```
// Collect async iterator to array
const results = await Array.fromAsync(asyncGenerator());
```

17.12.3 Set Methods

```
// Set theory operations
setA.intersection(setB);
setA.union(setB);
setA.difference(setB);
setA.symmetricDifference(setB);
```

17.12.4 Object.groupBy

```
// Native grouping
const grouped = Object.groupBy(items, item => item.category);
```

17.13 Chapter 17 Summary

Chapter Summary

This reference provides quick access to:

Section	Contents
17.1	Array method signatures and behaviors
17.2	Decision trees for choosing methods
17.3	Pattern implementations (factory, memoize, etc.)
17.4	Common data transformations
17.5	Performance guidelines
17.6	Functional error handling
17.7	Lens operations
17.8	Async patterns
17.9	React HOF patterns
17.10	Common mistakes
17.11	Library comparison
17.12	Modern JavaScript features

Part V Summary: Mastery Achieved

You now have the tools for complete HOF mastery:

Topic	Key Takeaways
Debugging	tap, trace, timing, snapshots
Testing	Mock functions, fake timers, property tests
Quick Reference	Method signatures, decision trees
Patterns	Factory, memoize, debounce, throttle, pipe
Performance	When to optimize, how to measure
Error Handling	Either, Maybe, Result patterns
Modern JS	Iterator helpers, async patterns

Final Practice Project: Build Your Own Utility Library

Practice Project: Build Your Own Utility Library

Create a complete utility library using all the patterns from this guide. Your library should include: composition functions (`pipe`, `compose`), currying utilities, function modifiers (memoize, debounce, throttle), array utilities (groupBy, partition, chunk), object utilities, functional types (Maybe, Either, Result), lens utilities, and async helpers.

Implementation Requirements:

- Implement each function using the patterns learned throughout this guide
- Write comprehensive tests for each function
- Document with JSDoc or TypeScript types
- Include usage examples for each utility

fp-utils.js

```
// Core composition
export const pipe = (...fns) => (x) => fns.reduce((v, f) => f(v), x);
export const compose = (...fns) => (x) => fns.reduceRight((v, f) => f(v), x);

// Currying and partial application
export const curry = (fn) => { /* implementation */ };
export const partial = (fn, ...args) => { /* implementation */ };

// Function modifiers
export const memoize = (fn) => { /* implementation */ };
```

```
export const debounce = (fn, ms) => { /* implementation */ };
export const throttle = (fn, ms) => { /* implementation */ };
export const once = (fn) => { /* implementation */ };
export const retry = (fn, attempts, delay) => { /* implementation */ };

// Array utilities
export const groupBy = (arr, keyFn) => { /* implementation */ };
export const partition = (arr, predicate) => { /* implementation */ };
export const chunk = (arr, size) => { /* implementation */ };
export const unique = (arr) => { /* implementation */ };
export const zip = (...arrays) => { /* implementation */ };

// Object utilities
export const pick = (obj, keys) => { /* implementation */ };
export const omit = (obj, keys) => { /* implementation */ };
export const mapValues = (obj, fn) => { /* implementation */ };
export const mapKeys = (obj, fn) => { /* implementation */ };

// Functional types
export const Maybe = { /* implementation */ };
export const Either = { /* implementation */ };
export const Result = { /* implementation */ };

// Lens utilities
export const lens = (getter, setter) => { /* implementation */ };
export const prop = (key) => { /* implementation */ };
export const path = (...keys) => { /* implementation */ };

// Debug utilities
export const tap = (fn) => (x) => { fn(x); return x; };
export const trace = (label) => tap(x => console.log(label, x));

// Async utilities
export const pipeAsync = (...fns) => { /* implementation */ };
export const parallel = (fns) => { /* implementation */ };
export const sequential = (fns) => { /* implementation */ };

// Validation
export const createValidator = (rules) => { /* implementation */ };
```

Conclusion

You've completed a comprehensive journey through higher-order functions in JavaScript:

Key Insight

Part I: Foundations

- First-class functions and HOF basics
- The core three: map, filter, reduce
- Beyond basics: find, some, every, flatMap

Part II: Patterns & Techniques

- Function factories and closures

- Composition and pipelines
- Currying and partial application
- Converting imperative to functional

Part III: Real-World Web Development

- DOM and events
- React patterns
- State management
- API and async operations

Part IV: Advanced Patterns

- Performance optimization
- Functional error handling
- Lenses and optics
- Metaprogramming with Proxies

Part V: Mastery & Reference

- Debugging strategies
- Testing techniques
- Quick reference guides

Remember: The goal isn't to use HOFs everywhere, but to recognize when they make code clearer, more reusable, and more maintainable. Start simple, compose gradually, and always prioritize readability.

Happy coding!